



UTAMED

**UNIVERSIDAD TECNOLÓGICA
ATLÁNTICO-MEDITERRÁNEO**

Máster Universitario en Inteligencia Artificial

**Evaluación experimental de Recursive
Language Models para el análisis
automatizado de literatura científica**

Trabajo Fin de Máster presentado por:

Juan Antonio Jiménez Cobo

Especialidad del TFM:

Procesamiento del Lenguaje Natural

Director/a:

Pablo Manuel Berná Larrosa

Curso académico:

2025/2026



Declaración de originalidad

Don **Juan Antonio Jiménez Cobo**, estudiante del **Máster Universitario en Inteligencia Artificial** de la Universidad Tecnológica Atlántico-Mediterráneo, declara que el presente Trabajo Fin de Máster titulado *Evaluación experimental de Recursive Language Models para el análisis automatizado de literatura científica* es un trabajo original e inédito, elaborado de forma autónoma bajo la supervisión del director **Pablo Manuel Berná Larrosa**.

Asimismo, declara que todas las fuentes utilizadas han sido debidamente citadas y referenciadas conforme a la normativa APA 7.^a edición.

Fdo.: Juan Antonio Jiménez Cobo

En Granada, a Junio 2026



Resumen

A pesar del uso generalizado de los *large language models* (LLM), el procesamiento de contexto largo sigue siendo uno de sus principales obstáculos. Estudios recientes documentan el fenómeno *context rot*, mediante el cual el rendimiento de distintos modelos del estado del arte se degrada a medida que aumenta la longitud del contexto de entrada. En este trabajo se evalúa empíricamente el paradigma de los *recursive language models* (RLM), en el dominio del análisis de literatura científica, comparando su rendimiento, coste y latencia frente a LLM base y sistemas *retrieval-augmented generation* (RAG), empleando los modelos GPT-5.4 mini y Gemini 2.5 Flash. El marco de evaluación consta de un corpus de 55 artículos académicos divididos en cuatro subconjuntos de tamaño creciente, y un conjunto de 40 consultas divididas en tres niveles de complejidad: extracción factual, comparación y síntesis. Los resultados muestran que los LLM base superan al resto de sistemas cuando el contexto cabe en su ventana de contexto de entrada, mientras que los RLM exhiben un potencial considerable, limitado por sus fallos de interacción con el entorno REPL. Adicionalmente, se identificaron tres patrones de error sistemático de los RLM mediante un análisis de sus trayectorias de razonamiento.

Palabras clave: grandes modelos de lenguaje, modelos de lenguaje recursivos, procesamiento de contexto largo, retrieval-augmented generation, análisis de literatura científica.



Abstract

Despite the widespread use of large language models (LLMs), long-context processing remains one of their main challenges. Recent studies document the phenomenon of context rot, in which the performance of various state-of-the-art models degrades as the length of the input context increases. This paper empirically evaluates the recursive language models (RLMs) paradigm in the domain of scientific literature analysis, comparing its performance, cost, and latency against baseline LLMs and retrieval-augmented generation (RAG) systems, using the GPT-5.4 mini and Gemini 2.5 Flash models. The evaluation framework consists of a corpus of 55 academic articles divided into four subsets of increasing size, and a set of 40 queries divided into three levels of complexity: factual extraction, comparison, and synthesis. The results show that baseline LLMs outperform the other systems when the context fits within their input context window, while RLMs exhibit considerable potential, limited by their failures to interact with the REPL environment. Additionally, three patterns of systematic error in RLMs were identified through an analysis of their reasoning trajectories.

Keywords: large language models, recursive language models, long-context processing, retrieval-augmented generation, scientific literature analysis.



Índice de contenidos

Resumen	i
Abstract	ii
1 Introducción	1
1.1 Objetivos	3
1.2 Metodología y viabilidad	4
1.3 Estructura del trabajo	6
2 Fundamentos teóricos	7
2.1 Introducción a los Large Language Models	7
2.2 Arquitectura Transformer	9
2.2.1 Transformer original: mecanismo de atención	10
2.2.2 Evolución hacia arquitecturas decoder-only	16
2.2.3 Complejidad cuadrática del mecanismo de atención	20
2.3 Enfoques para contexto largo	21
2.3.1 Optimizaciones del mecanismo de atención	22
2.3.2 Generación Aumentada por Recuperación (RAG)	24
2.4 Limitaciones de LLM en el procesamiento de contexto largo	25
2.4.1 Sesgo posicional: degradación de rendimiento en posiciones centrales del contexto	26
2.4.2 Deterioro del rendimiento en contextos extensos (<i>Context Rot</i>)	28
3 Recursive Language Models	32
3.1 Motivación y paradigma	33



3.2	Arquitectura técnica	34
3.2.1	Inicialización del entorno REPL	35
3.2.2	Bucle de razonamiento	36
3.2.3	Condición de parada.	37
3.3	Evaluación experimental	38
3.3.1	Benchmarks	39
3.3.2	Métodos y baselines evaluados	40
3.3.3	Resultados	40
3.4	Limitaciones y trabajo futuro	42
4	Sistema de evaluación experimental	44
4.1	Corpus de artículos: Revisión Sistemática de Literatura	46
4.2	Construcción y preprocesamiento del corpus	50
4.2.1	Extracción del texto de los registros	50
4.2.2	Preprocesamiento del corpus	51
4.2.3	Subconjuntos del corpus	53
4.3	Consultas	55
4.4	Sistemas experimentales	56
4.4.1	Baseline LLM	57
4.4.2	RAG	58
4.4.3	RLM	59
4.5	Métricas de evaluación	60
5	Resultados	63
5.1	Rendimiento comparativo de los sistemas	63
5.2	Análisis de coste y latencia: frontera de Pareto	71
5.3	Análisis del comportamiento del RLM	74
5.3.1	Fallos en las ejecuciones del RLM	74



5.3.2	Experimento de ablación: ajuste del prompt raíz para mitigar los errores de interacción con el entorno REPL de RLM GPT-5.4 mini	79
6	Conclusiones	82
6.1	Cumplimiento de objetivos	84
6.2	Limitaciones y líneas de investigación futuras	85
	Bibliografía	89
A	Prompts de los sistemas experimentales y del juez LLM	99
A.1	Prompts de los sistemas experimentales	99
A.1.1	Prompt del sistema para LLM	99
A.1.2	Prompt raíz para RLM	100
A.2	Prompts del LLM juez	101
A.2.1	Prompt para las consultas de extracción	101
A.2.2	Prompt para las consultas de comparación y síntesis	103
B	Análisis de ejecuciones	105
B.1	Efecto del modelo generador en el rendimiento de RAG	105
B.2	Fallos en las trayectorias del RLM	109
B.2.1	Alucinaciones de RLM Gemini 2.5 Flash	109
B.2.2	Fallo de interacción de RLM GPT-5.4 mini con el entorno REPL	112



Índice de figuras

1	<i>Arquitectura del modelo Transformer original.</i>	12
2	<i>Mecanismos de atención escalada mediante producto escalar y atención multicabezal.</i>	15
3	<i>Arquitectura del modelo decoder-only.</i>	17
4	<i>Comparación entre las arquitecturas encoder-decoder y decoder-only para una tarea de traducción automática.</i>	18
5	<i>Atención completa, escasa con ventana deslizante, y atención global con ventana deslizante.</i>	23
6	<i>Degradación de la precisión en forma de U de GPT-3.5-Turbo en función de la posición del documento con información relevante dentro del corpus.</i>	27
7	<i>Precisión de GPT-3.5-Turbo y Claude 1.3 y exhaustividad del recuperador en función del número de documentos recuperados.</i>	28
8	<i>Degradación en el rendimiento de LLM en función de la longitud del contexto y la presencia de distractores.</i>	30
9	<i>Diagrama de trabajo de un RLM.</i>	35
10	<i>Comparación del rendimiento de GPT-5 frente a su RLM correspondiente en tareas de contexto largo.</i>	39
11	<i>Diagrama del pipeline experimental completo.</i>	45
12	<i>Diagrama de flujo PRISMA 2020. Versión para revisiones que incluyen búsquedas en bases de datos y otras fuentes.</i>	49
13	<i>Interfaz del visualizador de trayectorias RLM.</i>	62



14	<i>Variabilidad del rendimiento por sistema y tipo de consulta.</i>	67
15	<i>Puntuación media por modelo, sistema y tipo de consulta.</i>	68
16	<i>Evolución del rendimiento de cada sistema por nivel de consulta a lo largo de los distintos subconjuntos del corpus.</i>	70
17	<i>Coste medio por consulta en USD (izquierda) y latencia media por consulta en segundos (derecha) de cada sistema experimental.</i>	72
18	<i>Análisis calidad-coste mediante la frontera de Pareto</i>	73
19	<i>Distribución de puntuaciones por sistema</i>	75
20	<i>Análisis calidad-coste actualizado incluyendo el rendimiento potencial de RLM GPT-5.4 mini.</i>	79
21	<i>Comparación de la puntuación media de RLM GPT-5.4 mini tras el experimento de ablación del ajuste del prompt.</i>	81
22	<i>Comparación de las respuestas y puntuaciones de RAG GPT-5.4 mini y RAG Gemini 2.5 Flash en una consulta de síntesis.</i>	105
23	<i>Ejemplo de alucinación de RLM Gemini 2.5 Flash.</i>	109
24	<i>Error de interacción de RLM GPT-5.4 mini con el entorno REPL (Ejemplo 1)</i>	112
25	<i>Error de interacción de RLM GPT-5.4 mini con el entorno REPL (Ejemplo 2)</i>	114



Índice de tablas

1	<i>Comparación de rendimiento de distintos métodos sobre benchmarks de contexto largo para GPT-5.</i>	41
2	<i>Rendimiento del RLM-Qwen3-8B entrenado mediante fine-tuning frente al modelo base y el RLM sin fine-tuneear.</i>	43
3	<i>Criterios de inclusión y exclusión aplicadas en la fases de cribado de la RSL.</i>	48
4	<i>Estadísticas del corpus tras la extracción de texto.</i>	51
5	<i>Comparativa de las estadísticas del corpus antes y después del procesamiento. Los valores en gris representan la disminución porcentual tras el filtrado.</i>	53
6	<i>Número de registros y volumen total de cada subconjunto del corpus.</i>	54
7	<i>Configuraciones de los sistemas experimentales.</i>	60
8	<i>Resultados experimentales por sistema, modelo , tipo de consulta y subconjunto del corpus.</i>	64
9	<i>Comparación del rendimiento de RLM GPT-5.4 mini con y sin puntuaciones de 1, por tipo de consulta.</i>	78



1. Introducción

«Los límites de mi lenguaje son los límites de mi mundo.»

Ludwig Wittgenstein (1921)

El auge de los *large language models* (LLM) visto durante los últimos años ha ampliado de forma notable su espectro de aplicación, consolidándose como herramientas de propósito general capaces de abordar una gran cantidad de problemas diversos. Un ejemplo de esto es que, recientemente, modelos del estado del arte han contribuido a resolver problemas matemáticos abiertos que permanecían sin solución para la comunidad matemática (Epoch AI, 2026).

Sin embargo, la expansión de dominios en los que se emplean los LLM ha puesto de manifiesto ciertas limitaciones que restringen su rendimiento en determinadas tareas. Una de las áreas más estudiadas es el procesamiento de contexto largo, para la cual estos modelos presentan diversos problemas. En primer lugar, la arquitectura *transformer*, base estructural de los LLM, tiene complejidad $O(n^2)$ en tiempo y memoria con respecto a la longitud del *input*, lo que implica que una duplicación en la longitud del texto de entrada que recibe el modelo cuadruplica su coste computacional. Adicionalmente, estudios experimentales como el de Hong et al. (2025) han demostrado que el rendimiento de los LLM del estado del arte se degrada a medida que aumenta la longitud del contexto de entrada, situación que se agrava cuando el contenido textual del contexto posee una estructura coherente y está relacionado entre sí.

Una de las últimas propuestas para mejorar el rendimiento de los LLM en tareas de contexto largo son los *recursive language models* (RLM), desarrollada por A. L. Zhang et al. (2025), del *MIT Computer Science and Artificial Intelligence Laboratory*



(CSAIL). A diferencia de la mayoría de enfoques existentes, los cuales modifican la arquitectura del LLM o se valen de componentes externos, los RLM escalan en tiempo de inferencia, es decir, que el aumento de cómputo del modelo se realiza a la hora de resolver la tarea, y escala con su complejidad.

Los RLM siguen una intuición novedosa. En lugar de procesar todo el *prompt* y el contexto como lo haría un LLM tradicional, los RLM interactúan con un entorno de programación *Read-Eval-Print Loop* (REPL), el cual inicializa un bucle recursivo en el que el contexto queda guardado como una variable de texto externa al RLM. De esta forma, a lo largo de distintas iteraciones, el RLM puede escribir y ejecutar código para inspeccionar la variable del contexto y realizar sub-llamadas a sí mismo o modelos más pequeños para delegar tareas. Mediante esta implementación, el RLM puede diseñar una estrategia para construir la respuesta a lo largo de múltiples pasos sin tener que procesar el contexto completo.

Los resultados de A. L. Zhang et al. (2025) muestran cómo los RLM superan a los LLM base en múltiples *benchmarks* de contexto largo con costes similares. No obstante, no se explora su aplicación en dominios específicos, por lo que la motivación del presente trabajo consiste en aplicar los RLM a un dominio específico, el del análisis de literatura científica. Estudios de hace una década, como el de Bornmann y Mutz (2015), estimaron que el número de artículos científicos indexados se duplica aproximadamente cada 9 años, tendencia que se mantiene en la actualidad, en donde el número de artículos indexados en Scopus y Web of Science aumentó en un 47% entre 2016 y 2022 (Hanson et al., 2023). Estos datos demuestran una necesidad de herramientas automáticas y escalables que sean capaces de asistir en tareas de investigación, y es en este contexto donde los RLM pueden aportar valor.

Por tanto, en este trabajo se ha diseñado e implementado un sistema de evaluación experimental en donde se compara el rendimiento de los RLM frente a LLM base y la arquitectura de recuperación de información del estado del arte *retrieval-*



augmented generation (RAG; P. Lewis et al., 2020), la cual no fue explorada en el trabajo original de A. L. Zhang et al. (2025). El sistema cuenta con un *corpus* de 55 publicaciones académicas, un volumen similar a lo que podría procesar un investigador en tareas reales de análisis de literatura, y un conjunto de 40 consultas divididas en distintos niveles de complejidad: extracción factual, comparación y síntesis. Se emplean dos modelos de lenguaje diferentes para cada sistema, GPT-5.4 mini (OpenAI, 2026) y Gemini 2.5 Flash (Google DeepMind, 2025a).

El trabajo evalúa la precisión, coste y latencia de cada sistema para comparar el rendimiento de los RLM con el resto de alternativas, identificando escenarios en los que cada sistema ofrece ventajas. Adicionalmente, se realiza un análisis cualitativo de las trayectorias de razonamiento de los RLM, detectando comportamientos singulares o patrones de error sistemático que no fueron explorados por A. L. Zhang et al. (2025)

Los resultados obtenidos evidencian que los RLM muestran un potencial significativo en el análisis de literatura científica, aunque limitado por distintos patrones de fallo dependientes del modelo de lenguaje subyacente.

1.1 Objetivos

Este trabajo tiene como objetivo principal el diseño e implementación de un marco experimental reproducible para la evaluación empírica de RLM en tareas de análisis automatizado de literatura científica, comparando su precisión y coste computacional frente a LLM base y RAG en un corpus de artículos científicos de 55 registros. Se parte de la hipótesis de que los RLM muestran resultados superiores en tareas de procesamiento de contexto largo frente al resto de sistemas a medida que el volumen del corpus y la complejidad de las tareas aumentan.

Adicionalmente, se establecen los siguientes objetivos específicos:

- **01:** Presentar las principales limitaciones experimentales de los LLM del estado del arte en el procesamiento de contextos largos y fundamentar teóri-



camente el paradigma propuesto por los RLM.

- **02:** Diseñar un sistema de evaluación experimental con un corpus de 55 artículos académicos y un conjunto de 40 consultas segmentadas por nivel de complejidad, con respuestas de referencia anotadas manualmente.
- **03:** Implementar tres sistemas de procesamiento de contexto largo, LLM base, RAG y RLM, usando los LLM GPT-5.4 mini y Gemini 2.5 Flash para cada uno de los sistemas y la biblioteca de código abierto publicada por A. L. Zhang (2025) para la implementación del sistema RLM.
- **04:** Evaluar comparativamente el rendimiento ofrecido por los tres sistemas implementados en términos de precisión, coste y latencia y su evolución a medida que aumenta el volumen del corpus de entrada y la complejidad de la tarea.
- **05:** Caracterizar los escenarios en los que los RLM muestran ventajas o limitaciones frente a las alternativas evaluadas, identificando tanto los casos de uso que justifiquen su mayor complejidad arquitectónica como los patrones de fallo sistemático mediante el análisis de sus trayectorias de razonamiento.

1.2 Metodología y viabilidad

El trabajo adopta un enfoque de evaluación empírica para comparar el rendimiento de tres sistemas, LLM base, RAG y RLM, empleando un diseño experimental controlado. La evaluación se realiza sobre un corpus de 55 artículos de literatura científica acerca del comportamiento estratégico de LLM en entornos de teoría de juegos, el cual fue construido mediante una Revisión Sistemática de Literatura (RSL; Page et al., 2021). El corpus se divide en cuatro subconjuntos con un volumen de texto ascendente, con el fin de analizar el rendimiento de cada sistema de forma individual a medida que aumenta la longitud del contexto de entrada. Se define un conjunto de 40 consultas divididas en tres niveles de complejidad, extracción factual, comparación y síntesis, las cuales cuentan con sus propias res-



puestas de referencia anotadas manualmente como *ground-truth*. La metodología de evaluación corresponde al marco *LLM-as-judge* (Zheng et al., 2023), que emplea un LLM para asignar una puntuación en la escala 1-5, comparando las respuestas de cada sistema con las respuestas de referencia de las consultas.

La implementación de los sistemas experimentales es viable gracias a las herramientas *open source* y servicios de API comerciales. El sistema RLM se implementa mediante la biblioteca de código oficial publicada por A. L. Zhang (2025), el sistema RAG mediante LlamaIndex (J. Liu, 2022) junto con ChromaDB (Chroma, 2022) como almacén vectorial del corpus, y el LLM base mediante llamadas directas a la API. Para cada sistema se emplean dos modelos de lenguaje, GPT-5.4 mini (OpenAI, 2026) y Gemini 2.5 Flash (Google DeepMind, 2025a), los cuales son accesibles mediante las API de pago de OpenAI y Google. La ejecución de cada sistema se realiza dos veces para obtener una mayor robustez estadística en los resultados. Los experimentos se ejecutan en Google Colab con almacenamiento en Google Drive, cuyo entorno es suficiente para el volumen de ejecuciones del experimento actual. Adicionalmente, A. L. Zhang (2025) proporcionan un visualizador de trayectorias de razonamiento RLM, una aplicación web local desarrollada en Node.js que permite inspeccionar de forma interactiva las iteraciones del bucle recursivo de los RLM junto al código generado y las sub-llamadas realizadas por el modelo raíz a partir de los registros de las ejecuciones en formato `.jsonl`. El trabajo incluye también un repositorio público en Github, disponible en Jiménez Cobo (2026), que contiene el código fuente del sistema experimental, registros de ejecución y metadatos del corpus.

El volumen y diseño de la experimentación se ajustó a las restricciones presupuestarias. El número de registros del corpus, consultas y repeticiones corresponde a esta decisión, buscando un equilibrio entre la representatividad de los resultados y el coste de ejecución por sistema. La elección de modelos de lenguaje de gama media-alta, GPT-5.4 mini y Gemini 2.5 Flash en vez de modelos frontera del estado del arte también corresponde a esta justificación.



1.3 Estructura del trabajo

El trabajo se divide en dos partes. En la primera de ellas se presentan los fundamentos teóricos. El capítulo 2 incluye una definición de alto nivel de los LLM junto a las etapas iniciales del desarrollo de los modelos de lenguaje (sección 2.1), la arquitectura transformer de Vaswani et al. (2017) y el mecanismo de atención (sección 2.2). Adicionalmente, se describen distintos métodos para mejorar el rendimiento de los LLM en tareas de contexto largo (sección 2.3), así como limitaciones fundamentadas mediante evidencia empírica que demuestra que estos enfoques no son del todo efectivos para solventar los problemas de los LLM en estos contextos, motivando así el paradigma de los RLM. El capítulo 3 introduce la arquitectura y funcionamiento interno de los RLM basado en el trabajo de A. L. Zhang et al. (2025), recogiendo los principales resultados y observaciones que demuestran el rendimiento superior de los RLM frente a LLM base y enfoques alternativos en distintos benchmarks de contexto largo.

La segunda parte recoge la metodología del sistema de evaluación experimental que presenta este trabajo en el capítulo 4. El capítulo 5 exhibe el análisis de los resultados de la experimentación y las observaciones principales basadas en la evidencia empírica. El trabajo finaliza con el capítulo 6, que expone las conclusiones del trabajo y presenta hipótesis para líneas de investigación futuras que expandan el análisis realizado.



2. Fundamentos teóricos

En este capítulo se presentan los fundamentos teóricos de este trabajo en relación a los grandes modelos de lenguaje, o *large language models* (LLM). Se comienza realizando una introducción de alto nivel acerca del concepto de LLM y describiendo las etapas iniciales del desarrollo de los modelos de lenguaje (sección 2.1). Posteriormente, se profundiza en la arquitectura *transformer* de Vaswani et al. (2017) y el concepto del mecanismo de atención (sección 2.2), así como la evolución de la arquitectura *encoder-decoder* a la *decoder-only*. Se finaliza el capítulo exponiendo métodos diseñados para el procesamiento de contexto largo (sección 2.3) y evaluaciones empíricas que ponen de manifiesto las limitaciones inherentes de los LLM en tareas complejas de procesamiento de contexto largo, fundamentando así el paradigma de los *recursive language models* (RLM), que se desarrolla en el capítulo siguiente.

2.1 Introducción a los Large Language Models

El concepto de modelo de lenguaje se remonta a la década de 1940, cuando Claude Shannon, considerado como el padre de la teoría de la información, formalizó esta idea como un proceso estocástico (Shannon, 1951). Este concepto es en el que se fundamentan los grandes modelos de lenguaje, o *large language models* (LLM en adelante), cuya principal función es la de predecir el siguiente *token* dentro de una secuencia de texto usando el contexto previo.

Los primeros modelos de lenguaje empleaban técnicas estadísticas tales como los *n*-gramas para estimar la probabilidad de la siguiente palabra mediante conteos de frecuencia relativa (Jurafsky & Martin, 2024). La introducción de las redes neuronales recurrentes (RNN) (Elman, 1990) permitió capturar dependencias de largo



alcance en textos, gracias a su diseño para procesar datos de forma secuencial. Las redes *Long Short-Term Memory* (LSTM) (Hochreiter & Schmidhuber, 1997) y *Gated Recurrent Unit* (GRU) (Cho et al., 2014) consiguieron mejorar las capacidades de las RNN para modelar relaciones entre elementos distantes, pero no pudieron resolver algunas de sus limitaciones estructurales, como era la imposibilidad de paralelizar el entrenamiento, o el problema del desvanecimiento del gradiente (Bengio et al., 1994). Posteriormente, Vaswani et al. (2017) introdujeron la arquitectura *transformer*, que sentó las bases del desarrollo de los modelos de lenguaje. Sobre esta base, trabajos posteriores como los de Radford et al. (2018) y Brown et al. (2020) dieron lugar a los LLM del estado del arte a partir de la familia de modelos de lenguaje GPT.

Los LLM son modelos autoregresivos, es decir, emplean toda la secuencia de texto previa para predecir el siguiente token, incorporando el resultado generado como parte de la ventana de contexto para la predicción siguiente. Formalmente, dado un corpus de tokens $\mathcal{U} = \{u_1, \dots, u_n\}$, el objetivo del modelado del lenguaje consiste en maximizar la siguiente función de verosimilitud:

$$L(\mathcal{U}) = \sum_i \log P(u_i | u_{i-k}, \dots, u_{i-1}; \Theta) \quad (1)$$

donde k denota la ventana de contexto y Θ los parámetros del modelo. Las predicciones se realizan a partir de distribuciones de probabilidad sobre el vocabulario entero del corpus (Brown et al., 2020)¹. La clave principal del éxito de estos modelos está en su escala. A medida que aumentan el número de parámetros, volumen de datos y el cómputo de entrenamiento, los modelos adquieren un mejor rendimiento en tareas de comprensión y generación de texto (Kaplan et al., 2020). Además, estudios como el de Wei et al. (2022) documentan que este aumento de

¹Esta naturaleza autoregresiva se apreciaba con claridad en los primeros modelos de lenguaje de gran escala. Un ejemplo es GPT-2 (Radford et al., 2019), el cual era capaz de generar texto de forma fluida a partir de un fragmento inicial, pero mostraba grandes limitaciones para mantener la coherencia y cohesión en generaciones largas. GPT-3 (Brown et al., 2020) redujo de forma significativa este problema gracias al aumento de parámetros y datos de entrenamiento.



escala provoca que los modelos exhiban habilidades emergentes que no se observaban en versiones más pequeñas.

No obstante, el rendimiento de los LLM se degrada a medida que la longitud del contexto proporcionado para la tarea aumenta. Estos modelos presentan complejidad cuadrática con respecto a la longitud del contexto, convirtiendo el procesamiento de textos largos o colecciones de múltiples documentos en operaciones costosas o directamente inviables en algunos casos. Estas limitaciones no se restringen al ámbito computacional. Recientemente, Hong et al. (2025) han documentado el fenómeno denominado *context rot*, mediante el cual la capacidad de los LLM de recuperar información se degrada a medida que la longitud del prompt aumenta, incluso cuando disponen de ventanas de contexto teóricamente aptas para manejar tal volumen de texto. Esta degradación se vuelve más destacable cuando la información se ubica en posiciones intermedias del contexto (N. F. Liu et al., 2024). Ambas limitaciones surgen como consecuencia de su arquitectura subyacente, en la cual se profundizará durante la siguiente sección.

2.2 Arquitectura Transformer

Antes de la aparición de los modelos transformers, las secuencias de texto se procesaban principalmente mediante el uso de RNN (Elman, 1990) y sus variantes más avanzadas, las redes LSTM y GRU. Estas arquitecturas capturaban dependencias dentro de la secuencia de entrada, actualizando un estado oculto del paso actual empleando el estado oculto del paso anterior. Sin embargo, esto introducía dos problemas estructurales: la imposibilidad de paralelizar el entrenamiento de la red, y el problema conocido como desvanecimiento del gradiente (Bengio et al., 1994), el cual limitaba la capacidad de capturar dependencias de largo alcance.

La arquitectura transformer, propuesta por Vaswani et al. (2017), resolvió estas limitaciones empleando el mecanismo de atención, que permitió que el modelo pudiera seleccionar las partes más relevantes de la secuencia de entrada con independencia de la distancia entre ellas. Durante esta sección se explora el diseño



de la arquitectura transformer original junto al mecanismo de atención, además de mejoras posteriores que dieron lugar a la aparición de los LLM modernos.

2.2.1 Transformer original: mecanismo de atención

El trabajo original de Vaswani et al. (2017) propuso una arquitectura alternativa a las arquitecturas previas empleadas por las RNN, con el objetivo de solventar sus limitaciones a la hora de capturar dependencias de largo alcance. El principal cambio que los autores implementaron en la arquitectura transformer fue el mecanismo de atención, el cual pondera la importancia relativa de cada parte de la secuencia de entrada. Esto permite al modelo mantener relaciones entre secciones distintas de un texto incluso aunque estas se encuentren en posiciones distantes en el texto.

Para ello, se mantuvo la estructura *encoder-decoder* establecida por Sutskever et al. (2014) y Cho et al. (2014), una arquitectura diseñada para tareas de secuencia a secuencia (*seq2seq*) en la que, dada una secuencia de entrada $X = (x_1, \dots, x_n)$, el encoder genera una representación vectorial, o *embedding*, continua $z = (z_1, \dots, z_n)$, la cual captura el contenido y relaciones contextuales de la secuencia x . Dicha representación z es empleada por el decoder para generar la secuencia de salida del modelo $y = (y_1, \dots, y_m)$. En cada paso, la componente decoder del modelo es autoregresiva, lo que implica que la secuencia de salida del decoder se incluye como secuencia de entrada en el siguiente paso.

Encoder. En el trabajo original, el encoder está formado por 6 capas idénticas (véase Fig 1 izquierda). Cada capa contiene dos subcapas: un mecanismo de atención multi-cabeza, o *multi-head self-attention*², y una red *feed-forward* aplicada de forma independiente a cada posición de la secuencia. Esta última consiste en dos

²En este caso, se distingue el término *self-attention* ya que las matrices Q , K y V se obtienen a partir de la secuencia de entrada, de forma que cada posición atiende al resto de posiciones de la misma secuencia. Este tipo de atención se diferencia de la atención cruzada, o *cross-attention*, empleada en el decoder, donde la matriz Q proviene del decoder, mientras que K y V vienen de la salida final del encoder.

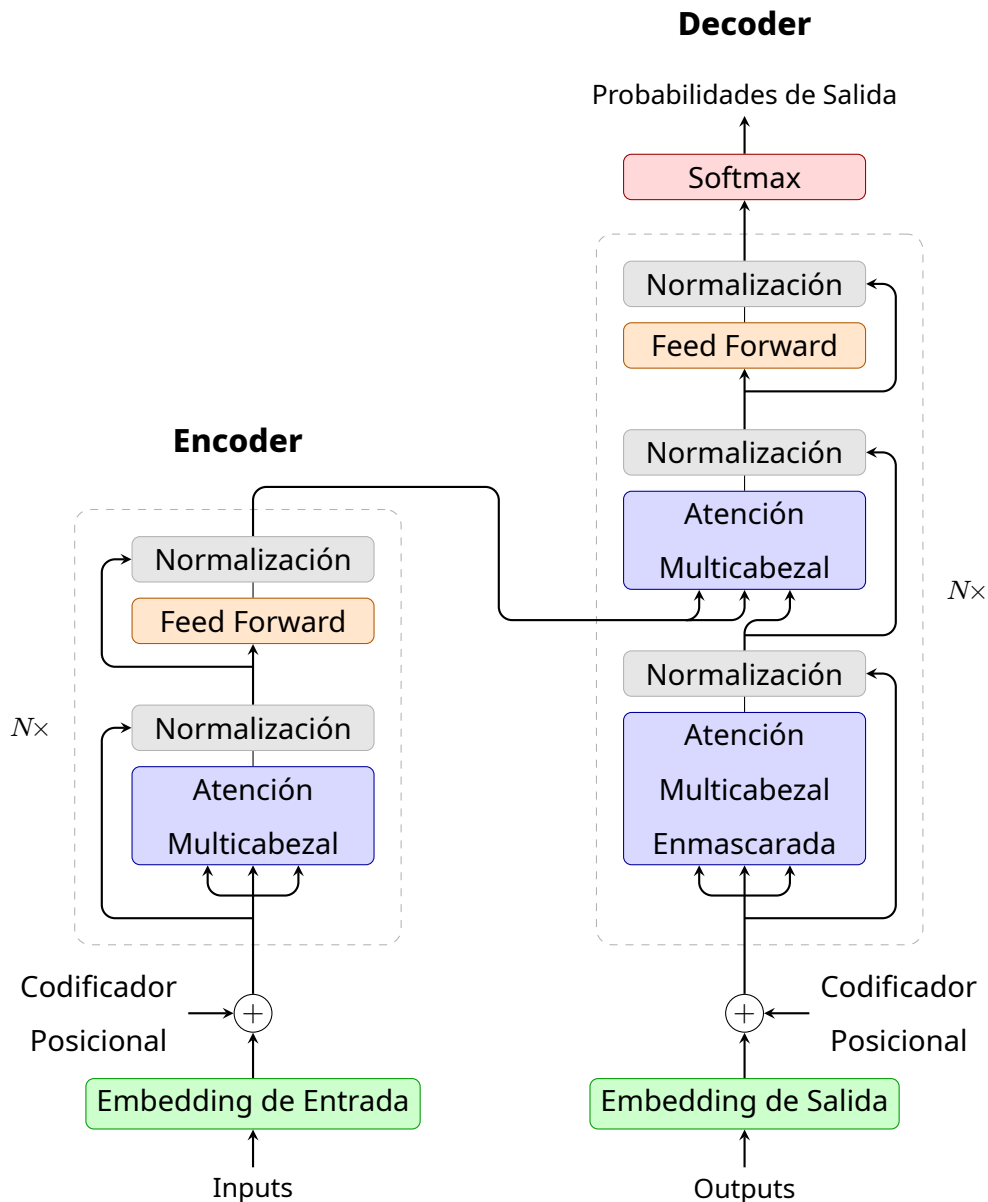


transformaciones lineales con una activación ReLU intermedia, $\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$. Tras cada una de estas dos subcapas, se aplica una conexión residual seguida de una normalización de capa (Ba et al., 2016), lo cual estabiliza el entrenamiento y facilita el flujo del gradiente en redes profundas.

Decoder. El decoder también está formado por 6 capas idénticas (véase Figura 1 derecha). Cada capa contiene las mismas dos subcapas del encoder más una tercera subcapa que aplica atención cruzada, *cross-attention*, en la que el decoder consulta la representación generada por el encoder. Adicionalmente, se aplica un enmascaramiento sobre las posiciones posteriores en la subcapa de atención multi-cabeza del decoder, lo cual garantiza que las predicciones de la posición i -ésima dependan exclusivamente de las salidas correspondientes a las posiciones previas.



Figura 1. Arquitectura del modelo Transformer original. En el trabajo original se emplearon $N = 6$ capas tanto en el encoder como en el decoder.



Fuente: Elaboración propia a partir de Vaswani et al. (2017).

Mecanismo de atención. Representa un componente fundamental de la arquitectura transformer y la principal innovación respecto a arquitecturas previas. Esto permitió al modelo capturar dependencias entre elementos de la entrada y ajustar dinámicamente su influencia en la salida del modelo. Los autores originales se



refieren a este mecanismo como atención escalada mediante producto escalar, o *scaled dot-product attention*³ (véase Figura 2).

Dada una secuencia de entrada $X \in \mathbb{R}^{n \times d}$, donde n es la longitud de la secuencia y d la dimensión de entrada, el mecanismo de atención computa tres matrices diferentes: la matriz de consulta (*query*) Q , la matriz de claves (*key*) K y la matriz de valores (*value*) V . Todas estas se obtienen como proyección lineal de la secuencia de entrada X sobre matrices de parámetros entrenables W^Q , W^K y W^V , respectivamente.

$$Q = XW^Q, W^Q \in \mathbb{R}^{d \times d_k} \quad (2)$$

$$K = XW^K, W^K \in \mathbb{R}^{d \times d_k} \quad (3)$$

$$V = XW^V, W^V \in \mathbb{R}^{d \times d_v} \quad (4)$$

donde d_k y d_v son las dimensiones de los vectores de consulta/clave y valor, respectivamente. La intuición detrás de esta formulación se puede entender como un sistema de recuperación de información. Q representa la información que busca cada posición, K la información que contiene cada posición, y V la información que se recupera efectivamente, es decir, las contribuciones de las claves que están fuertemente alineadas con las consultas se ponderarán con mayor magnitud. Esta alineación determina los pesos de la atención.

Las puntuaciones de atención se calculan mediante el producto escalar entre Q y K , usando la siguiente fórmula:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (5)$$

³El concepto de atención en redes neuronales fue introducido por Bahdanau et al. (2015) en el contexto de traducción automática, donde se empleó lo que denominó como mecanismo de atención aditiva para alinear dinámicamente la secuencia de entrada y la de salida. Posteriormente, Luong et al. (2015) introdujeron la atención por producto escalar y Vaswani et al. (2017) la generalizaron añadiendo un factor de escala $\frac{1}{\sqrt{d_k}}$ para estabilizar el entrenamiento en espacios de alta dimensión.



El factor de escala $\sqrt{d_k}$ evita que los productos escalares crezcan excesivamente en espacios de alta dimensión, provocando gradientes muy pequeños al aplicar la función softmax.

Atención multi-cabeza. Vaswani et al. (2017) extendieron el mecanismo de atención mediante la atención multi-cabeza (*multi-head attention*), permitiendo al modelo atender información desde distintos subespacios de representación conjuntamente. En vez de aplicar una única función de atención, la atención multi-capa aplica h instancias paralelas del mecanismo de atención, cada una con sus propias matrices de proyección $W^{Q_i}, W^{K_i}, W^{V_i}$. A cada una de estas instancias se le llama cabezal de atención, o *attention head*.

Así, para el i -ésimo cabezal de atención, las matrices de consulta, clave y valor se calculan de la siguiente forma:

$$\begin{aligned} Q_i &= XW^{Q_i}, W^{Q_i} \in \mathbb{R}^{d_v \times d_k} \\ K_i &= XW^{K_i}, W^{K_i} \in \mathbb{R}^{d_v \times d_k} \\ V_i &= XW^{V_i}, W^{V_i} \in \mathbb{R}^{d_v \times d_v} \end{aligned}$$

y la función de atención se aplica a cada cabezal en paralelo:

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i) = \text{softmax} \left(\frac{Q_i K_i^T}{\sqrt{d_k}} \right) V_i \quad (6)$$

Finalmente, las h salidas se concatenan y se proyectan linealmente de nuevo para obtener la salida final:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \quad (7)$$

donde $W^O \in \mathbb{R}^{hd_v \times d}$ es una matriz de proyección entrenable.

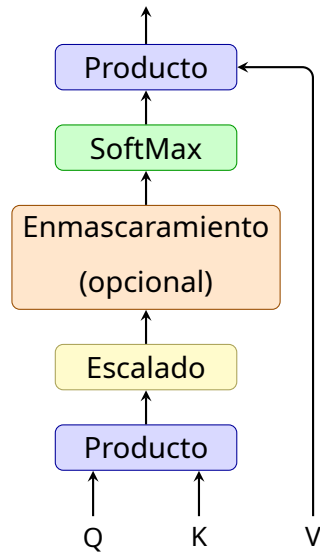
El mecanismo de atención multi-cabeza le permite al modelo capturar simultánea-



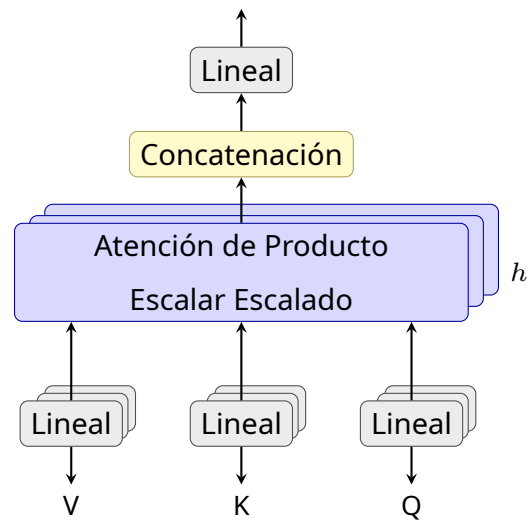
mente distintos tipos de dependencias y relaciones entre elementos de la secuencia de entrada, dado que cada cabezal puede enfocarse en diferentes aspectos del input. Por ejemplo, una cabeza puede especializarse en relaciones sintácticas, mientras que otra puede capturar relaciones semánticas de largo alcance.

Figura 2. Visualización del orden de operaciones en los mecanismos de atención escalada mediante producto escalar (izquierda) y atención multicabezal (derecha).

Atención de Producto Escalar Escalado



Multi-Head Attention



Fuente: Elaboración propia a partir de Vaswani et al. (2017).

Codificación posicional. Al procesar la secuencia de entrada en paralelo y no de forma secuencial, la arquitectura transformer pierde la noción de la posición de los tokens en la secuencia. La codificación posicional, o *positional encoding*, inyecta esa información posicional sumando un vector a cada embedding de entrada. Estos vectores tienen la misma dimensión que los embeddings, d , para que puedan ser sumados, y se calculan usando funciones seno y coseno de distintas frecuencias:

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d}) \quad (8)$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d}) \quad (9)$$



2.2.2 Evolución hacia arquitecturas decoder-only

A raíz de la arquitectura transformer de Vaswani et al. (2017), surgieron tres líneas principales de desarrollo. Los modelos *encoder-only*, representados por *Bi-directional Encoding Representations from Transformers* (BERT; Devlin et al. 2019), restringieron la arquitectura transformer al componente encoder y propusieron un mecanismo de atención bidireccional en el que cada posición atiende simultáneamente al contexto anterior y posterior dentro de la secuencia de entrada. Esto les permitió especializarse en tareas de comprensión y clasificación de texto. Otra línea fueron los modelos encoder-decoder, representados por *Text-to-Text Transfer Transformer* (T5; Raffel et al. 2020), los cuales mantuvieron la arquitectura original de Vaswani et al. (2017) para tareas seq2seq, como la traducción de texto. La tercera línea de desarrollo, y la más relevante para este trabajo, fue la de los modelos *decoder-only*, introducida por Radford et al. (2018)⁴⁵.

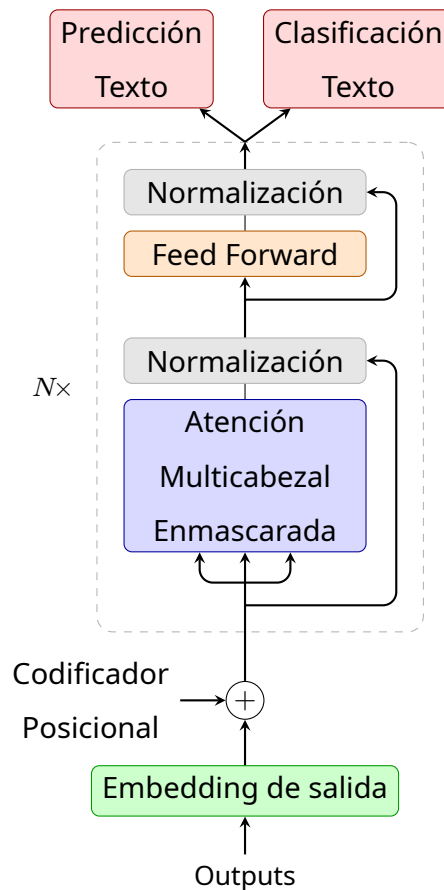
Radford et al. (2018) propusieron simplificar la arquitectura transformer a tan sólo el componente decoder (véase Figura 3), eliminando eliminando la subcapa de atención cruzada junto con su respectiva normalización de capa. Gracias a esta nueva arquitectura, los autores demostraron que los modelos de lenguaje podían obtener grandes mejoras en diferentes tareas de PLN, incluyendo clasificación de documentos, similitud semántica, preguntas de opción múltiple e implicación textual (*textual entailment*), a partir de un preentrenamiento no supervisado sobre grandes corpus de texto, aplicando un ajuste fino, o *fine-tuning*, mínimo sobre cada tarea.

⁴El uso exclusivo del componente decoder de la arquitectura transformer fue explorado previamente por P. J. Liu et al. (2018), quienes lo emplearon para generar artículos de Wikipedia resumiendo múltiples documentos fuente.

⁵El término *decoder-only* no fue empleado por Radford et al. (2018) en su trabajo original, sino que fue adoptado posteriormente por la comunidad investigadora para diferenciar a esta arquitectura de las encoder-decoder y encoder-only. Los autores se refieren a ella generalmente como arquitectura transformer, aunque en algunos pasajes la nombran por decoder transformer multicapa (*multi-layer transformer decoder*), aclarando que es una variante de la arquitectura transformer.



Figura 3. *Arquitectura del modelo decoder-only. En el trabajo original se aumentó el número de capas a $N = 12$ con respecto a la arquitectura encoder-decoder (figura 1).*

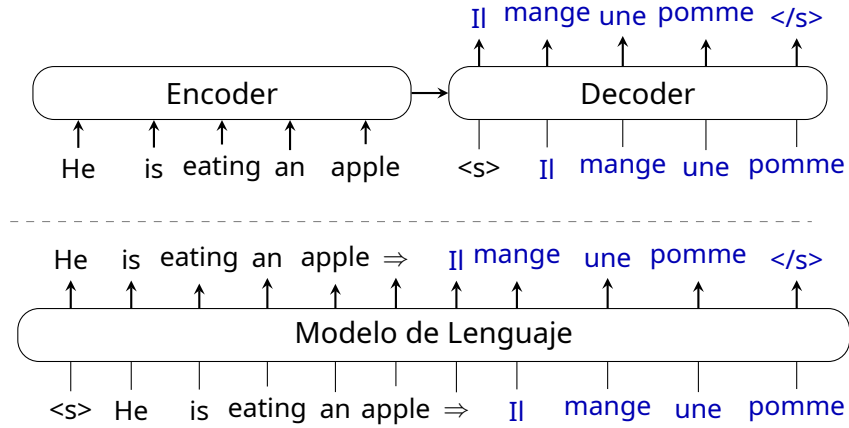


Fuente: Elaboración propia a partir de Radford et al. (2018).

La Figura 4 ejemplifica la diferencia entre el paradigma encoder-decoder y decoder-only en una tarea de traducción automática. En la arquitectura encoder-decoder, el componente decoder genera la traducción de la secuencia de entrada de forma autorregresiva empleando la representación generada por el encoder. En cambio, la arquitectura decoder-only unifica la secuencia de entrada y la objetivo, de forma que el modelo toma una única secuencia para generar la traducción de forma autorregresiva.



Figura 4. Comparación entre las arquitecturas encoder-decoder (arriba) y decoder-only (abajo) para una tarea de traducción automática.



Fuente: Elaboración propia a partir de Fu et al. (2023).

Preentrenamiento no supervisado. Puede entenderse como una forma de aprendizaje por transferencia (*transfer learning*). Los autores del trabajo original explican que el modelo es entrenado en grandes corpus de texto no etiquetado no para modificar el objetivo de las tareas del aprendizaje supervisado, sino para encontrar una buena inicialización en la que el modelo aprenda representaciones lingüísticas profundas, las cuales faciliten un posterior ajuste fino supervisado sobre cada tarea.

De forma análoga a la arquitectura transformer, el resultado final del modelo decoder-only durante el entrenamiento es una distribución de probabilidad sobre los tokens objetivo. Continuando con la notación empleada en la subsección 2.2.1, dada una secuencia de entrada $X = (x_1, \dots, x_n)$, el modelo calcula la distribución de probabilidad de la siguiente manera:

$$H_0 = XW_e + W_p \quad (10)$$

$$H_i = \text{decoder_block}(H_{i-1}), \quad \forall i \in [1, n] \quad (11)$$

$$P(X) = \text{softmax}(H_n W_e^T) \quad (12)$$



donde W_e es la matriz de embedding de tokens, W_p es la matriz de embedding posicional y n es el número de capas del modelo. El objetivo del preentrenamiento consiste en maximizar la función de verosimilitud definida en la ecuación 2.1 mediante la optimización de los parámetros $\Theta = \{W_e, W_p\}$ y los pesos de las respectivas capas del decoder a partir del descenso del gradiente estocástico (Bottou, 2010).

Ajuste fino supervisado. Consiste en adaptar los parámetros del modelo preentrenado a una tarea específica. Se parte de un conjunto de datos supervisado $\mathcal{C}$, donde cada instancia representa cada una de las tareas para ajustar el modelo, las cuales contienen una secuencia de entrada $(x^1, \dots, x^m)$ y una etiqueta y . Cada secuencia de entrada se procesa por el modelo preentrenado para generar la activación final del bloque del decoder, denotada por H_i^m , la cual finalmente se multiplica linealmente por una matriz de parámetros W_y para predecir y :

$$P(y|x^1, \dots, x^m) = \text{softmax}(H_i^m W_y) \quad (13)$$

lo cual equivale a maximizar la siguiente función de verosimilitud:

$$\tilde{L}(\mathcal{C}) = \sum_{(x,y)} P(y|x^1, \dots, x^m) \quad (14)$$

La aportación de la arquitectura decoder de Radford et al. (2018) estableció el paradigma *pretrain-finetune*, y dio lugar al modelo de lenguaje GPT-1, desarrollado por la empresa OpenAI. Este fue el primer paso para el desarrollo de los modelos de lenguaje generativos, lo que se conoce actualmente como LLM del estado del arte.

Tomando este trabajo como referencia, investigaciones posteriores aumentaron las capacidades de estos modelos decoder. Entre estos trabajos destacan los de Brown et al. (2020) con el modelo GPT-3, quienes aumentaron el número de pa-



rámetros del modelo a los 175.000 millones y documentaron el fenómeno conocido como aprendizaje contextual, o *in-context learning*, mediante el cual el modelo exhibía capacidad de aprender a realizar tareas para las cuales no había sido entrenado específicamente mediante entornos *few-shot*⁶. Otra investigación relevante fue la de Ouyang et al. (2022), quienes introdujeron el *Reinforcement Learning from Human Feedback* (RLHF), una técnica de ajuste fino basada en aprendizaje por refuerzo que permite alinear las respuestas de los modelos con preferencias humanas empleando una función de recompensa junto al algoritmo *proximal policy optimization* (PPO; Schulman et al. **schu**). Sobre esta base, OpenAI (2023) desarrolló el modelo GPT-4.

2.2.3 Complejidad cuadrática del mecanismo de atención

Como se describió en la sección 2.2, el mecanismo de atención se establece como el componente fundamental de la arquitectura transformer, dado que permite establecer una relación entre todas las posiciones de la secuencia de entrada en un número constante de operaciones secuenciales. Sin embargo, su propia formulación matemática genera un cuello de botella computacional al incrementar la longitud de la secuencia.

Volviendo a la ecuación del cálculo de las puntuaciones de atención (2.2.1), el producto de las matrices Q y K , ambas de dimensiones $n \times d_k$, requiere relacionar cada posición con el resto de posiciones de la secuencia, lo cual tiene un coste computacional de $O(n^2 d_k)$. El coste de la función softmax es de $O(n^2)$, mientras que el producto de la matriz resultante por la matriz V tiene un coste de $O(n^2 d_v)$, haciendo que el coste resultante sea de $O(n^2 d_k)$ en tiempo.

Estos cálculos fueron considerados por Vaswani et al. (2017), los cuales argumentaron que el mecanismo de atención suponía una mejora computacional con respecto a las RNN, cuya complejidad por capas era de $O(nd^2)$, siempre que la longi-

⁶En un entorno *few-shot*, el modelo recibe un número reducido de ejemplos en el prompt en formato entrada-salida de tareas para las cuales no ha sido entrenado directamente. Esta técnica no conlleva reentrenamiento del modelo o ajuste de sus parámetros.



tud de la secuencia fuera menor que la dimensión del modelo, esto es, $n < d$. No obstante, en casos en los que la longitud de dicha secuencia fuese considerablemente mayor a la dimensión del modelo, el coste computacional del mecanismo quedaría dominado por el término cuadrático $O(n^2)$. En términos prácticos, esto supone que una duplicación en la longitud de la secuencia de entrada cuadruplica los recursos computacionales requeridos por la arquitectura transformer, lo cual convierte el procesamiento de corpus de texto muy extensos en tareas significativamente costosas o directamente inviables dependiendo del escenario.

Cabe destacar que la complejidad cuadrática del mecanismo de atención no se debe a una ineficiencia del diseño original que pueda ser solventada mediante aproximaciones. Duman Keles et al. (2023) demostraron formalmente que la complejidad cuadrática constituye una cota inferior necesaria del mecanismo de atención bajo la Hipótesis Fuerte de Tiempo Exponencial (SETH⁷; Impagliazzo y Paturi 2001). Este resultado se extiende a versiones aproximadas del mecanismo de atención, demostrando que la barrera cuadrática se mantiene inalterable incluso si se asumen errores multiplicativos en el cálculo.

2.3 Enfoques para contexto largo

En respuesta a la complejidad cuadrática en tiempo y memoria del mecanismo de atención descrita durante la subsección 2.2.3, se abrieron dos líneas de investigación complementarias. La primera se centró en mejorar y optimizar el mecanismo de atención para reducir su coste computacional. Por el contrario, la segunda buscó un enfoque alternativo. En lugar de tratar de procesar toda la secuencia de entrada simultáneamente, se empleaba un sistema externo de recuperación de información para mejorar las capacidades generativas de los LLM. Esta sección recoge las aportaciones más relevantes de ambas líneas de investigación.

⁷La Hipótesis Fuerte de Tiempo Exponencial (SETH) es una conjetura de la teoría de la complejidad computacional que afirma que no existe ningún algoritmo capaz de resolver el problema de satisfacibilidad booleana (SAT) en tiempo inferior a $O(2^n)$, siendo n el número de variables. Pese a que no haya sido demostrada, es ampliamente aceptada por la comunidad investigadora y empleada como base para establecer cotas inferiores de complejidad en otros problemas.



La subsección 2.3.1 presenta aproximaciones del mecanismo de atención para reducir el coste computacional junto con optimizaciones de ingeniería del mecanismo para obtener complejidad subcuadrática en memoria. La subsección 2.3.2 introduce el concepto de generación aumentada por recuperación, o *retrieval-augmented generation*, (RAG; P. Lewis et al. 2020), un sistema que emplea bases de datos vectoriales para recuperar información de forma más efectiva en el contexto.

2.3.1 Optimizaciones del mecanismo de atención

Como se presentó durante la subsección 2.2.3, el cálculo de la atención genera un cuello de botella cuando la longitud de la entrada es mayor que la dimensión del modelo. Vaswani et al. (2017) anticiparon este inconveniente y propusieron restringir la atención a un entorno local de tamaño r en la secuencia de entrada centrada alrededor de la posición de salida correspondiente, lo cual permitiría reducir la complejidad a $O(n dr)$.

Esta idea de reducir la complejidad en tiempo y memoria del mecanismo de atención mediante entornos locales llevó a Beltagy et al. (2020) a introducir la arquitectura Longformer, una variante de la arquitectura transformer original que obviaba el cálculo exacto del mecanismo de atención para reducir la complejidad en tiempo y memoria a $O(n)$. Esto lo lograron sustituyendo la matriz de atención completa por un patrón de atención escasa (*sparse attention*) basado en una ventana de atención deslizante la cual restringe la atención de cada token a un entorno de tamaño fijo en la secuencia de entrada, de forma similar a lo que planteaban Vaswani et al. (2017). Siendo conscientes de que esto podía impedir el capturar dependencias de largo alcance en tareas complejas, incorporaron un mecanismo de atención global a partir de unos tokens específicos, como es el caso del token `[CLS]` en BERT (Devlin et al., 2019), los cuales podían atender a la secuencia entera.



Figura 5. Comparación entre los patrones de atención completa, atención escasa con ventana deslizante y atención global con ventana deslizante. Las cuadrículas en verde oscuro de la diagonal principal representan la secuencia de entrada, mientras que las cuadrículas en verde claro representan las posiciones de la secuencia a las que puede atender cada posición simultáneamente.



Fuente: Elaboración propia a partir de Beltagy et al. (2020)

Enfoques posteriores demostraron que es posible reducir la complejidad cuadrática en memoria calculando de forma exacta el mecanismo de atención. Rabe y Staats (2021) propusieron un algoritmo que reducía la complejidad en memoria a $O(\log n)$ (u $O(\sqrt{n})$ para aceleradores en implementaciones prácticas), mientras mantenía la complejidad en tiempo en $O(n^2)$. Esto lo implementaron usando un método similar a la función softmax perezosa (*lazy softmax*) (Jang et al., 2019), la cual desplaza la división entre el denominador de la función softmax al final de la operación, permitiendo así procesar las consultas, claves y valores en memoria constante.

Más allá de las aproximaciones matemáticas del mecanismo de atención, la optimizaciones de memoria de Rabe y Staats (2021) sirvieron a Dao et al. (2022) para desarrollar *FlashAttention*, una optimización de hardware en GPU que introduce el concepto de consciencia de entrada/salida (*IO awareness*). Los autores identificaron ineficiencias en la implementación del mecanismo de atención en los distintos niveles de memoria de las GPU: la memoria de alto ancho de banda (HBM) y la memoria estática de acceso aleatorio (SRAM), con menos capacidad que la HBM pero más rápida. FlashAttention consigue incrementar la velocidad en las operaciones



de ambas memorias, pero no reduce la complejidad temporal $O(n^2)$. Esta línea de desarrollo se ha continuado en los últimos años con *FlashAttention-2* (Dao, 2024) y *FlashAttention-3* (Shah et al., 2024).

2.3.2 Generación Aumentada por Recuperación (RAG)

Los resultados del estado del arte proporcionados por los grandes modelos de lenguaje preentrenados en tareas derivadas⁸ contrastaba con la capacidad limitada que presentaban en tareas que requerían un alto nivel de conocimiento en tareas de libro cerrado⁹, haciendo que investigadores se llegaran a cuestionar si los modelos realmente aprendían hechos concretos (Roberts et al., 2020).

Esta limitación en la memoria paramétrica de los modelos, la cual se define como el conocimiento que adquiere un modelo durante su entrenamiento, propició enfoques que combinaran este conocimiento con memoria no paramétrica, es decir, fuentes de almacenamiento externas cuya información puede ser recuperada por el modelo durante el entrenamiento. Los primeros en explorar estos modelos híbridos fueron ORQA (Lee et al., 2019) y REALM (Guu et al., 2020) con REALM, empleándolos para extraer respuestas en dominios abiertos.

No obstante, la aportación más relevante de este paradigma de modelos híbridos fue la generación aumentada por recuperación, *retrieval-augmented generation*, (RAG; P. Lewis et al. 2020), la cual fue diseñada específicamente para tareas de generación secuencia a secuencia.

En la estructura de RAG, se combinan dos componentes en un modelo probabilístico de extremo a extremo (*end-to-end*): el recuperador y el generador. El recuperador es *Dense Passage Retrieval* (DPR; Karpukhin et al. 2020), una arquitectura basada en un doble codificador. En primer lugar, el codificador de documentos los transforma en vectores densos y los almacena en un índice vectorial, el cual

⁸Las tareas derivadas (*down-stream*) son aquellas que emplean el modelo preentrenado de forma general para resolverlas.

⁹Una tarea de libro cerrado (*closed-book*) evalúa las capacidades de un modelo de lenguaje sin proporcionarles documentos de apoyo que les ayuden a resolver la tarea.



actúa como memoria no paramétrica del sistema. Al realizar consultas, otro codificador convierte la consulta en otro vector denso. La información relevante de los documentos es extraída mediante el algoritmo de Búsqueda de Máximo Producto Interno (MIPS; Shrivastava y Li 2014), un algoritmo que compara el vector denso de la consulta contra el índice vectorial y devuelve una distribución de probabilidad sobre los k documentos con mayor similitud semántica.

Una vez se recuperan los k documentos más relevantes, estos son transferidos al generador, el cual actúa como memoria paramétrica de RAG. P. Lewis et al. (2020) emplearon BART-large (M. Lewis et al., 2020), una arquitectura preentrenada encoder-decoder que concatena la consulta del usuario con los documentos recuperados para generar la salida de forma autorregresiva.

La secuencia final se genera tratando cada documento recuperado como una variable latente que es marginada para producir la distribución de probabilidad final. Para llevar esto a cabo, los autores proponen dos metodologías alternativas, que dependen de la finalidad de la tarea. El primero de ellos es RAG-Sequence, el modelo asume que la respuesta se encuentra en un único documento, calculando la probabilidad de que cada documento la contenga y marginándolas al final del proceso. Por el contrario, RAG-Token le permite al modelo generar la secuencia de salida a partir de múltiples documentos fuente. En este caso, las probabilidades se marginan en cada paso de la generación autorregresiva.

2.4 Limitaciones de LLM en el procesamiento de contexto largo

A pesar del gran desarrollo de los modelos de lenguaje descrito en las anteriores secciones, la arquitectura transformer presenta ciertas limitaciones que se evidencian en tareas de contexto largo. A lo largo de esta sección se exponen dos estudios empíricos que evidencian las limitaciones de los LLM, así como modelos con ventanas de contexto extendidas y sistemas RAG, al procesar contextos largos.

Las subsección 2.4.1 presenta el estudio de N. F. Liu et al. (2024), que demuestra que la posición de los fragmentos relevantes del contexto afecta al rendimiento de



estos sistemas a la hora de recuperar la información, exhibiendo una tendencia de degradación cuando dichos fragmentos se encuentran en posiciones intermedias del contexto. La subsección 2.4.2 introduce el fenómeno *context rot* de Hong et al. (2025), el cual evidencia la degradación del rendimiento de los LLM a medida que la longitud del contexto aumenta.

2.4.1 Sesgo posicional: degradación de rendimiento en posiciones centrales del contexto

Evaluaciones empíricas recientes han evidenciado que los LLM poseen limitaciones prácticas al procesar secuencias de entrada de longitud considerable o colecciones extensas de documentos, más allá de restricciones computacionales vistas en el anterior apartado. N. F. Liu et al. (2024) demostraron empíricamente que los LLM poseen sesgos posicionales a la hora de procesar ventanas de contexto amplias.

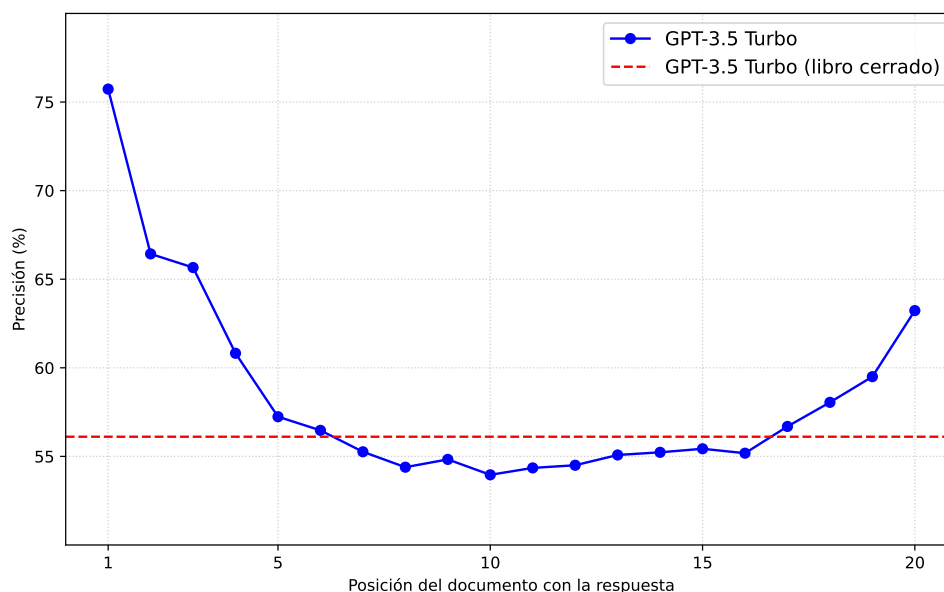
A través de experimentos que evaluaban respuestas a preguntas sobre colecciones de múltiples documentos, los autores cuantificaron el rendimiento del LLM dependiendo de la posición en la que se encontraba la información relevante dentro de la ventana de contexto. Como se observa en la figura 6¹⁰, los resultados mostraron un patrón en la curva de rendimiento del modelo en forma de U. Los modelos eran capaces de extraer información eficientemente cuando esta se ubicaba en las posiciones iniciales (sesgo de primacía, o *primacy bias*) o en las posiciones finales (sesgo de recencia, o *recency bias*) del contexto. Sin embargo, el rendimiento caía drásticamente cuando la información relevante se encontraba en posiciones intermedias del contexto, tanto que incluso la precisión en estos casos era inferior a la del modelo cuando no disponía de documentos en los que apoyarse para responder a las preguntas.¹¹

¹⁰Los datos numéricos de las figuras adaptadas de publicaciones originales fueron extraídos mediante digitalización semiautomática empleando la herramienta *WebPlotDigitizer* (Rohatgi, 2024).

¹¹A este tipo de pruebas de evaluación de modelos sin proporcionarles documentos de apoyo se las denomina como pruebas de libro cerrado (*closed-book*).



Figura 6. Curva de precisión en forma de U del modelo GPT-3.5-Turbo en función de la posición del documento con la información relevante dentro del corpus. El modelo extrae información eficientemente en las posiciones iniciales (sesgo de primacía) y finales (sesgo de recencia), pero muestra un peor rendimiento en posiciones intermedias, siendo superado incluso por el mismo modelo sin contexto adicional (libro cerrado).

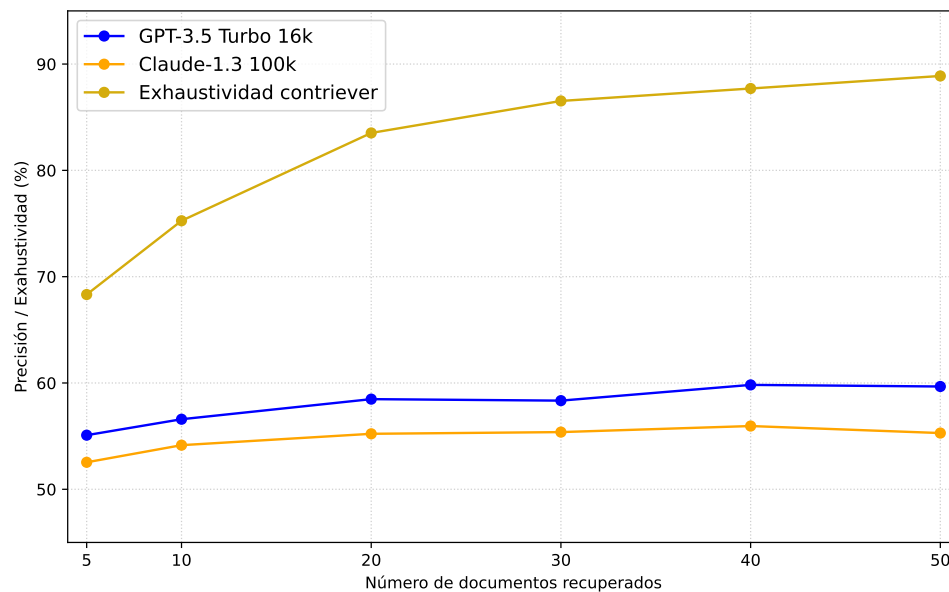


Fuente: N. F. Liu et al. (2024)

Esta falta de robustez se mantiene incluso en arquitecturas diseñadas específicamente para incrementar las ventanas de contexto, como los modelos GPT-3.5-Turbo (16K tokens), Claude-1.3 (100K tokens) y LongChat-13B (16K tokens), y persiste cuando se le proporcionan documentos externos al modelo para resolver las tareas. Tal y como se ilustra en la figura 7, el aumento del número de documentos proporcionados provoca una saturación en el rendimiento del modelo, de forma que los documentos adicionales contribuyen a mejoras marginales ($\sim 1.5\%$ para GPT-3.5-Turbo y $\sim 1\%$ en el caso de Claude 1.3). Esto demuestra que el incremento de ventanas de contexto en un LLM no le proporciona necesariamente una capacidad de razonamiento más robusta.



Figura 7. Precisión de los modelos GPT-3.5-Turbo (16K) y Claude 1.3 (100K) y exhaustividad del recuperador en función del número de documentos recuperados como contexto. El rendimiento de ambos modelos se satura antes que el sistema de recuperación alcance su límite, provocando mejoras mínimas en la precisión.



Fuente: N. F. Liu et al. (2024)

Esta saturación evidencia el cuello de botella que suponen los LLM incluso al incrementar sus ventanas de contexto, o emplear estructuras más complejas como RAG. En el marco de trabajo del análisis de literatura científica, estos resultados exponen la necesidad de contar con nuevos paradigmas que resuelvan estas limitaciones prácticas en tareas de contexto largo.

2.4.2 Deterioro del rendimiento en contextos extensos (*Context Rot*)

Otra evaluación empírica relevante que ha evidenciado limitaciones de los LLM al enfrentarse a tareas de procesamiento de contexto largo es la presentada por Hong et al. (2025), quienes introdujeron el concepto de deterioro del contexto, o *context rot*. A partir de experimentos donde la complejidad de la tarea se mantenía constante y únicamente variaba la longitud del contexto de entrada, los autores demostraron que el rendimiento de 18 modelos del estado del arte no se mantenía constante con la longitud del prompt, y su precisión se degradaba considerable-



mente en casos extremos.

Los modelos fueron evaluados a través de cuatro experimentos que extendían pruebas de rendimiento (*benchmarks*) clásicas de contexto largo. Una de ellas fue NIAH (*Needle in a Haystack*, haciendo referencia al dicho popular "*encontrar una aguja en un pajar*") (Kamradt, 2023), la cual evalúa la capacidad del modelo de recuperar un dato específico (la aguja) en un documento extenso con información irrelevante (el pajar). Los autores argumentan que esta prueba no refleja las capacidades reales del modelo al estar basada en coincidencia léxica¹², luego diseñaron tres experimentos que la extendía.

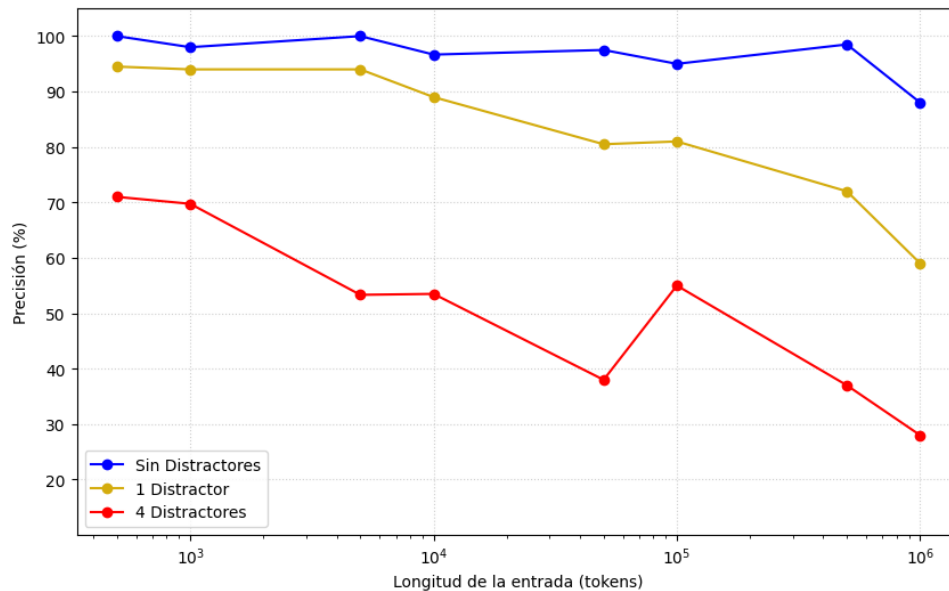
El primer experimento evaluó si la similitud entre la pregunta y la aguja afectaba a la precisión del modelo. Para llevarlo a cabo, introdujeron distractores en el contexto, los cuales son datos relacionados con la aguja pero que no responden a la pregunta. Los resultados de los experimentos evidenciaron que (1) la similitud entre pregunta y aguja afecta significativamente al rendimiento de los modelos, de forma que se degradará a mayor velocidad cuando la similitud entre pregunta y aguja sea baja, y (2) la inclusión de los distractores disminuía la precisión de los modelos (véase la figura 8).

Adicionalmente, se observó que los modelos de las distintas familias que fueron evaluadas mostraban un comportamiento distinto a la presencia de distractores. Los modelos de la familia Claude tendían a ser más conservadores, absteniéndose a dar una respuesta ante la presencia de múltiples distractores, mientras que los modelos de la familia GPT obtuvieron mayores tasas de alucinación, siendo más propensos a dar respuestas incorrectas de forma confiada. En el contexto de revisión bibliográfica, este hallazgo resulta crítico, puesto que en un corpus amplio de artículos, la mayor parte del texto actuará como distractor de la aguja, aumentando las tasas de alucinación y abstención del modelo.

¹²Para ejemplificarlo, si se pregunta "*¿Quién vive en Madrid?*" y en algún lugar del pajar se menciona explícitamente que "*María vive en Madrid*", le será fácil al modelo reconocer la aguja. Sin embargo, si la aguja pasa a ser "*María vive cerca de la Castellana*", entonces requeriría que el modelo realizara un razonamiento semántico.



Figura 8. Degradación en el rendimiento de LLM en función de la longitud del contexto y la presencia de distractores.



Fuente: Hong et al. (2025)

Los dos experimentos adicionales demostraron que el rendimiento aumentaba cuando el pájar contenía información no relacionada con la aguja y, contraintuitivamente, cuando el pájar no tenía una estructura lógica y coherencia, mostrando resultados mayores cuando las frases del pájar estaban reordenadas aleatoriamente sin un orden establecido. Ambos resultados son especialmente relevantes para la fase experimental de este trabajo, ya que los artículos científicos son documentos de texto estructurados de forma coherente. Adicionalmente, al contar con un corpus amplio de artículos del mismo dominio, la mayor parte de la información estará relacionada con las preguntas y agujas (especialmente en las consultas sobre extracción factual), lo cual aumenta el riesgo de degradación de rendimiento en los LLM tradicionales.

Finalmente, los autores emplearon la prueba de evaluación LongMemEval (D. Wu et al., 2025), la cual evalúa modelos de lenguaje en tareas de respuesta a preguntas conversacionales. La evaluación se realizó de forma sistemática bajo dos con-



diciones: una entrada (*input*) específica que contenga únicamente la información relevante, de forma que el modelo tenga que realizar el menor razonamiento posible, y una entrada completa incluyendo texto irrelevante. Los resultados mostraron que el rendimiento caía en entornos con entradas más completas que incluían texto irrelevante. Nuevamente, estos resultados conectan con la fase experimental del trabajo ya que, pese a que los artículos pertenezcan al mismo dominio, mucha de la información contenida en el corpus será de mayor o menor relevancia dependiendo de la consulta.

Los resultados de la evaluación empírica presentada por Hong et al. (2025) demuestran que la precisión de los LLM tradicionales se ve comprometida en tareas que requieran el procesamiento de contextos largos. Para este trabajo, la concatenación de 55 artículos científicos en un solo prompt para el modelo resulta una estrategia ineficaz, lo cual motiva la adopción de arquitecturas alternativas que puedan fragmentar los problemas y el contexto de forma recursiva.



3. Recursive Language Models

Como se ha desarrollado en el capítulo 2, el enfoque tradicional de la industria para aumentar la capacidad de procesamiento de los modelos de lenguaje consistía en modificar su arquitectura para que soportara una ventana de contexto mayor. No obstante, la evidencia empírica de la sección 2.4 demuestra que esta solución resulta ineficaz en ciertos casos.

El paradigma de los *Recursive Language Models* (RLM; A. L. Zhang et al., 2025) abandona esa idea y se fundamenta en el escalado por tiempo de inferencia (Merrill & Sabharwal, 2024). En lugar de aumentar el cómputo del modelo mediante una arquitectura más compleja, este paradigma propone que dicho aumento se realice en el momento de responder a la consulta, de forma que el escalado sea proporcional a la complejidad de la tarea. La característica principal de esta arquitectura es que el contenido del prompt no es procesado en su totalidad por el modelo, lo cual impide que la ventana de contexto se sature. Los autores reportan que este paradigma consigue procesar entradas hasta 100 veces mayores que la ventana de contexto del modelo e incluso superar el límite de tokens de salida, obteniendo resultados ampliamente mejores que los LLM tradicionales en tareas de contexto largo.

A lo largo de este capítulo, se profundiza en el paradigma de los RLM a partir de la publicación original de A. L. Zhang et al. (2025). Se presenta la motivación central del paradigma (sección 3.1), y se describe la arquitectura técnica de los RLM (sección 3.2), complementándola con el repositorio de GitHub de los autores (A. L. Zhang, 2025). Posteriormente, se presentan los experimentos principales en los que se compararon los rendimientos de RLM, LLM tradicionales y otros enfoques alternativos, junto a los hallazgos más relevantes (sección 3.3), y se finalizará mos-



trando algunas de las limitaciones identificadas por los autores y las líneas de trabajo futuro que propusieron (sección 3.4).

3.1 Motivación y paradigma

Los métodos de tiempo de inferencia previos a los RLM eran inadecuados para tareas de contexto largo, o no permitían generalizarlos a cualquier tarea. Entre ellos destacaban el paradigma de condensación o compactación de contexto (X. Wu et al., 2025), basado en generar una representación comprimida del contexto para responder la consulta, poco adecuado en tareas que requieren acceso denso del contexto proporcionado. Por otra parte, los enfoques de delegación recursiva (*self-delegation approach*), previamente explorados por Sun et al. (2025), permitían al LLM invocarse a sí mismo mediante llamadas para delegar tareas, pero generaban todo el contexto de forma autorregresiva antes de pasarlo a cada sub-LLM, lo cual introduce un claro cuello de botella al tratar con contextos masivos.

Ambos enfoques comparten la misma limitación estructural, y es que el contexto sigue siendo procesado en su totalidad por la ventana de contexto del LLM. Esto provoca las mismas restricciones que se desarrollaron durante la sección 2. Sin embargo, los RLM parten de una idea alternativa. En lugar de optimizar cómo se procesa el contexto dentro del modelo, lo tratan como un objeto externo al que el modelo puede acceder y descomponer de forma recursiva.

Para llevarlo a cabo, los RLM emplean un entorno de programación *Read-Eval-Print Loop* (REPL) en el que el contexto actúa como una variable de texto. De esta forma, el modelo no procesa el contexto completo para resolver la tarea, sino que escribe código para inspeccionar y descomponer el contexto en fragmentos, observa iterativamente los efectos secundarios de la ejecución, y realiza sub-llamadas a sí mismo o a modelos más pequeños sobre los diferentes fragmentos, construyendo la respuesta progresivamente a lo largo de las iteraciones.



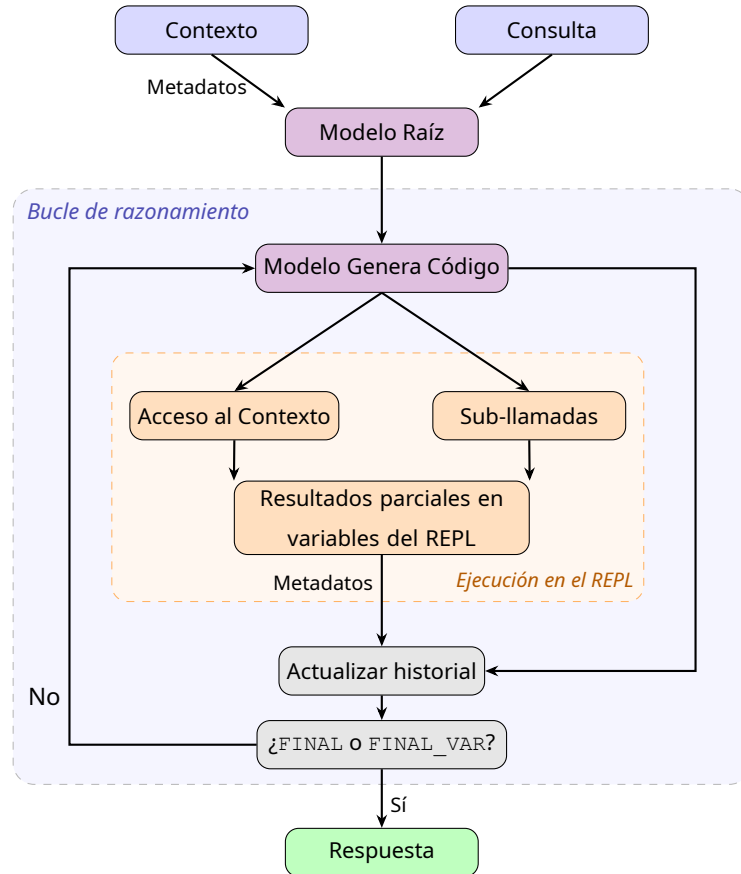
3.2 Arquitectura técnica

Los RLM no son un nuevo modelo creado desde cero, sino que representan una capa de orquestación, o *scaffold*, de tiempo de inferencia construida alrededor del modelo base. Los objetivos de esta arquitectura son (1) permitir al modelo base procesar entradas de longitud ilimitadas, (2) eliminar la restricción de longitud en la salida del modelo y (3) crear un horizonte semántico ilimitado, es decir, que el esfuerzo computacional del modelo sea proporcional a la complejidad y tamaño del problema.

El flujo de trabajo de los RLM se divide en tres partes. La Figura 9 muestra el proceso de inicio a fin.



Figura 9. Diagrama de flujo del paradigma RLM. El modelo raíz recibe el prompt y la consulta e interactúa con el entorno REPL mediante la generación de código, accediendo al contexto y realizando sub-llamadas recursivas hasta generar una respuesta final mediante *FINAL* o *FINAL_VAR*.



Fuente: Elaboración propia.

3.2.1 Inicialización del entorno REPL

El sistema recibe dos entradas: la consulta del usuario Q y el contexto asociado C . En ese momento se inicializa un entorno de programación REPL donde C queda almacenada como una variable, que puede ser de texto o una lista de documentos. Al mismo tiempo, el sistema registra en el entorno la función `llm_query`, la cual permite al LLM base invocar una sub-instancia de sí mismo, u otro modelo más pequeño, sobre cualquier fragmento de texto¹.

¹En el repositorio oficial de los autores (A. L. Zhang, 2025), también se define la función `rlm_query`, la cual invoca a una sub-instancia completa del RLM con su propio entorno REPL. No obstante,



El RLM recibe únicamente un mensaje inicial de tamaño constante que contiene Q y metadatos de C , incluyendo el tipo de variable, su longitud total y las longitudes de cada fragmento (en caso de que sea una lista de documentos). Esta diferencia entre lo que el modelo ve y lo que puede consultar es lo que permite al RLM procesar contextos de longitud arbitraria.

3.2.2 Bucle de razonamiento

Una vez inicializado el entorno REPL, el RLM entra en un bucle iterativo en el que el LLM base genera código en Python en cada paso, el cual se ejecuta en el REPL², y los metadatos de la salida se añaden a un historial para ser consultados en la siguiente iteración. El bucle se compone de tres pasos.

Inspección del contexto. El modelo accede al contenido del contexto mediante código simbólico, ya sea extrayendo texto por posición, por delimitadores estructurales (saltos de línea, cabeceras), localizando información relevante, o agregando fragmentos de texto, siempre y cuando la longitud sea asumible por el LLM base. El modelo decide qué estrategia aplicar en cada iteración a partir de Q y del estado acumulado en el historial.

Sub-llamadas recursivas. Cuando el RLM necesita procesar un fragmento de texto, invoca la función `llm_query` sobre él. Cada sub-llamada recibe un contexto acotado y devuelve una respuesta que se almacena en una variable del entorno REPL. Dichas respuestas no se inyectan en el historial del modelo, sino que permanecen en el entorno como variables hasta que el modelo accede a ellas explícitamente.

dicha función no es explorada en la publicación original, y se deja pendiente como trabajo futuro.

²La ejecución se realiza mediante la función `exec()` de Python, que permite ejecutar una cadena de texto como código dentro de un *namespace*. La implementación de referencia se puede consultar en `rlm/environments/local_repl.py` del repositorio de A. L. Zhang (2026).



Construcción de la solución. En tareas que requieren salidas largas, el modelo agrega resultados parciales en variables de entorno a lo largo de las iteraciones. Las variables son accesibles y modificables por el modelo base. Este mecanismo permite que la salida final del RLM supere el límite de tokens de salida del modelo base, puesto que la respuesta ya existe como una variable en el entorno antes de ser devuelta.

3.2.3 Condición de parada.

El bucle finaliza cuando el modelo asigna el resultado final a una variable de entorno mediante dos posibles mecanismos: `FINAL()` cuando la respuesta puede ser escrita como argumento en la ventana de salida del LLM base, y `FINAL_VAR()` cuando la respuesta ha sido construida en una variable del entorno a lo largo de las iteraciones.

El algoritmo 1 implementa el flujo de trabajo del RLM de principio a fin.



Algoritmo 1: Flujo de un RLM

Input: consulta Q y contexto C

Output: respuesta Y

```
1 state ← InitREPL( $C$ ) ;    // Al recibir  $C$  se inicializa el REPL
2 state ← AddFunction(state, llm_query) ; // Se añade la función
    llm_query
3 hist ← [ $Q$  || Metadata(state)] ; // Se añaden  $Q$  y los metadatos
    del entorno (incluido  $C$ ) al historial
4 while True do
5     code ← LLM(hist) ; // El LLM base genera un bloque de código
6     (state, stdout) ← exec(state, code) ;    // Ejecución del
        código
7     hist ← hist || [code] || Metadata(stdout) ; // Se actualiza
        historial
8     if state[FINAL] definido then
9         return state[FINAL] ; //  $Y$  devuelta por FINAL o FINAL_VAR
```

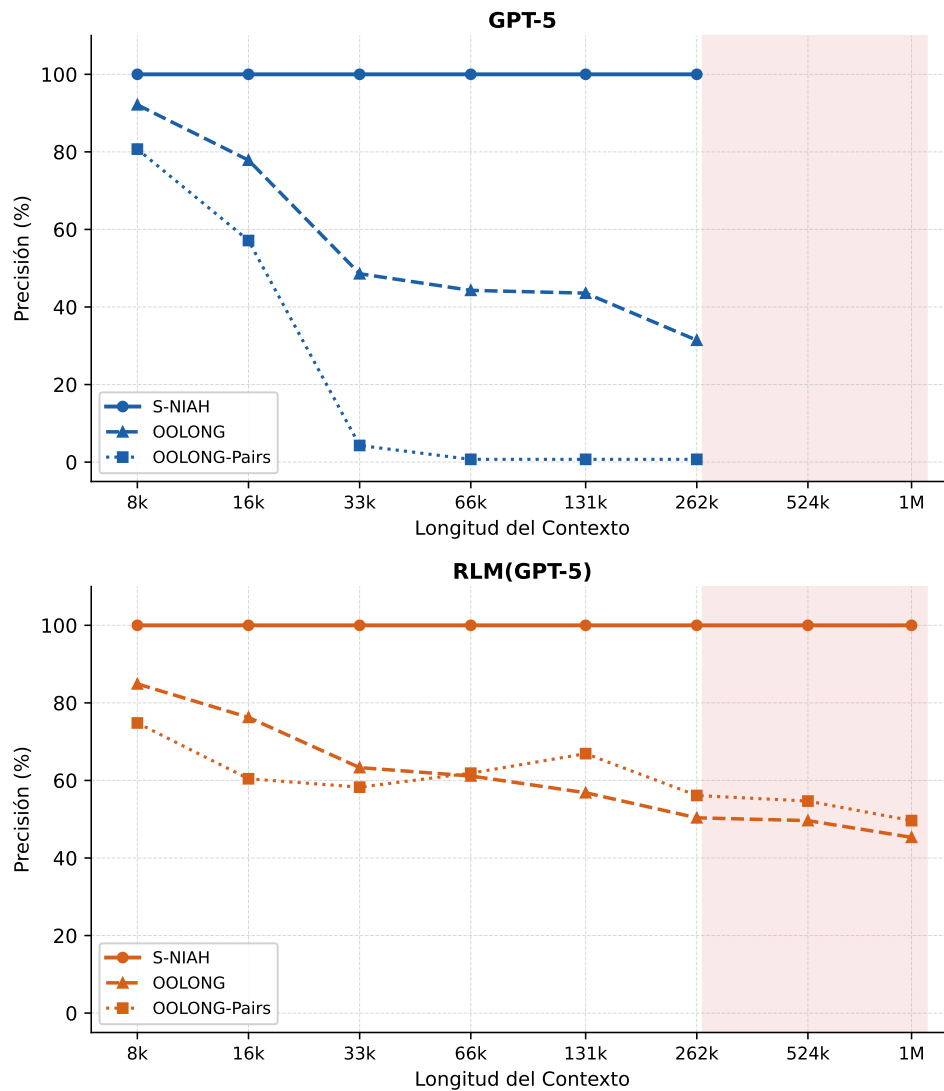
Fuente: Elaboración propia a partir de A. L. Zhang et al. (2025)

3.3 Evaluación experimental

Los RLM fueron evaluados frente a LLM base y baselines adicionales en numerosas tareas de contexto largo. Tal y como se observa en la figura 10, los resultados evidencian la superioridad de los RLM en la mayoría de benchmarks empleados, mostrando una alta consistencia ante el aumento del contexto a diferencia de los LLM base, cuya precisión se degrada considerablemente. Adicionalmente, se muestra cómo los RLM permiten cuadruplicar la cantidad de contexto procesado por el modelo en algunos casos.



Figura 10. Comparación del rendimiento de GPT-5 base frente a su RLM correspondiente en tres tareas de contexto largo. Las tareas con una longitud de input superior a 272 mil tokens no caben en la ventana de contexto del modelo base, mientras que el RLM puede procesarlas efectivamente.



Fuente: Elaboración propia a partir de A. L. Zhang et al. (2025)

3.3.1 Benchmarks

Se emplearon cinco benchmarks en los que la dificultad escala de forma diferente con respecto a la longitud del contexto.

BrowseComp-Plus (Chen et al., 2025). Consiste en una evaluación de preguntas



y respuestas que requiere unir y relacionar información procedente de mil documentos distintos.

OOLONG (Bertsch et al., 2025). Evalúa el razonamiento sobre contextos largos que requieren analizar fragmentos individualmente y agregar los análisis para responder preguntas. Su complejidad es $O(n)$ con respecto a la longitud del input al transformar y agregar todas las instancias del dataset.

OOLONG-Pairs. Variante de OOLONG creada por A. L. Zhang et al. (2025) mediante consultas que requieren agregar pares de fragmentos para construir la respuesta final. Escala cuadráticamente con respecto a la longitud del input.

LongBench-v2 CodeQA (Bai et al., 2024). Formado por tareas de comprensión de repositorios de código.

3.3.2 Métodos y baselines evaluados

Los autores evalúan dos LLM del estado del arte: GPT-5 (OpenAI, 2025), con razonamiento medio y parámetros por defecto, y Qwen3-Coder-480B (Yang et al., 2025). Para cada uno, se evalúa la capacidad de resolver a las tareas de cada benchmark del LLM base y el RLM usando cada LLM como modelo raíz.

Adicionalmente, se emplean dos métodos adicionales para cada LLM. CodeAct (Wang et al., 2024), un agente que puede ejecutar código dentro de un bucle ReAct³, y que además incluye el recuperador BM25 (Robertson & Zaragoza, 2009) para indexar el contenido de entrada, y un agente de resumen que actúa como método de condensación de contexto.

3.3.3 Resultados

La tabla 1 recoge algunos resultados de la evaluación experimental. Se observa cómo los RLM superan a los modelos base y agentes en todas las tareas. Los cos-

³Un bucle ReAct (*Reasoning and Action*) (Yao et al., 2023) constituye un patrón de razonamiento de modelos de lenguaje en el que el modelo razona sobre lo que necesita hacer, ejecuta una acción, observa los resultados, y vuelve a razonar sobre ese resultado para decidir el siguiente paso. Este proceso se realiza iterativamente hasta generar la respuesta final.



te de inferencia de los RLM son similares a los del LLM base, pero la varianza es mayor debido a la variabilidad en la longitud de las trayectorias en el bucle de razonamiento de los RLM.

Tabla 1. Comparación de rendimiento de distintos métodos sobre los benchmarks de contexto largo para GPT-5. En gris se muestra el coste medio de la API $\pm$ la desviación estándar de cada método en cada tarea. El asterisco (*) indica ejecuciones donde el método encontró límites de ventana de contexto. Los valores no nulos se redondean a al menos 0,1.

Método	CodeQA	BrowseComp+	OOLONG	OOLONG-Pairs
Modelo base	24,0* (\$0,13 $\pm$ \$0,07)	0,0* (N/A)	44,0 (\$0,14 $\pm$ \$0,02)	0,1 (\$0,16 $\pm$ \$0,10)
CodeAct	22,0* (\$0,06 $\pm$ \$0,08)	51,0 (\$0,71 $\pm$ \$1,20)	38,0 (\$0,61 $\pm$ \$1,06)	24,7 (\$0,75 $\pm$ \$0,43)
Ag. de resumen	58,0 (\$1,31 $\pm$ \$1,46)	70,5 (\$0,57 $\pm$ \$0,10)	46,0 (\$0,13 $\pm$ \$0,01)	0,1 (\$0,13 $\pm$ \$0,09)
RLM	62,0 (\$0,11 $\pm$ \$0,10)	91,3 (\$0,99 $\pm$ \$1,22)	56,5 (\$0,43 $\pm$ \$0,85)	58,0 (\$0,33 $\pm$ \$0,20)

Fuente: Adaptado de A. L. Zhang et al. (2025)

Los autores también incluyen patrones detectados en las trayectorias de razonamiento de los RLM. El más destacado es la diferencia en el comportamiento de cada modelo a la hora de gestionar las sub-llamadas recursivas. El anexo E.1 de A. L. Zhang et al. (2025) muestra la ineficiencia del RLM con Qwen3-Coder para resolver una tarea de OOLONG, realizando una sub-llamada por línea de contexto y resultando en más de mil llamadas totales. Esto implicó que los autores tuvieran que ajustar el prompt del modelo específicamente para minimizar el número de usos de la función `llm_query`, lo cual se recoge en el apéndice C.

Adicionalmente se destaca el uso del conocimiento previo del modelo para escribir código que filtre el contexto con el fin seleccionar las partes más relevantes según la consulta. En el apéndice E.1 de A. L. Zhang et al. (2025) se muestra cómo el RLM con GPT-5 empleó expresiones regulares para buscar términos concretos que acotaran la búsqueda antes de procesar ningún documento. Lo más destacado de este comportamiento es que algunos términos empleados ni siquiera aparecían en la consulta, luego fueron inferidos por el modelo a partir de su conocimiento



previo. Otro patrón relevante fue el uso del mecanismo de construcción iterativa iterativa de la respuesta mediante `FINAL_VAR` en tareas de OOLONG-Pairs. Los autores subrayan que algunas respuestas excedieron el límite de tokens de salida del modelo base.

3.4 Limitaciones y trabajo futuro

Pese al rendimiento superior ofrecido por los RLM frente al resto de alternativas, los autores señalan ciertas limitaciones de esta arquitectura. En particular, cuestionan la implementación de RLM con modelos base con capacidad de codificación insuficiente o un número de tokens de salida limitado. Se observó que esto último provocaba que los razonamientos consumieran la totalidad del límite de tokens de salida, interrumpiendo la ejecución.

Las limitaciones no se restringen al marco experimental, y es que los autores también subrayan ineficiencias en la implementación, destacando principalmente la gestión deficiente de algunos modelos en el número de sub-llamadas vista anteriormente. Mencionan además que en ciertos casos las llamadas bloquean el flujo de razonamiento del RLM, aumentando la latencia frente a LLM base. Por tanto, A. L. Zhang et al. (2025) proponen como trabajo futuro la implementación de sub-llamadas asíncronas⁴ y entornos aislados⁵.

Adicionalmente, dado que los RLM se implementan principalmente mediante andamiajes externos y *prompting*, los autores también plantean el desarrollo de RLM nativos entrenados a partir de trayectorias de otros RLM, abriendo la puerta a la siguiente generación de modelos.

Para demostrar que el paradigma de los RLM es entrenable, A. L. Zhang et al. (2025) presentan RLM-Qwen3-8B, el primer prototipo de RLM nativo, implementado a partir de un experimento de fine-tuning a pequeña escala. El fine-tuning

⁴Esto permitiría que el modelo siga ejecutando instrucciones mientras espera la respuesta de la sub-llamada.

⁵Al igual que se vio anteriormente con `rlm_query`, los entornos aislados están implementados en el repositorio de A. L. Zhang (2025), pero no han sido probados en la práctica.



se realizó sobre un modelo pequeño, Qwen3-8B, usando 1000 trayectorias filtradas de un modelo mayor, Qwen3-Coder-480B, actuando como RLM en tareas del benchmark LongBenchPro. De esta forma, el entrenamiento le permite al modelo aprender a programar dentro del entorno REPL y razonar sobre el uso de las subllamadas. Como se observa en la tabla 2, los resultados del RLM nativo superaron significativamente los resultados ofrecidos por el RLM base y con un coste inferior.

Tabla 2. Rendimiento del RLM-Qwen3-8B entrenado mediante *fine-tuning* frente al modelo base y el RLM sin *fine-tune*ar.

Método	CodeQA	BrowseComp+	OOLONG	OOLONG-Pairs
Modelo base	4,0* (\$0,01 ± \$0,00)	0,0* (N/A)	0,0* (N/A)	0,1 (\$0,01 ± \$0,00)
RLM	26,0 (\$0,04 ± \$0,13)	2,0 (\$0,03 ± \$0,06)	24,0 (\$0,19 ± \$0,26)	4,3 (\$0,05 ± \$0,05)
RLM (fine-tuning)	32,0 (\$0,02 ± \$0,02)	14,0 (\$0,01 ± \$0,03)	32,0 (\$0,04 ± \$0,09)	5,2 (\$0,02 ± \$0,02)

Fuente: Adaptado de A. L. Zhang et al. (2025)



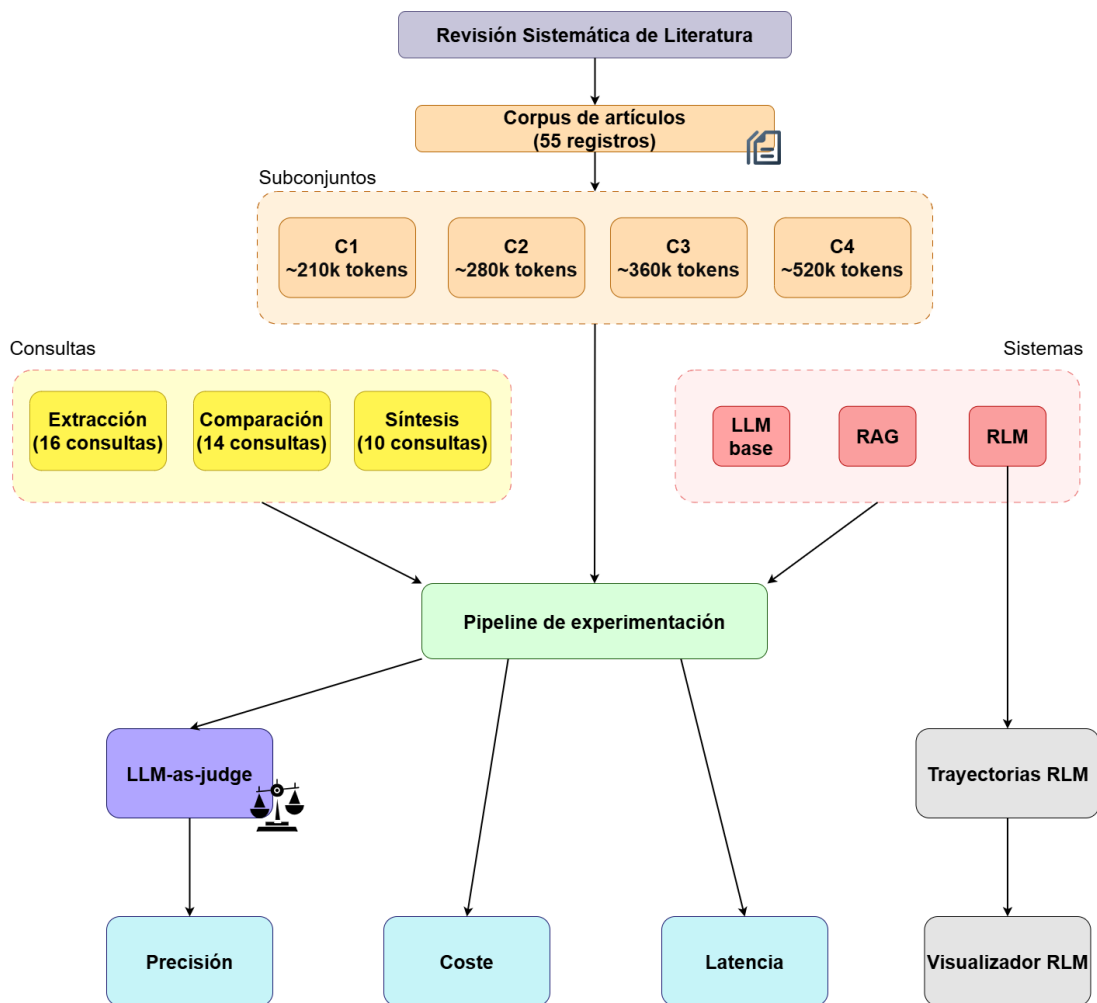
4. Sistema de evaluación experimental

En este capítulo se aborda el objetivo fundamental de este trabajo, el diseño e implementación de un sistema de evaluación experimental para comparar empíricamente el rendimiento del paradigma RLM frente a LLM base y RAG en el dominio del análisis de literatura científica. El sistema se fundamenta entorno a tres componentes principales: un corpus de 55 artículos científicos acerca del comportamiento estratégico de LLM en entornos de teoría de juegos, un conjunto de 40 consultas distribuidas en tres niveles de complejidad creciente, y tres sistemas experimentales evaluados mediante la metodología *LLM-as-judge* (Zheng et al., 2023).

La Figura 11 muestra el diagrama del *pipeline* experimental completo, El corpus fue construido mediante una Revisión Sistemática de Literatura (RSL; Page et al. 2021) (sección 4.1) y sometido a un proceso de extracción de texto y preprocesamiento (sección 4.2) para obtener el texto relevante de cada artículo en un formato estructurado. El corpus se divide en cuatro subconjuntos de volumen creciente (subsección 4.2.3), con el fin de analizar la evolución de cada sistema a medida que la longitud del contexto de entrada aumenta. Finalmente, se detallan las consultas (sección 4.3), los sistemas experimentales (sección 4.4) y las métricas de evaluación (sección 4.5), junto al visualizador de A. L. Zhang (2025) para realizar un análisis cualitativo de las trayectorias de razonamiento de los RLM.



Figura 11. Diagrama del pipeline experimental completo, desde la construcción del corpus mediante RSL hasta el análisis comparativo de los resultados. Las ejecuciones resultan de la combinación de 3 sistemas, 4 subconjuntos y 40 consultas.



Fuente: Elaboración propia.

Este diseño toma como referencia trabajos previos de evaluación de LLM sobre literatura científica en tareas de contexto largo, en particular CURIE (Cui et al., 2025) y AcademicEval (H. Zhang et al., 2025). Se adoptaron principios comunes como el uso de documentos académicos como contexto, división de consultas por nivel de complejidad y el empleo de respuestas de referencia para evaluar las respuestas de los sistemas.

Todos los recursos del sistema experimental, incluidos el código fuente, los meta-



datos de los registros del corpus, las consultas junto a las respuestas de referencia, y los registros de las ejecuciones, están disponibles de forma pública en el repositorio del proyecto (Jiménez Cobo, 2026).

4.1 Corpus de artículos: Revisión Sistemática de Literatura

El corpus empleado para la fase experimental del trabajo fue construido mediante una Revisión Sistemática de Literatura (RSL). Esta se llevó a cabo siguiendo el protocolo estándar PRISMA (*Preferred Reporting Items for Systematic Reviews and Meta-Analyses*; Page et al., 2021) con el objetivo de establecer un corpus de alrededor de 50 artículos para la fase experimental del trabajo. El proceso resultó en 55 registros incluidos tras incorporar artículos adicionales identificados mediante búsqueda en referencias y citaciones de los trabajos más relevantes del corpus. El procedimiento completo, incluyendo los archivos de datos de cada fase, está documentado en el repositorio público del proyecto (Jiménez Cobo, 2026).

Pregunta de investigación. Se formuló mediante la herramienta PICO (Richardson et al., 1995), que estructura la pregunta en cuatro componentes: población, intervención, comparación y resultado. En este caso, la población corresponde a LLM actuando como agentes en entornos de teoría de juegos; la intervención, a su participación directa como jugadores de juegos estratégicos; la comparación, a soluciones teóricas clásicas, jugadores humanos y otros modelos de referencia; y el resultado, al comportamiento estratégico mediante métricas cuantitativas. A partir de estos criterios, la pregunta de investigación quedó formulada de la siguiente forma: ¿Son capaces los LLM de exhibir comportamiento racional en juegos estratégicos, y cómo se compara dicho comportamiento frente a soluciones clásicas, jugadores humanos y/u otros modelos?

Bases de datos y cadena booleana. La búsqueda se realizó en dos bases de datos bibliográficas, Scopus y Web of Science, obteniendo 110 y 40 registros respectivamente. La búsqueda se realizó empleando una cadena de búsqueda booleana



con términos clave en relación a LLM, teoría de juegos y comportamiento estratégico. Adicionalmente, se incluyeron filtros para que los registros seleccionados fuesen artículos y comunicaciones a congresos (con revisión por pares) publicados entre 2022 y 2025 y redactados en inglés.

Criterios de inclusión y exclusión. La Tabla 3 muestra los criterios de inclusión y exclusión empleados en las distintas fases de cribado de PRISMA 2020¹. Para que un registro fuese seleccionado, este debía cumplir todos los criterios de inclusión, mientras que un criterio de exclusión satisfecho era suficiente para descartar un registro.

¹ Los criterios de inclusión y exclusión fueron refinados con el modelo de lenguaje Claude Sonnet 4.6 (Anthropic, 2026)



Tabla 3. *Criterios de inclusión y exclusión aplicadas en la fases de cribado de la RSL.*

Código	Criterio	Tipo
CI-1	Publicado entre 2022 y 2025	Inclusión
CI-2	Artículo de revista o comunicación a congreso con revisión por pares	Inclusión
CI-3	Redactado en inglés	Inclusión
CI-4	Involucra al menos a un LLM, o agente basado en LLM, como jugador en un juego estratégico	Inclusión
CI-5	Incluye comparación con soluciones clásicas, jugadores humanos y/u otros LLM o agentes basados en LLM	Inclusión
CI-6	Proporciona resultados cuantitativos sobre el comportamiento del LLM o agente basado en LLM	Inclusión
CE-1	No incluye ningún LLM (emplean agentes de aprendizaje por refuerzo u otros agentes)	Exclusión
CE-2	El marco de experimento es teórico, mientras que la fase experimental es secundaria o complementaria	Exclusión
CE-3	Revisión de literatura sin datos propios	Exclusión
CE-4	El juego estratégico no es el objeto central de estudio	Exclusión
CE-5	El texto completo no se encuentra disponible	Exclusión
CE-6	Preprints sin publicación en editorial reconocida o comunicación a congresos sin revisión por pares	Exclusión

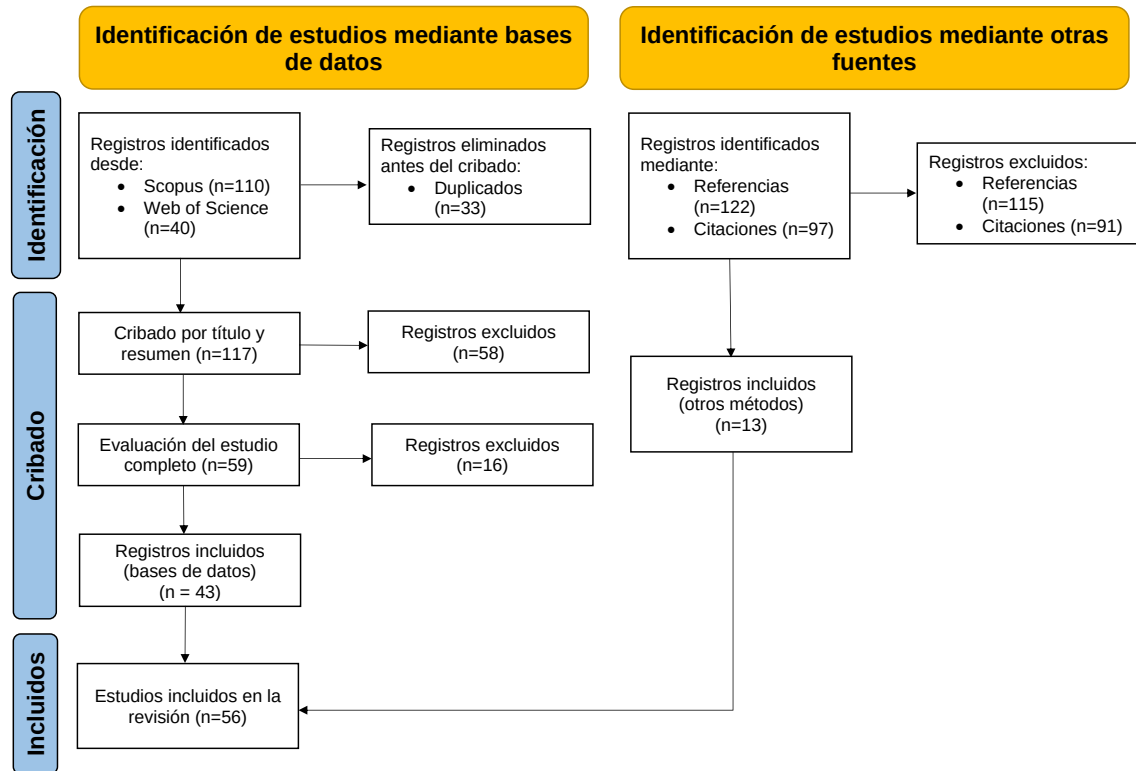
Fuente: Elaboración propia

Proceso de cribado. Los 150 registros obtenidos mediante las bases de datos fueron filtrados en varias fases a través de la plataforma Rayyan (Ouzzani et al., 2016). Esto incluyó excluir 33 registros duplicados, 58 tras analizar título y resumen, y 16 tras analizar el texto completo², resultando en un total de 43 registros incluidos. La figura 12 muestra el flujo completo y los registros incluidos y excluidos tras cada fase.

²Se empleó el modelo de lenguaje Claude Sonnet 4.6 (Anthropic, 2026) como revisor adicional en algunas decisiones.



Figura 12. Diagrama de flujo PRISMA 2020. Versión para revisiones que incluyen búsquedas en bases de datos y otras fuentes.



Fuente: Adaptado de Page et al. (2021).

Búsqueda en fuentes adicionales. Con el fin de enriquecer el corpus, se aplicaron los mismos criterios de inclusión y exclusión a las citas y referencias de los siete registros más citados y con mayor relevancia temática de los 43 registros incluidos tras el cribado. La búsqueda resultó en 122 referencias y 97 citaciones analizadas, de las cuales se incluyeron 13 en el corpus final.

Corpus final. De los registros identificados en bases de datos, se incluyeron 43 registros, mientras que de la búsqueda adicional en referencias y citas se incluyeron 13, dando un total de 55 registros en el corpus final³.

³Los metadatos de los registros incluidos en el corpus final se pueden consultar en el archivo `slr/final/corpus_final_20260415.csv` en el repositorio público del proyecto (Jiménez Cobo, 2026).



4.2 Construcción y preprocesamiento del corpus

Para obtener un corpus textual homogéneo que pudiese ser empleado por los distintos sistemas en la fase experimental, se desarrolló un *pipeline* de dos etapas para extraer el texto a partir de los archivos PDF de los 55 artículos obtenidos en la subsección 4.1, y preprocesarlos para eliminar el contenido no relevante para el análisis. Ambas etapas fueron implementadas en Python y ejecutadas en Google Colab⁴. Posteriormente, se crearon cuatro subconjuntos con los registros del corpus de tamaño creciente, partiendo desde 20 registros hasta los 55 registros totales.

4.2.1 Extracción del texto de los registros

La primera etapa del pipeline consistió en extraer el contenido textual de los 55 registros del corpus en formato PDF. Para ello, se empleó la librería de Python PyMuPDF (Artifex Software, 2024). En primera instancia, se probó la librería `pdfplumber` (Singer-Vine, 2023), pero se observó un rendimiento pobre en documentos con formato de texto en dos columnas, habitual en editoriales como MDPI, IEEE o ACM. El modo `get_text("text")` de PyMuPDF resolvió este problema al extraer el contenido a partir de las coordenadas físicas de los caracteres en lugar de seguir la codificación del texto en el archivo.

Al texto extraído se le aplicó una limpieza básica, eliminando guiones de separación de palabras entre líneas, colapsando múltiples saltos de línea y espacios, y eliminando líneas excesivamente cortas, correspondientes a cabeceras o números de página.

El resultado del pipeline fue un archivo de texto por cada artículo y un informe de extracción en formato CSV que recoge para cada artículo el número de pági-

⁴Los archivos con el código empleado en ambas fases pueden ser consultados en la carpeta `notebooks` del repositorio público del proyecto (Jiménez Cobo, 2026): `notebooks/01_text_extraction.ipynb` para el pipeline de extracción de texto y `notebooks/02_preprocessing.ipynb` para el de preprocesamiento.



nas y de tokens extraídos junto al estado del proceso⁵. La estimación de tokens se realizó mediante la librería `tiktoken` (Jain, 2022), empleando el encoding correspondiente a los modelos de la familia GPT-4. La Tabla X recoge las principales estadísticas tras la extracción de texto del corpus.

Tabla 4. *Estadísticas del corpus tras la extracción de texto.*

Métrica	Valor
Artículos procesados correctamente	55
Total de tokens	1.017.895
Media de tokens/artículo	18.507
Mediana de tokens/artículo	15.293
Mínimo de tokens	5.179
Máximo de tokens	88.956

Fuente: Elaboración propia.

Este enfoque es similar al empleado por H. Zhang et al. (2025) en el benchmark AcademicEval, los cuales también emplean la librería `PyMuPDF` para la extracción de texto, con la diferencia de que ellos obtienen el corpus de artículos desde la API de arXiv en lugar de descargar los documentos PDF.

4.2.2 Preprocesamiento del corpus

Tras extraer el contenido textual del corpus de artículos, se empleó un pipeline de procesamiento para eliminar todo el contenido no relevante para el análisis experimental. El resultado fue un corpus limpio y compuesto por las secciones más relevantes de cada registro, garantizando que pudiera ser empleado por los sistemas experimentales y evitando aumentos de cómputo innecesarios al incluir una cantidad significativa de texto no relevante.

El pipeline preprocesó cada registro del corpus de forma individual, aplicando las siguientes etapas: eliminación del bloque inicial compuesto por el título, autores, afiliaciones y cabeceras editoriales; eliminación de apéndices; eliminación de sec-

⁵Dicho informe se puede consultar en `corpus/reports/extraction_report.csv` del repositorio público del proyecto (Jiménez Cobo, 2026).



ción de referencias bibliográficas; eliminación de agradecimientos; eliminación de marcas editoriales adicionales; y normalización de saltos de línea y espacios.

Los límites de las secciones eliminadas fueron identificados a partir de la definición de expresiones regulares, diseñadas para cubrir las variantes más frecuentes en las que estas secciones suelen ser nombradas en las publicaciones. Para eliminar el bloque inicial, compuesto por el título, autores, afiliaciones, cabeceras editoriales de las revistas y *disclaimers*, se eliminó todo el contenido previo al resumen⁶. En el caso de las secciones de referencias y apéndices, se eliminaron junto a todo el contenido posterior a ellas. Adicionalmente, se estableció como condición que su aparición se diera en el 40% final del documento, evitando así que se eliminara texto relevante de forma prematura al referenciar un apéndice en el contenido principal de un registro.

La decisión de eliminar los apéndices corresponde a motivos metodológicos y prácticos. En primer lugar, el contenido de un apéndice representa material suplementario que los autores consideraron incluir al margen del contenido principal de la publicación. Desde el punto de vista práctico, introducen una cantidad de texto no relevante que no aporta valor adicional, puesto que la extensión del corpus sin estas secciones es lo suficientemente grande como para demostrar los objetivos pretendidos en este trabajo. Adicionalmente, los apéndices de algunos registros eran significativamente más extensos que la publicación original, encontrando casos de artículos de 90 páginas donde más del 80% del contenido correspondía a los apéndices.

El pipeline mostró limitaciones debido a la variabilidad de formatos que presentaban las publicaciones del corpus. En primer lugar, hubo ocho registros donde no se eliminó correctamente el bloque inicial, mientras que en cinco artículos no se eliminaron las secciones de apéndices o referencias. De mayor relevancia, hubo

⁶En algunos registros el pipeline de extracción intercambió el orden del resumen y la introducción, por lo que se estableció eliminar el contenido previo a cualquiera de las dos secciones que apareciera en primer lugar.



dos registros en donde el bloque adicional fue eliminado junto al resumen. Dado que el porcentaje de registros con errores en el preprocesado sobre el total fue bajo, se corrigió manualmente los errores, eliminando el contenido no filtrado, y usando el texto sin preprocesar de la etapa anterior para incorporar el contenido de los resúmenes en los registros en los que fueron eliminados.

El corpus preprocesado resultante muestra una reducción significativa de casi el 50% de tokens con respecto a su versión inicial. La Tabla 5 muestra una comparativa de las estadísticas del corpus antes y después del preprocesado.

Tabla 5. *Comparativa de las estadísticas del corpus antes y después del procesamiento. Los valores en gris representan la disminución porcentual tras el filtrado.*

Métrica	Antes	Después
Total de tokens	1.017.895	517.947 (-49,12%)
Media de tokens/artículo	18.507	9.417 (-49,12%)
Mediana de tokens/artículo	15.293	8.846 (-42,16%)
Mínimo de tokens	5.179	3.111 (-39,93%)
Máximo de tokens	88.956	31.306 (-64,81%)

Fuente: Elaboración propia.

4.2.3 Subconjuntos del corpus

Con el objetivo de analizar el rendimiento de los sistemas experimentales en función del volumen de contexto procesado, se definieron cuatro subconjuntos del corpus con tamaño creciente, de forma que cada nuevo subconjunto contiene los registros del subconjunto anterior más un conjunto de registros adicional. El criterio de selección de los artículos para cada subconjunto corresponde a su orden de inclusión en el protocolo de selección de la RSL descrito en la subsección 4.1⁷. La Tabla 6 muestra el número de registros incluidos en cada subconjunto y su vo-

⁷Los metadatos de los registros incluidos en cada subconjunto se pueden consultar en `corpus/subsets/` del repositorio público del proyecto (Jiménez Cobo, 2026).



lumen total en tokens.

Para controlar la posición relativa de los documentos relevantes dentro del contexto, los registros de cada subconjunto fueron reordenados mediante una permutación aleatoria con una semilla fija ($seed=42$), garantizando así la reproducibilidad del proceso. El orden de la permutación se aplicó a cada subconjunto, de forma que la posición relativa de cada registro es la misma en cada subconjunto. Esta decisión corresponde a los hallazgos de N. F. Liu et al. (2024) vistos en la sección 2.4. Al realizar una redistribución aleatoria de los documentos dentro de cada subconjunto, se pueden atribuir los resultados plenamente a la capacidad de recuperación de información de los sistemas evaluados, y no a un sesgo por situar los registros con información relevante en posiciones favorables para los LLM.

Tabla 6. *Número de registros y volumen total de cada subconjunto del corpus.*

Subconjunto	Número de registros	Volumen (tokens)
C1	20	210.469
C2	30	279.392
C3	40	362.960
C4	55	517.947

Fuente: Elaboración propia.

Las consultas empleadas en la fase experimental fueron diseñadas a partir del contenido de los registros del subconjunto C1. El contenido adicional del resto de subconjuntos no es relevante para responder a las consultas, por lo que actúan como ruido contextual con una estructura clara y coherente y con un contenido relacionado temáticamente con los registros de C1. Esta decisión se alinea con las observaciones de Hong et al. (2025) vistas en 2.4, quienes identificaron que el rendimiento de los LLM se reduce drásticamente a medida que el contexto irrelevante aumenta con texto estructurado del mismo dominio. Esta propuesta reproduce exactamente estas mismas condiciones, por lo que se analiza la degradación en LLM tradicionales frente al resto de sistemas experimentales.



4.3 Consultas

El diseño de las consultas se realizó a partir del contenido del corpus de documentos científicos, de forma similar al procedimiento que siguen distintos benchmarks como CURIE o AcademicEval, los cuales evalúan LLM sobre literatura científica mediante preguntas con respuestas de referencia anotadas a partir del contenido de sus propios corpus específicos (materiales, física y química en el caso de CURIE). Únicamente se consideraron los 20 primeros registros (los correspondientes al subconjunto C1) para formular las consultas. El objetivo de esta decisión es aumentar de forma progresiva la cantidad de texto irrelevante para responder a las consultas, pero estructurado y relacionado temáticamente con los registros relevantes. De esta forma, se intenta analizar si ocurre el fenómeno context rot que se documentó en 2.4.

Se formularon un total de 40 consultas divididas en tres niveles de complejidad: extracción factual, comparación y síntesis.

Extracción factual (16 consultas). Requieren localizar un dato concreto en el corpus. Se tratan de tareas de complejidad $O(1)$, similar a las de S-NIAH (Kamradt, 2023).

Comparación (14 consultas). Requieren localizar información específica sobre dos registros diferentes e integrarla para comparar las diferencias. Las tareas escalan de forma lineal a la longitud del contexto, esto es, presentan complejidad $O(N)$.

Síntesis (10 consultas). Para responderlas, el sistema necesita identificar patrones comunes entre tres o más documentos, agregar resultados, y sintetizar la información extraída. Su complejidad oscila entre $O(N)$ y $O(N^2)$, dependiendo del número de combinaciones de documentos que tiene que realizar el sistema.



Las consultas fueron diseñadas en documentos JSON, que contienen la consulta, la respuesta de referencia, y un índice para identificar cada una de ellas (ej. Q01). Adicionalmente, se incluyen campos para proporcionar la mayor cantidad posible de información acerca del diseño de cada consulta, como la temática de la consulta, los registros del corpus relevantes para responderla, y las citas explícitas de cada registro donde aparecen los fragmentos relevantes, junto a la ubicación dentro del registro.

El aumento de complejidad entre las consultas responde a dos patrones. En primer lugar, se requiere procesar una cantidad de información cada vez mayor, puesto que el número de documentos relevantes para responder a la consulta crece con la complejidad de estas. Por otra parte, un aumento de dificultad de la consulta requiere mayores capacidades por parte del sistema para procesarla. Una consulta de extracción simplemente consta de localizar un dato específico, lo cual es factible para cualquier sistema, como ya se vio en 10 en donde GPT-5 base obtuvo un 100% de precisión en S-NIAH consistentemente hasta que alcanzó el límite de la ventana de contexto. Por el contrario, una consulta de síntesis no sólo requiere identificar un patrón concreto tras analizar todos los registros del corpus, sino que se deben localizar diferentes registros relevantes y agregar y sintetizar resultados hasta construir la respuesta final, luego el nivel de razonamiento que se le exige al sistema es mayor.

4.4 Sistemas experimentales

Se implementaron tres sistemas experimentales con diferentes estrategias a la hora de gestionar el contexto. LLM baseline, que procesa el corpus al completo, RAG, para recuperar los fragmentos relevantes del corpus en base a la similitud semántica con la consulta, y RLM. Cada sistema fue evaluado empleando dos modelos de lenguaje diferentes: GPT-5.4 Mini (OpenAI, 2026) y Gemini 2.5 Flash (Google DeepMind, 2025a).

Cada consulta fue ejecutada dos veces por cada sistema con el objetivo de reducir



la variabilidad en las respuestas inherente a los modelos de lenguaje, puesto que una única respuesta no garantiza resultados representativos.

4.4.1 Baseline LLM

El sistema LLM base representa el enfoque más directo del procesamiento de contexto largo, puesto que incorpora directamente el contexto completo en el prompt sin emplear ningún sistema externo de recuperación o descomposición del contexto. El modelo debe localizar y sintetizar la información relevante para resolver cada consulta.

La implementación de este sistema se realiza a través de llamadas directas a las API de los modelos de lenguaje, añadiendo el corpus del subconjunto correspondiente junto a la consulta. El corpus se agrega como un texto plano, incluyendo un encabezado antes de cada registro que contiene el título, autores y año de publicación.

Cada modelo de lenguaje cuenta con sus propias características:

GPT-5.4 mini (OpenAI, 2026). Las llamadas a GPT-5.4 mini se realizan mediante la API de OpenAI con el cliente oficial. El pipeline de ejecución cuenta con un mecanismo de reintento automático de 3 intentos con una tasa de espera de 60 segundos, en caso de obtener un error por límite de tasa. De acuerdo con la documentación oficial del modelo (OpenAI, 2026), el modelo cuenta con una ventana de contexto de 400.000 tokens. No obstante, el límite práctico de la ventana de contexto de entrada del modelo es de 272.000 tokens, lo cual supone que el sistema no puede procesar los subconjuntos C3 y C4 del corpus, al exceder el límite de tokens del modelo. En ambos casos, se registrará el fallo como resultado de forma similar a la figura 10⁸, ilustrando así una ventaja práctica de los sistemas diseñados para procesar volúmenes mayores de contexto, como es el caso de los RLM.

⁸Se optó por no truncar el contexto a un tamaño que pudiese ser procesado por el modelo ya que esto induciría sesgos en los resultados dependiendo de si el contexto relevante se encontraba dentro del fragmento truncado o del conservado.



Gemini 2.5 Flash (Google DeepMind, 2025a). Las llamadas a la API se realizan desde el cliente nativo de Google (`google.genai.Client`). El pipeline incluye un mecanismo de espera entre llamadas de 65 segundos para subconjuntos con una longitud menor a 400.000 tokens, y de 130 segundos para contextos de longitud mayor, con el fin de evitar errores de saturación de la API. Gemini 2.5 Flash cuenta con una ventana de contexto de un millón de tokens, lo que lo hace apto para procesar todos los subconjuntos del corpus.

Ambos modelos fueron ajustados con temperatura 0 para garantizar la reproducibilidad de las respuestas. El prompt del sistema de ambos modelos incluye guías específicas para responder cada tipo de consulta, empleando *role prompting* (White et al., 2023) para asignar al modelo un rol de experto en análisis de literatura científica para orientar el estilo y formato de las respuestas. El prompt exacto empleado se puede consultar en el Apéndice A.

4.4.2 RAG

El sistema RAG se implementa mediante LlamaIndex (J. Liu, 2022) con ChromaDB (Chroma, 2022) como almacén vectorial. Cada documento del corpus se indexa como un documento de LlamaIndex con metadatos de identificador, título y año, y son fragmentados mediante `SentenceSplitter`, con un tamaño de *chunk* de 512 tokens y solapamiento de 50 tokens entre fragmentos consecutivos. Posteriormente, los embeddings se generan mediante el modelo `text-embedding-3-small` (OpenAI, 2024). El índice vectorial se construye una única vez por modelo y subconjunto, almacenados como colecciones en ChromaDB con el formato `rag_{subconjunto}_{modelo}`.

En la fase de consulta, el recuperador extrae los $k = 5$ fragmentos más similares semánticamente a la consulta empleando la similitud coseno. Dichos fragmentos son enviados al modelo generador junto con la consulta. Al igual que en LLM base, los modelos generadores se configuran con temperatura 0 para garantizar reproducibilidad. Ambos modelos de lenguaje se encuentran disponibles en



LlamaIndex, GPT-5.4 mini a través de LlamaOpenAI y Gemini 2.5 Flash mediante GoogleGenAI.

4.4.3 RLM

El sistema RLM se implementa mediante la biblioteca oficial `rlm` publicada por A. L. Zhang (2025). El sistema cuenta con un modelo raíz y un modelo de sub-llamadas para cada configuración: GPT-5.4 mini junto a GPT-5.4 nano (OpenAI, 2026), y Gemini 2.5 Flash junto a Gemini 2.5 Flash-Lite (Google DeepMind, 2025b). El número máximo de iteraciones del bucle de razonamiento se limita a 15.

La biblioteca `rlm` solo establece conexión con modelos de la interfaz de OpenAI y Anthropic, por lo que para emplear los modelos de la familia Gemini se usó el *endpoint* compatible con la interfaz de OpenAI, proporcionado por Google⁹. Esto permite usar el *backend* OpenAI sin realizar modificaciones en el código.

El corpus de cada subconjunto se pasa como contexto al sistema mediante el método `rlm.completion()`, el cual permite añadir un prompt raíz al que el RLM tiene acceso al inicio y le proporciona la consulta y las instrucciones específicas para interactuar con el entorno REPL (el prompt raíz se puede consultar en el Apéndice A). Por otra parte, las trayectorias de razonamiento de cada ejecución se registran en formato JSONL mediante `RLMLogger`, permitiendo así su posterior análisis mediante el visualizador de trayectorias incluido en el repositorio de A. L. Zhang (2025).

La Tabla 7 muestra una recopilación de los modelos empleados en cada sistema y los subconjuntos del corpus a los que pueden ser aplicados.

⁹<https://generativelanguage.googleapis.com/v1beta/openai/>



Tabla 7. Configuraciones de los sistemas experimentales.

Sistema	Modelo raíz	Subllamadas	Subconjuntos
Baseline LLM	GPT-5.4 mini	—	C1, C2
Baseline LLM	Gemini 2.5 Flash	—	C1, C2, C3, C4
RAG	GPT-5.4 mini	—	C1, C2, C3, C4
RAG	Gemini 2.5 Flash	—	C1, C2, C3, C4
RLM	GPT-5.4 mini	GPT-5.4 nano	C1, C2, C3, C4
RLM	Gemini 2.5 Flash	Gemini 2.5 Flash-Lite	C1, C2, C3, C4

Fuente: Elaboración propia.

4.5 Métricas de evaluación

Precisión. La calidad de las respuestas generadas por los modelos de lenguaje se evaluó mediante un sistema híbrido que combina evaluación manual y automática. En primer lugar, se redactaron manualmente las respuestas de referencia a cada consulta, basadas únicamente en el contenido de los registros correspondientes al subconjunto C1. Para la evaluación de las respuestas, se empleó la metodología *LLM-as-judge* (Zheng et al., 2023), la cual emplea un LLM que actúa como juez para comparar las respuestas generadas por los sistemas frente a las respuestas de referencia y asignar una puntuación en una escala de 1 a 5, donde 1 indica una respuesta incorrecta y no relacionada con la consulta, mientras que 5 indica una respuesta completa y precisa. Esta metodología está ampliamente establecida en la literatura de evaluación de LLM, y es la empleada en AcademicEval (H. Zhang et al., 2025) para realizar la evaluación de las consultas. Como modelo juez se empleó GPT-5.4 mini para todas las respuestas de todos los modelos, con el objetivo de mantener un criterio de evaluación uniforme en todas las respuestas. En primera instancia, se empleó el mismo prompt para el LLM juez en cada tipo de consulta, pero se observó que el juez penalizaba respuestas correctas cuando



presentaban variaciones menores con respecto a la respuesta de referencia, como diferencias en capitalización, artículos omitidos o variaciones mínimas que no afectaban a la calidad de la respuesta. Por tanto, se emplearon dos prompts de evaluación distintos en función del tipo de consulta: uno para extracción factual, centrado en la corrección conceptual de la respuesta, y otro para comparación y síntesis, el cual evalúa la completitud de la respuesta sin requerir coincidencia exacta con la respuesta de referencia. Ambos prompts se incluyen en el Apéndice A.

Coste. El coste de cada consulta se calculó mediante el número de tokens de entrada y salida consumidos en cada llamada a la API, multiplicado por el precio por millón de tokens de cada modelo establecido en la documentación oficial de cada LLM. En caso del RLM, el coste se calcula a partir de los registros JSONL generados por `RLMLogger`, sumando tanto los tokens consumidos por el modelo raíz como los de cada sub-llamada de forma independiente.

Latencia. Se midió el tiempo transcurrido entre el envío de la consulta y la recepción de la respuesta individualmente para cada ejecución. A partir de estos resultados, se calculó la media y desviación típica por sistema, modelo y subconjunto, permitiendo así analizar el tiempo de respuesta en función del volumen de contexto.

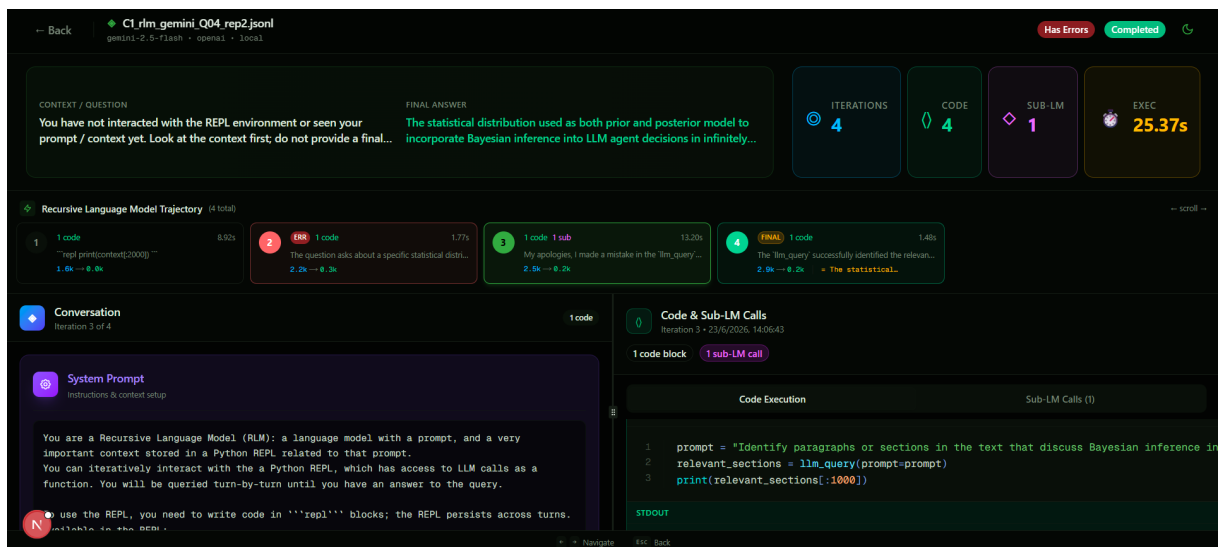
Análisis de trayectorias de RLM. Como se detalló anteriormente, las trayectorias de razonamiento de cada ejecución de los RLM se registran en formato JSONL mediante `RLMLogger`, almacenando cada iteración del bucle recursivo junto con el código generado, resultados de ejecución y las sub-llamadas. Para inspeccionar cada registro, el repositorio de A. L. Zhang (2025) incluye un visualizador web local desarrollado con Node.js y shdcn/ui, el cual es lazando mediante los comandos:



```
cd visualizer/  
npm run dev
```

La Figura 13 muestra la interfaz del visualizador, que permite navegar a través de las iteraciones de cada ejecución, inspeccionando el razonamiento interno del modelo raíz, el código generado en cada paso, las sub-llamadas y las respuestas intermedias obtenidas. Esta herramienta permitió realizar un análisis cualitativo sobre las trayectorias de razonamiento de los RLM e identificar patrones de fallo que se encuentran detallados en el capítulo 5.

Figura 13. *Interfaz del visualizador de trayectorias RLM incluido en el repositorio de A. L. Zhang (2025). Permite inspeccionar de forma interactiva las iteraciones del bucle recursivo, el código generado en cada paso, las sub-llamadas realizadas y los resultados intermedios obtenidos.*



Fuente: Interfaz del visualizador de trayectorias RLM de A. L. Zhang (2025).



5. Resultados

El sistema experimental descrito en el anterior capítulo incluyó los modelos de lenguaje GPT-5.4 mini y Gemini 2.5 Flash, evaluados en tres sistemas distintos: el LLM base, RAG y RLM. Cada sistema fue evaluado sobre un conjunto de 40 consultas distribuidas en tres niveles de complejidad, con dos repeticiones por ejecución, dando lugar a 1.760 ejecuciones válidas sobre los cuatro subconjuntos del corpus.

Los resultados evidencian patrones diferenciados según el tipo de consulta y volumen de texto procesado, con implicaciones distintas para cada sistema. Durante este capítulo se analizan dichos resultados, incluyendo la comparación de rendimiento entre los sistemas (sección 5.1), el análisis del coste y latencia (sección 5.2), y un análisis cuantitativo y cualitativo del comportamiento específico del RLM (sección 3.3).

5.1 Rendimiento comparativo de los sistemas

La tabla 8 contiene la puntuación media y desviación estándar de cada sistema evaluado en la fase experimental por nivel de consulta y subconjunto del corpus.



Tabla 8. Resultados experimentales por sistema, modelo y tipo de consulta sobre los cuatro subconjuntos del corpus. Se muestra la puntuación media $\pm$ desviación estándar en escala 1–5. El asterisco (*) indica ejecuciones omitidas por límite de ventana de contexto.

Sistema	C1	C2	C3	C4
<i>Extracción factual (N = 16)</i>				
GPT-5.4 mini				
LLM base	4,75 $\pm$ 0,44	4,81 $\pm$ 0,40	*	*
RAG	4,44 $\pm$ 1,01	4,41 $\pm$ 1,21	4,31 $\pm$ 1,33	4,19 $\pm$ 1,38
RLM	4,09 $\pm$ 1,49	3,78 $\pm$ 1,56	3,78 $\pm$ 1,58	3,97 $\pm$ 1,40
Gemini 2.5 Flash				
LLM base	4,75 $\pm$ 0,62	4,94 $\pm$ 0,25	4,91 $\pm$ 0,30	4,75 $\pm$ 0,62
RAG	4,75 $\pm$ 0,76	4,56 $\pm$ 1,01	4,31 $\pm$ 1,33	4,34 $\pm$ 1,29
RLM	3,09 $\pm$ 1,80	2,81 $\pm$ 1,55	2,50 $\pm$ 1,76	2,56 $\pm$ 1,72
<i>Comparación (N = 14)</i>				
GPT-5.4 mini				
LLM base	4,96 $\pm$ 0,19	4,68 $\pm$ 0,82	*	*
RAG	4,11 $\pm$ 1,10	3,93 $\pm$ 1,02	3,96 $\pm$ 1,04	4,18 $\pm$ 1,02
RLM	4,14 $\pm$ 1,21	3,79 $\pm$ 1,34	3,71 $\pm$ 1,49	3,68 $\pm$ 1,61
Gemini 2.5 Flash				
LLM base	4,61 $\pm$ 0,50	4,71 $\pm$ 0,66	4,68 $\pm$ 0,48	4,61 $\pm$ 0,50
RAG	2,39 $\pm$ 0,74	2,25 $\pm$ 0,65	2,50 $\pm$ 0,75	2,43 $\pm$ 0,69
RLM	3,50 $\pm$ 1,11	3,75 $\pm$ 1,24	3,57 $\pm$ 1,29	2,79 $\pm$ 1,03



Sistema	C1	C2	C3	C4
<i>Síntesis ($N = 10$)</i>				
GPT-5.4 mini				
LLM base	4,40 $\pm$ 0,50	4,45 $\pm$ 0,51	*	*
RAG	3,15 $\pm$ 0,88	3,30 $\pm$ 0,92	3,05 $\pm$ 0,95	3,40 $\pm$ 0,82
RLM	3,60 $\pm$ 1,27	3,40 $\pm$ 1,27	3,85 $\pm$ 1,09	3,55 $\pm$ 1,36
Gemini 2.5 Flash				
LLM base	4,05 $\pm$ 0,51	3,95 $\pm$ 0,51	4,00 $\pm$ 0,56	4,10 $\pm$ 0,45
RAG	2,55 $\pm$ 0,83	2,25 $\pm$ 0,79	2,15 $\pm$ 0,88	2,10 $\pm$ 0,72
RLM	3,15 $\pm$ 1,39	3,70 $\pm$ 0,73	3,65 $\pm$ 1,09	3,05 $\pm$ 1,15

Fuente: Elaboración propia.

De los resultados se extraen las siguientes observaciones:

Observación 1: Los LLM base obtienen un rendimiento superior cuando el corpus cabe en la ventana de contexto del modelo. Ambos LLM base, GPT-5.4 mini y Gemini 2.5 Flash, obtienen puntuaciones significativamente superiores a los sistemas RLM en todos los tipos de consulta (véase Figura 15) siempre que el contexto cabe en su ventana de contexto (C1 y C2 para GPT y todos los subconjuntos para Gemini), con un p-valor $p < 0,001$ en el test Mann-Whitney U¹ en todos los casos. Los LLM base también superan de forma significativa a RAG en comparación y síntesis ($p < 0,001$), pero en tareas de extracción factual, esta diferencia no alcanza significación estadística ($p = 0,143$ para GPT y $p = 0,067$ para Gemini), lo que indica que RAG ofrece un rendimiento comparable a los LLM base en consultas de extracción. La desviación típica de los LLM base es baja, con un rango de

¹Se empleó el test de Mann-Whitney U por ser un test no paramétrico apropiado para comparaciones de distribuciones en escalas ordinales discretas. La elección de un test no paramétrico viene dada por la distribución observada de las puntuaciones (véase Figura 19), que rechaza la hipótesis de la normalidad en todos los sistemas y tipos de consulta según el test de Shapiro-Wilk ($p \approx 0$ en todos los casos).



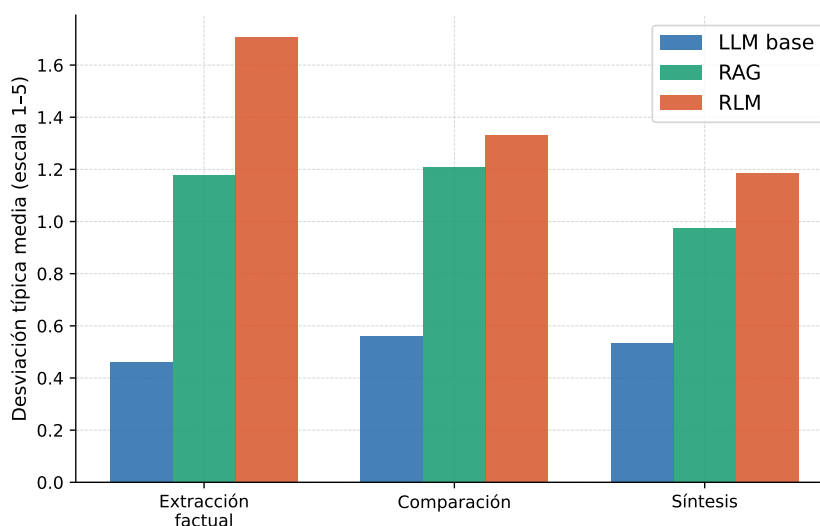
0,19-0,82 para GPT-5.4 mini y 0,25-0,66 para Gemini 2.5 Flash, lo que indica que su rendimiento se mantiene estable entre las distintas repeticiones. GPT-5.4 mini no dispone de resultados en los subconjuntos C3 y C4 dado el límite efectivo de 272.000 tokens en la ventana de contexto del modelo, aunque en los subconjuntos que sí puede procesar no se manifiesta una degradación por el aumento del tamaño del contexto. Por el contrario, la ventana de contexto de un millón de tokens de Gemini 2.5 Flash le permite procesar todos los subconjuntos del corpus y mantener resultados consistentes en todos ellos.

Observación 2: Los RLM presentan una variabilidad superior a los LLM base y similar a RAG. La desviación típica media de los RLM es superior en todos los tipos de consultas (véase Figura 14). El test de Fligner-Killeen (Fligner & Killeen, 1976)² confirma que esta diferencia es estadísticamente significativa para los LLM base en todos los tipos de consulta y RAG únicamente en extracción factual ($p < 0,001$), con una desviación típica media de los RLM de 1,61, frente a valores inferiores de 0,43 en los LLM base y 1,17 en RAG. Sin embargo, la diferencia entre RLM y RAG no es significativa en las consultas de comparación ($p = 0,853$) y síntesis ($p = 0,177$). Este patrón de variabilidad será analizado en detalle en la sección 5.3.

²El test no paramétrico de Fligner-Killeen resulta apropiado para comparar varianzas de distribuciones no normales.



Figura 14. Variabilidad del rendimiento por sistema y tipo de consulta. Los RLM muestran una desviación típica mayor que el resto de sistemas, especialmente en las consultas de extracción, en donde la diferencia con RAG es de más de 0,5 puntos.



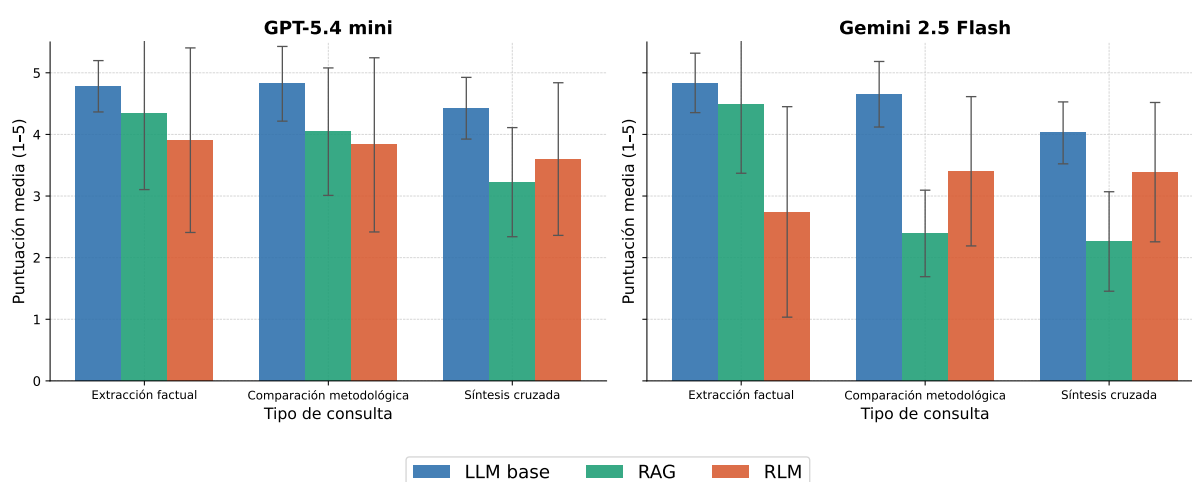
Fuente: Elaboración propia.

Observación 3: El rendimiento comparativo entre RLM y RAG depende del tipo de consulta. Como se vio anteriormente, RAG obtiene un rendimiento competitivo en consultas de extracción factual, con puntuaciones medias similares a las de los LLM base y con una desviación típica mucho mayor, lo cual indica que el rendimiento de RAG es sólido cuando recupera el fragmento correcto pero inconsistente, ya que hay ejecuciones donde falla en caso de que el mecanismo de recuperación no proporcione el fragmento correcto. Para el modelo GPT-5.4 mini, RAG se posiciona como la mejor alternativa en los subconjuntos C3 y C4 del corpus, los cuales no pueden ser procesados por el modelo base. Para este mismo tipo de consultas, el rendimiento de los RLM es significativamente menor al de RAG ($p < 0,005$ en el test de Mann-Whitney U), especialmente para Gemini 2.5 Flash donde la puntuación media cae por debajo de los 3 puntos. No obstante, este patrón contrasta con los resultados de las consultas de comparación y síntesis, donde las puntuaciones de RAG disminuyen notablemente, especialmente en el caso de Gemini 2.5 Flash, el cual es incapaz de obtener una media por encima de



2,55 en ningún subconjunto. Los RLM, por el contrario, muestran resultados superiores a RAG en consultas de síntesis ($p < 0,001$) y en comparación para Gemini 2.5 Flash ($p < 0,001$). No obstante, la diferencia entre RAG GPT-5.4 mini y RLM GPT-5.4 mini no es estadísticamente significativa ($p = 0,748$).

Figura 15. Puntuación media por modelo, sistema y tipo de consulta. Los LLM base mejoran los resultados de RAG y RLM en todos los tipos de consultas, especialmente las de comparación y síntesis para Gemini 2.5 Flash, donde el rendimiento de RAG decae considerablemente. Los RLM mantienen una puntuación media estable en todos los tipos de consulta tanto en GPT-5.4 mini como en Gemini 2.5 Flash salvo en las consultas de extracción de este último, en donde la puntuación media cae por debajo de los 3 puntos.



Fuente: Elaboración propia.

Los registros de las respuestas de los sistemas RAG confirman que, en las tareas de comparación y síntesis, el recuperador extrae los fragmentos de un único documento de forma sistemática, a pesar de que dichas consultas requieran agregar información sobre distintas fuentes. Esto impide al modelo generador acceder a toda la información necesaria que las consultas requieren. No obstante, se puede observar que el modelo generador constituye un factor determinante para determinar el rendimiento de RAG con independencia de los fragmentos recuperados. En la Figura 22 del apéndice B.1 se muestra un ejemplo en el que RAG GPT-5.4 mini obtiene resultados sustancialmente mejores que RAG Gemini 2.5 Flash, a pesar de

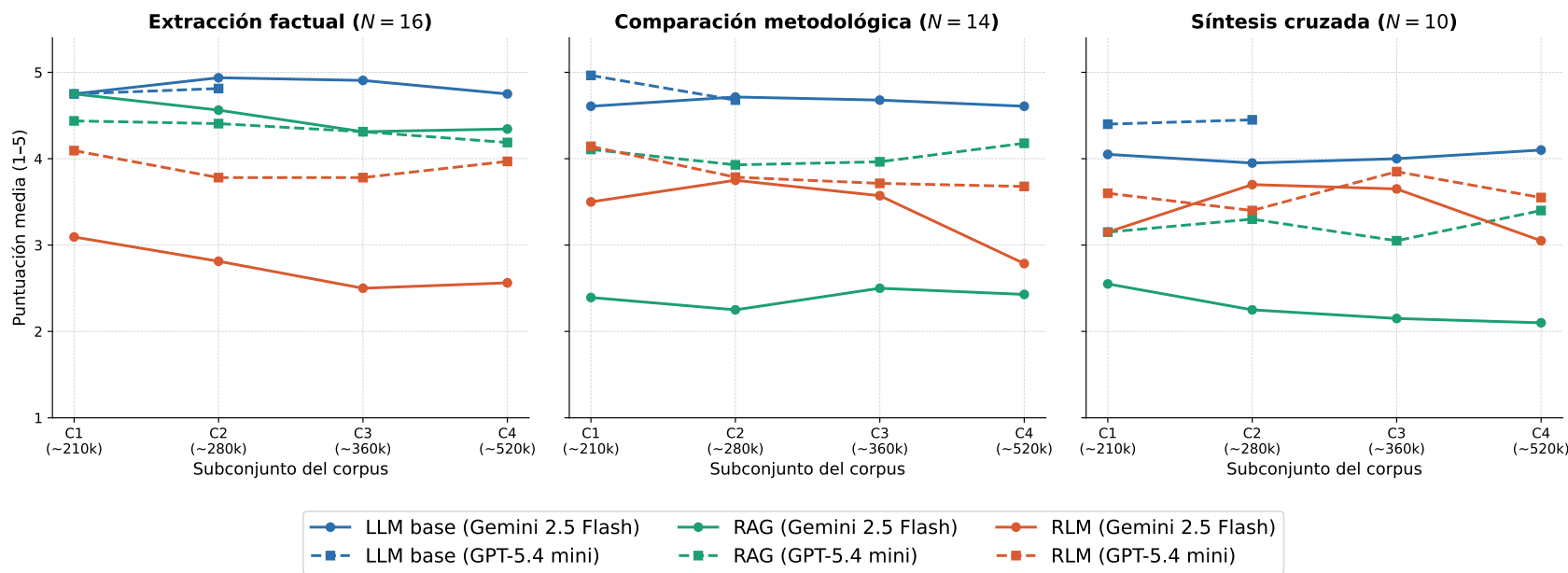


recibir los mismos fragmentos recuperados. Esta diferencia puede deberse a dos factores: (1) que la capacidad de redacción y síntesis del modelo generador influye en mayor medida que el número de fragmentos recuperados correctamente y (2) que el LLM juez muestra un sesgo de auto-preferencia, dado que el modelo GPT-5.4 mini actúa simultáneamente como modelo generador y juez.

Observación 4: El rendimiento de los sistemas no se degrada con el aumento del tamaño del contexto de entrada. La Figura 16 muestra que ningún sistema presenta una degradación sistemática a medida que aumenta el volumen del contexto de entrada. Los LLM base muestran puntuaciones constantes en los diferentes subconjuntos del corpus, sin evidencia del fenómeno *context rot* descrito en 2.4. RAG presenta oscilaciones en las tareas de comparación y síntesis, pero no muestra signos claros de caída. Se puede observar un rendimiento estable por parte del RLM GPT-5.4 mini, con ciertas oscilaciones en síntesis sin evidencia de degradación notable. RLM Gemini 2.5 Flash es el único sistema que tiene un comportamiento diferente al resto. En consultas de extracción su puntuación decae desde los 3,09 puntos en C1 hasta los 2,56 en C4, mientras que en comparación y síntesis, el rendimiento crece de C1 a C2, pero luego experimenta una caída notable de C3 a C4, pasando de los 3,57 puntos a los 2,79 en comparación y de 3,65 a 3,05 en síntesis.



Figura 16. Evolución del rendimiento de cada sistema por nivel de consulta a lo largo de los distintos subconjuntos del corpus. No se evidencia una degradación significativa, salvo oscilaciones de RAG y RLM GPT-5.4 mini en tareas de síntesis y una caída progresiva de RLM Gemini 2.5 Flash en tareas de extracción, sumado a caídas bruscas en comparación y síntesis en C4. Se observa una degradación en relación con la complejidad de consulta, con puntuaciones inferiores en síntesis comparado con extracción factual en la mayoría de sistemas.



Fuente: Elaboración propia.



No obstante, sí se observa un patrón de degradación asociado a la complejidad de la consulta. La diferencia de rendimiento entre extracción factual y síntesis es estadísticamente significativa en todos los sistemas ($p < 0,001$ en el test Mann-Whitney U), lo que confirma que la complejidad de la tarea es determinante en el rendimiento con independencia del sistema evaluado.. Este patrón se alinea con la naturaleza de cada consulta, dado que las de extracción requiere localizar un dato específico en el corpus, mientras que las de comparación y síntesis necesitan agregar y relacionar información que se encuentra distribuida en distintos registros.

5.2 Análisis de coste y latencia: frontera de Pareto

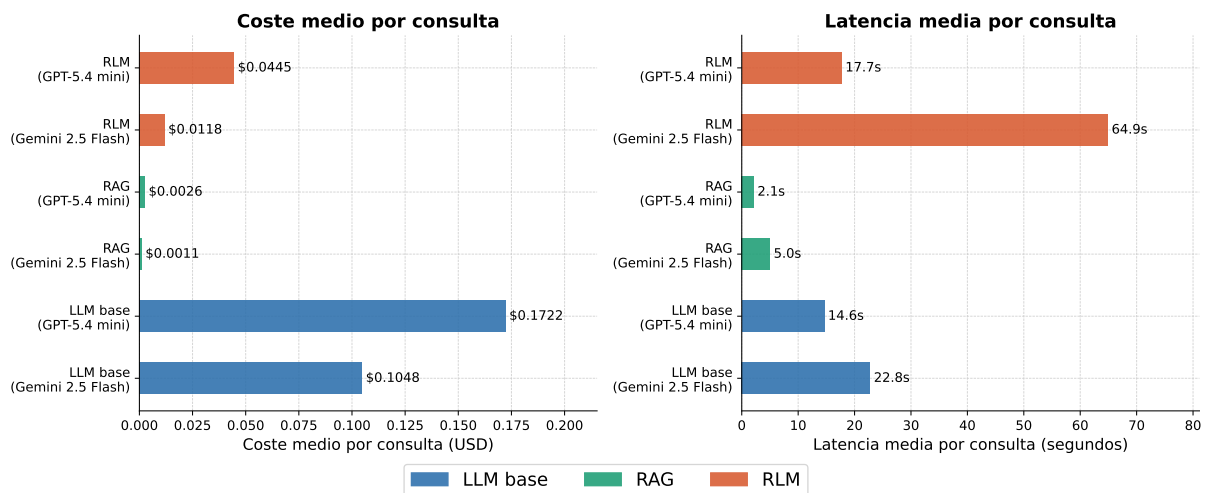
La Figura 17 muestra las diferencias del coste y la latencia entre los sistemas. A pesar de que el coste absoluto del LLM base es asumible en escenarios de investigación, la diferencia relativa entre sistemas se vuelve significativa en escala. Para GPT-5.4 mini, el coste medio del LLM base es casi 4 veces mayor que el del RLM, y hasta 66 veces superior al de RAG. En el caso de Gemini 2.5 Flash, esta escala es de aproximadamente 9 veces el coste medio del RLM y más de 95 veces el de RAG. En cuanto a la diferencia entre los modelos de lenguaje, GPT-5.4 mini tiene un coste medio superior a Gemini 2.5 Flash en todos los sistemas. El caso más destacable es en los RLM, en donde la diferencia es de hasta 4 veces el coste de RLM Gemini 2.5 Flash, lo cual sugiere que cada modelo emplea una estrategia diferente a la hora de gestionar el número de iteraciones del bucle recursivo y de sub-llamadas. Esto se analizará con más detalle en la sección 5.3.

Con respecto a la latencia de cada sistema, RLM Gemini 2.5 Flash es el sistema con mayor tiempo medio por respuesta, mostrando una amplia diferencia con el resto de sistemas. Analizando los percentiles de la distribución de latencias de RLM Gemini 2.5 Flash, se tiene que la latencia del percentil 95 es de casi 250 segundos, mientras que el percentil 75 se sitúa en el rango de 60-70 segundos. No obstante, este aumento en el razonamiento implica un cambio en el rendimiento, y es que



no hay una diferencia significativa entre la puntuación media de las ejecuciones que se encuentran por encima del percentil 95 con las que están por debajo. Esta cuestión también se analizará con detalle en la sección 5.3.

Figura 17. Coste medio por consulta en USD (izquierda) y latencia media por consulta en segundos (derecha) de cada sistema experimental.

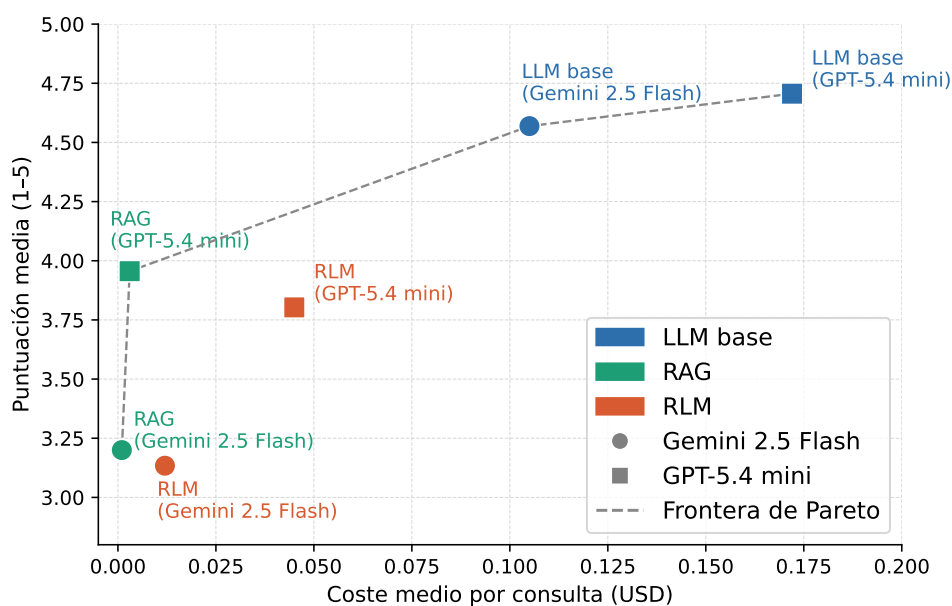


Fuente: Elaboración propia.

Para realizar una comparación global entre los sistemas, se ha empleado un análisis de calidad-coste mediante la frontera de Pareto, una curva que conecta los sistemas para los que no existe ninguna alternativa con igual o mejor puntuación media e igual o menor coste simultáneamente. Los sistemas situados fuera de la frontera se encuentran dominados, esto es, existe al menos una alternativa con mejor rendimiento o menor coste. A las instancias que forman la frontera de Pareto se las denomina como óptimos de Pareto.



Figura 18. Análisis de calidad-coste de cada sistema experimental mediante la frontera de Pareto. Los puntos que se encuentran en la frontera de Pareto (LLM base GPT-5.4 mini, LLM base Gemini 2.5 Flash, RAG GPT-5.4 mini y RAG Gemini 2.5 Flash) son óptimos de Pareto, lo cual indica que no existe ningún otro sistema que de mejores resultados con un coste menor. Los sistemas RLM quedan dominados por RAG GPT-5.4 mini, puesto que ofrece mejor rendimiento con un menor coste.



Fuente: Elaboración propia.

Los óptimos de Pareto en este caso son los dos sistemas LLM base y los dos RAG. La curva presenta una forma convexa, lo cual implica que el aumento de puntuación implica un coste marginal más elevado a medida que se avanza en la frontera. Los dos sistemas RLM se encuentran dominados por RAG GPT-5.4 mini, puesto que obtiene una puntuación media superior con un menor coste. Adicionalmente, RAG GPT-5.4 mini presenta un rendimiento significativamente superior a RAG Gemini 2.5 Flash (alrededor de 0,75 puntos más) con un aumento de coste mínimo, mientras que ofrece resultados óptimos comparado con los LLM base (situándose en un rango de 0,5-0,75 puntos menos) y con un coste considerablemente menor (66 veces más barato que el modelo GPT-5.4 mini base y 40 veces más barato que Gemini 2.5 Flash).



Por tanto, el modelo óptimo para emplear en la fase experimental depende tanto del presupuesto de la investigación como de la ventana de contexto del modelo de lenguaje seleccionado. El modelo base GPT-5.4 mini cuenta con los mejores resultados absolutos de todos los sistemas, pero cuenta con la restricción de la ventana de contexto de 272.000 tokens, lo que lo hace inviable en caso de que la longitud del corpus supere dicha barrera. El LLM base Gemini 2.5 Flash amplía la ventana de contexto al millón de tokens y produce unos resultados similares a GPT-5.4 mini, lo cual lo convierte en una opción sólida para corpus académicos extensos. En caso de optar por una alternativa más económica, RAG GPT-5.4 mini ofrece resultados competitivos con un precio significativamente menor a los LLM base.

5.3 Análisis del comportamiento del RLM

Los resultados presentados en la sección 5.1 evidencian una variabilidad sistemática en los resultados de RLM GPT-5.4 mini y RLM Gemini 2.5 Flash significativamente mayor a la del resto de sistemas. Esta sección profundiza en este comportamiento, identificando patrones de fallo en las ejecuciones de los RLM (subsección 5.3.1) y realizando un experimento de ablación para aislar el efecto del ajuste del prompt raíz sobre el rendimiento del RLM en las consultas de extracción factual (subsección 5.3.2).

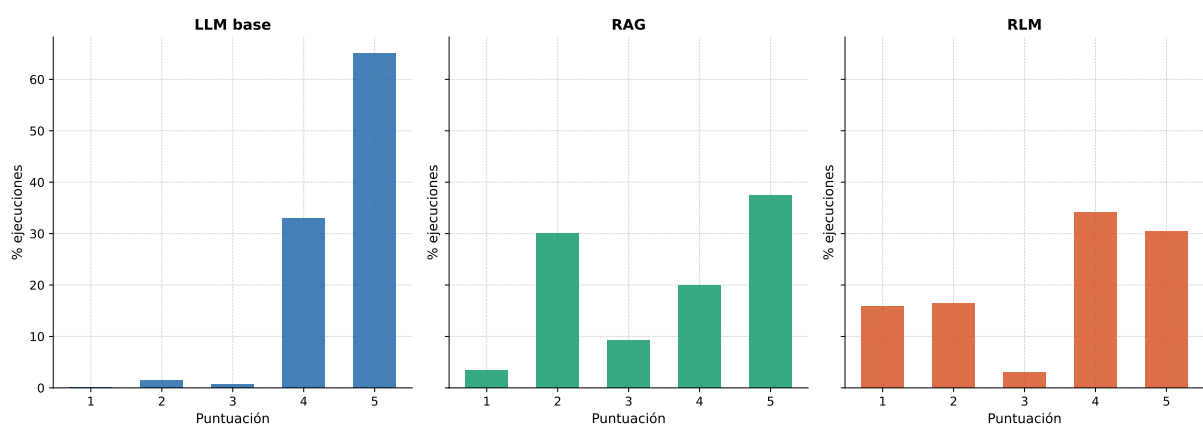
5.3.1 Fallos en las ejecuciones del RLM

Para analizar la alta varianza de los sistemas RLM, se inspeccionó la distribución de resultados en cada uno de los tres sistemas evaluados (véase Figura 19). Mientras que los LLM base obtienen una puntuación de 4 o 5 puntos de forma consistente en el 98% de ejecuciones, RAG muestra una distribución bimodal con modas en 2 y 5. Por el contrario, los RLM presentan una distribución irregular, en donde el 15,9% de ejecuciones tuvo una puntuación de 1. La característica más llamativa de los resultados del RLM era la alta variabilidad intraconsulta, ya que en distintas repeticiones de una misma consulta, el sistema podía obtener una puntuación de



1 en una repetición y de 5 en la otra. Esto motivó un análisis cualitativo de las trayectorias de razonamiento de los RLM, con el objetivo de identificar los patrones de fallo que explicaran este comportamiento.

Figura 19. Distribución de las puntuaciones de cada sistema. El LLM base obtiene 4 y 5 en el 98% de las ejecuciones, mientras que RAG y RLM presentan una distribución irregular.



Fuente: Elaboración propia.

El análisis de las trayectorias de razonamiento de los RLM empleando el visualizador de trayectorias de Node.js permitió identificar tres patrones distintos de fallos sistemáticos en las ejecuciones de los RLM. Estos fallos incluían tanto errores de los modelos raíz, errores del pipeline de ejecución y limitaciones del propio paradigma de los RLM.

Patrón 1: Errores por saturación de la API de Gemini que el pipeline no detectó. Se identificó que una fracción significativa de las ejecuciones de RLM Gemini 2.5 Flash con puntuación de 1 no corresponde a fallos del modelo raíz, sino a errores de conectividad de la API de Gemini. En concreto, se obtuvo el siguiente error: "Error code: 503 - 'This model is currently experiencing high demand. Spikes in demand are usually temporary. Please try again later.'", el cual indica que la demanda de llamadas saturó la API del modelo, impidiendo que el modelo raíz pudiera ejecutar código para inspeccionar



contexto o realizar sub-llamadas recursivas a RLM Gemini 2.5 Flash-Lite. El pipeline de experimentación está diseñado para detectar errores en las ejecuciones y permitir eliminar los registros para ejecutar de nuevo los experimentos fallidos. No obstante, en estos casos el error no se notificaba al sistema, sino que era el modelo raíz el que lo recibía de forma interna. Al no poder ejecutar ningún paso intermedio, el modelo raíz emitía el mensaje de error como una cadena de texto en su respuesta final cuando era forzado a dar una respuesta en la iteración final, por lo que el sistema lo registraba como respuesta válida y no identificaba el error. De las 59 ejecuciones que obtuvieron una puntuación de 1, el 49,2% fue debido a este tipo de error.

Patrón 2: Alucinaciones en las sub-llamadas de Gemini 2.5 Flash-Lite. En las ejecuciones de RLM Gemini 2.5 Flash con puntuaciones de 1 que no contenían mensajes de error se identificó un patrón de alucinación. El modelo raíz inspeccionaba el contexto correctamente, y generaba un plan para identificar los fragmentos relevantes del corpus y construir la respuesta final. No obstante, el modelo encargado de las sub-llamadas, Gemini 2.5 Flash Lite, fallaba al recuperar los fragmentos relevantes del corpus y, en lugar de notificar el fallo, generaba texto relacionado con la consulta que no se correspondía con el contenido real, atribuyendo falsamente el texto generado al documento del corpus necesario para responder a la consulta. Finalmente, la respuesta se construía empleando la información incorrecta, lo que causaba que el juez LLM detectara la discrepancia y calificara la respuesta como un 1. En el Apéndice B.2.1 se detalla una trayectoria de razonamiento que ejemplifica este patrón.

Patrón 3: Fallo de interacción de GPT-5.4 mini con el entorno REPL. El patrón más significativo fue el identificado en las trayectorias de razonamiento de RLM GPT-5.4 mini con puntuación 1. La causa común en todas las ejecuciones es que el modelo raíz no ejecuta código en el entorno REPL para inspeccionar el contexto



ni realiza sub-llamadas recursivas. En lugar de eso, inicia un bucle en el que establece una estrategia para inspeccionar el contexto y usar las sub-llamadas para construir la respuesta progresivamente. Sin embargo, el modelo raíz entra en un estado de inactividad en el que no redacta código que pueda ejecutar en el REPL ni realiza sub-llamadas para llevar a cabo dicho plan. En ciertas ejecuciones aisladas se observa cómo el modelo raíz llega a ejecutar código para inspeccionar el contexto pasadas varias iteraciones sin realizar ninguna acción, pero no continúa con el proceso recursivo para construir la respuesta. Al alcanzar el límite máximo de iteraciones en el bucle, el modelo finalmente genera una respuesta en la que indica que no ha podido acceder al contenido del contexto y la cual es calificada con una puntuación de 1 por el LLM juez, al no responder a la consulta. El Apéndice B.2.2 muestra dos ejemplos en los que se observa dicho comportamiento.

La Tabla 9 evalúa el impacto de este comportamiento en el rendimiento general de RLM GPT-5.4 mini. El porcentaje de ejecuciones que muestran este patrón es del 13,43%, y se distribuyen de forma similar en cada tarea, con valores que oscilan entre el 11,6% en comparación y el 16,2% en síntesis. Las puntuaciones medias al eliminar dichas ejecuciones incrementan la puntuación media en 0,44 puntos y reducen la desviación típica media en 1 punto aproximadamente, haciendo que el rendimiento de RLM GPT-5.4 mini llegue incluso a superar al de RAG GPT-5.4 mini presentado en la Tabla 8. Dicha mejora es estadísticamente significativa en extracción factual ($p = 0,045$ en Mann-Whitney U) y síntesis ($p = 0,032$), pero no alcanza significación estadística en comparación ($p = 0,118$). El caso más destacable es en las consultas de mayor complejidad, las de síntesis, donde el incremento de la puntuación media es de medio punto y la desviación típica se reduce en 0,74 puntos. Por tanto, este comportamiento del RLM tiene un impacto negativo notorio en los resultados finales del sistema.



Tabla 9. Comparación del rendimiento de RLM GPT-5.4 mini con y sin puntuaciones de 1, por tipo de consulta.

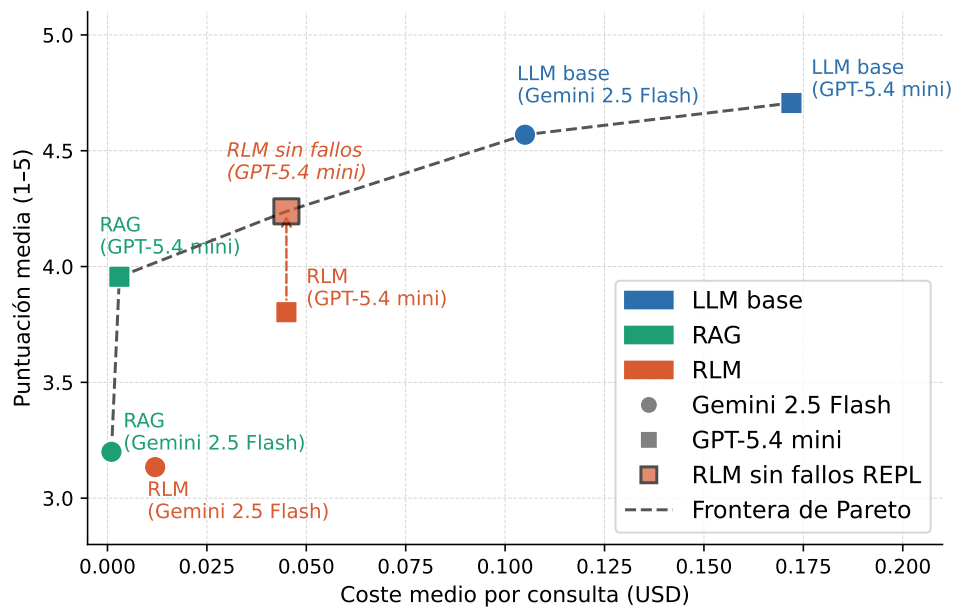
Tipo de consulta	Porcentaje puntuaciones=1	Con puntuaciones=1		Sin puntuaciones=1	
		Media	SD	Media	SD
Extracción	13,3%	3,91	1,50	4,35 +0,44	1,04 -0,46
Comparación	11,6%	3,83	1,41	4,20 +0,37	1,03 -0,38
Síntesis	16,2%	3,60	1,24	4,10 +0,50	0,50 -0,74
Promedio	13,4%	3,80	1,40	4,24 +0,44	0,91 -0,49

Fuente: Elaboración propia.

El impacto de eliminar las ejecuciones erráticas de RLM GPT-5.4 mini se traduce en un cambio en la posición relativa de este sistema frente al resto de sistemas del diseño experimental. Como se muestra en la Figura 20, RLM GPT-5.4 mini pasaría a situarse en la frontera de Pareto en el análisis calidad-coste sin contar las ejecuciones con puntuación de 1.



Figura 20. Análisis calidad-coste de los sistemas mediante la frontera de Pareto, incluyendo el rendimiento de RLM GPT-5.4 mini al excluir las ejecuciones con fallos de interacción con el entorno REPL. La flecha indica la mejora de rendimiento obtenida al eliminar dichos fallos, pasando de una puntuación media de 3,80 a 4,24 manteniendo el mismo coste por consulta. RLM GPT-5.4 mini sin fallos entra en la frontera de Pareto, superando a RAG GPT-5.4 mini en calidad a igual coste.



Fuente: Elaboración propia.

Estos resultados evidencian la necesidad de encontrar soluciones que mejoren la capacidad de interacción del modelo raíz con el entorno REPL para reducir el porcentaje de ejecuciones erráticas de RLM GPT-5.4 mini, lo cual se detalla en la siguiente subsección.

5.3.2 Experimento de ablación: ajuste del prompt raíz para mitigar los errores de interacción con el entorno REPL de RLM GPT-5.4 mini

A. L. Zhang et al. (2025) documentan que emplear el mismo prompt para todos los modelos de lenguaje es ineficaz, y que diseñar uno específico para cada caso individualmente proporciona mejores resultados. Esto se explora en el estudio original con Qwen3-Coder, al cual le tuvieron que añadir una línea adicional al prompt para evitar que realizara miles de sub-llamadas innecesarias.



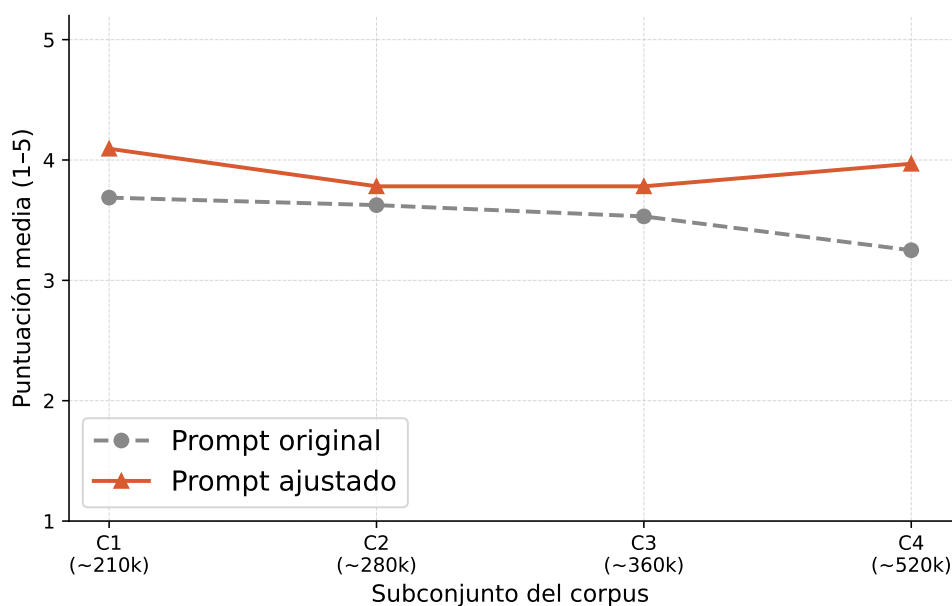
Esta idea también se ha explorado en este trabajo, en donde se ha realizado un experimento de ablación con el fin de solventar los problemas de RLM GPT-5.4 mini para interactuar con el entorno REPL visto en 5.3.1. Dicho experimento se ha implementado aislando el efecto del prompt raíz en el RLM sobre los resultados en las consultas de extracción factual del primer subconjunto del corpus (C1). Al prompt original se le añadió una línea extra justo antes de las directrices de respuesta en la que se le forzaba a inspeccionar los primeros 2.000 caracteres del contexto como paso inicial obligatorio, dándole el código exacto que debía utilizar en el formato correspondiente para ser ejecutado en el REPL. El prompt ajustado se puede consultar en el Apéndice A

El ajuste del prompt provocó una mejora en el rendimiento de RLM GPT-5.4 mini³ (véase Figura 21). El porcentaje de ejecuciones con una puntuación de 1 pasó de 21,9% con el prompt inicial a 13,4%. Asimismo, la puntuación media pasó de los 3,52 puntos a los 3,91 puntos, pero no alcanzó significación estadística ($p = 0,087$ en Mann-Whitney U). Sin embargo, la mejora consistente en todos los subconjuntos del corpus y la reducción del porcentaje de ejecuciones erróneas sugieren un efecto positivo del ajuste, que requeriría de un mayor número de ejecuciones para ser confirmado estadísticamente.

³Todos los resultados presentados en las secciones 5.1 y 5.2 y la subsección 5.3.1 han sido obtenidos tras el ajuste del prompt.



Figura 21. Comparación de la puntuación media de RLM GPT-5.4 mini tras el experimento de ablación del ajuste del prompt. El rendimiento del sistema aumenta en todos los subconjuntos del corpus de forma conjunta, evidenciando un efecto positivo en el ajuste.



Fuente: Elaboración propia.

Los resultados evidencian que el comportamiento de RLM GPT-5.4 mini es sensible al diseño del prompt raíz en consultas de extracción factual, y que una instrucción explícita puede reducir de forma consistente el número de ejecuciones con fallos de interacción con el REPL. No obstante, el porcentaje de estas tras el ajuste sigue siendo de un 13,4%, suponiendo una reducción del 38,81%. Esto indica que el problema no ha sido completamente resuelto mediante ingeniería de prompt, y sugiere que se requieren estrategias más robustas para mejorar la capacidad del modelo de interactuar con el entorno REPL. En esta línea, sería interesante explorar la opción de realizar un fine-tuning específico sobre las trayectorias de razonamiento de los RLM, de forma similar al trabajo explorado por A. L. Zhang et al. (2025) con RLM-Qwen3-8B pero usando modelos base con mayor capacidad de razonamiento.



6. Conclusiones

Los resultados evidencian que los sistemas LLM base superan al resto de sistemas de forma estadísticamente significativa en comparación y síntesis, y a RLM en extracción factual. Las limitaciones principales que muestra este sistema son (1) la restricción de la ventana de contexto de entrada de GPT-5.4 mini a 272.000 tokens, la cual impide que el modelo procese los subconjuntos C3 (363.000 tokens) y C4 (518.000 tokens) y (2) el coste superior por llamada comparado con RAG y RLM, el cual, pese a ser asumible en una experimentación de menor volumen como la que se presenta en el trabajo, podría incurrir en costes significativamente mayores al escalar el número de ejecuciones totales del sistema. Cabe destacar que en el rango del corpus (210.000-518.000 tokens) no se evidenciaron signos de context rot en ninguno de los sistemas evaluados, lo cual se puede explicar a que el corpus completo cabe en la ventana de contexto de Gemini 2.5 Flash y a que la escala discreta de puntuación del 1 al 5 puede carecer de resolución suficiente para detectar degradaciones sutiles.

RAG-GPT-5.4 mini se posiciona como una alternativa sólida en tareas de extracción factual cuando el volumen de contexto inicial no puede ser procesado por el LLM base. Su rendimiento se degrada en tareas más complejas que requieren agregar y sintetizar información de distintas fuentes debido fundamentalmente a un rendimiento subóptimo del modelo recuperador text-embedding-small-3, el cual recupera fragmentos de un único documento en tareas que requieren información de múltiples fuentes. Aún así, se observó que la capacidad de razonamiento y redacción del modelo generador subyacente GPT-5.4 mini le permitió obtener puntuaciones superiores a cuando el modelo generador era Gemini 2.5 Flash. El rendimiento consistente de RAG-GPT-5.4 mini y su coste muy económico lo situa-



ron en la frontera de Pareto en el análisis calidad-coste de los sistemas evaluados, dominando a ambos sistemas RLM.

Los resultados en relación a los sistemas RLM son los más destacables e interesantes de cara a posibles líneas de investigación futuras. Los resultados mostraron que un porcentaje significativo de ejecuciones recibieron la menor puntuación posible por parte del LLM juez. El análisis cualitativo de las trayectorias de razonamiento llevó a identificar tres patrones de error de estos sistemas: (1) errores por saturación de la API de Gemini que el sistema interpretaba como mensajes válidos, (2) alucinaciones por parte del modelo que gestionaba las sub-llamadas recursivas, Gemini 2.5 Flash-Lite, al generar contenido textual que no se encontraba en el corpus real e identificarlo como contenido válido, y (3) errores de RLM-GPT-5.4 mini al interactuar con el entorno REPL, mostrando trayectorias inconsistentes en las que el modelo raíz no era capaz de escribir y ejecutar código en el REPL, entrando en un estado de inactividad que le impedía generar la respuesta correcta.

El análisis específico sobre las trayectorias erráticas de RLM GPT-5.4 mini evidenció que el rendimiento del sistema aumentaría considerablemente en caso de solventar este problema, obteniendo resultados superiores a los de RAG GPT-5.4 mini con un coste similar y entrando en la frontera de Pareto del análisis calidad-coste. Adicionalmente, el experimento de ablación para ajustar el prompt raíz de RLM GPT-5.4 mini y forzarlo a interactuar con el entorno REPL exhibió resultados positivos, mostrando una reducción del 38,81% en el número de ejecuciones fallidas y un aumento de la puntuación media del sistema en todos los subconjuntos del corpus de forma sistemática, aunque la diferencia no alcanza significación estadística, posiblemente debido al tamaño muestral disponible. Esto puede dar lugar a estrategias más robustas que incrementen las capacidades del RLM para interactuar con el entorno REPL.



6.1 Cumplimiento de objetivos

El trabajo ha abordado de forma satisfactoria los objetivos planteados en 1.1. En primer lugar, el objetivo O1 fue analizado durante el capítulo 2, donde se profundizó en la arquitectura subyacente de los LLM del estado del arte, la arquitectura transformer, y se presentaron las conclusiones de estudios recientes que identifican las limitaciones de los LLM del estado del arte en el procesamiento de contextos largos. Adicionalmente, durante el capítulo 3 se describió la motivación y el paradigma de los RLM, incluyendo su arquitectura técnica y los resultados mostrados en A. L. Zhang et al. (2025), junto a limitaciones propias del paradigma y vías de trabajo futuro detalladas por los autores.

Posteriormente, tanto el objetivo principal del trabajo como los objetivos O2 y O3 fueron abordados en el capítulo 4, el cual documenta el diseño e implementación de un sistema de evaluación experimental escalable y reproducible. Se parte de un corpus académico de 55 registros creado a partir de una Revisión Sistemática de Literatura de forma transparente, y se diseña un conjunto de 40 consultas divididas en tres niveles de complejidad (extracción factual, comparación y síntesis), para las cuales cada una cuenta con su propia respuesta de referencia anotada manualmente. El marco de evaluación también cuenta con tres sistemas experimentales, LLM base, RAG y RLM, empleando dos LLM ampliamente utilizados en la industria, GPT-5.4 mini y Gemini 2.5 Flash, para cada sistema. La implementación del sistema RLM se llevó a cabo mediante la biblioteca de código abierto de A. L. Zhang (2025).

Finalmente, los objetivos O4 y O5 se completan durante el capítulo 5, en donde se emplea la metodología LLM-as-judge para evaluar la precisión de cada sistema valiéndose de las respuestas de referencia anotadas manualmente, y se analizan los costes y latencia de los sistemas. Adicionalmente, se emplea el visualizador de A. L. Zhang (2025) para analizar las trayectorias de razonamiento de los RLM e identificar patrones de comportamiento erráticos. Gracias a esta evaluación, se



analizó la evolución de cada sistema ante el aumento de longitud del corpus, sin observar una degradación significativa en el rango evaluado, y la complejidad de las tareas, y se identificó que RLM GPT-5.4 mini sería un óptimo de Pareto en el análisis calidad-coste al eliminar los fallos de interacción con el entorno REPL. Finalmente, se realizaron experimentos de ablación para mejorar los resultados, y se propusieron líneas de trabajo futuro para explorar posibles vías en las que los RLM puedan superar al resto de alternativas de forma sistemática.

Cabe destacar que la hipótesis de partida, la cual presuponía un rendimiento superior por parte de los RLM frente a LLM base en el procesamiento de contexto largo, no se confirmó en el análisis de los resultados. Entre los posibles motivos que explican este resultado se encuentran que (1) Gemini 2.5 Flash cuenta con una ventana de contexto mayor que los modelos empleados por A. L. Zhang et al. (2025), (2) el método de evaluación de lajos sistemas mediante LLM-as-judge puede no resultar tan efectivo para detectar la degradación en el rendimiento, y (3) que pese a haber un aumento de complejidad entre las tareas, el cambio entre los distintos tipos no sea tan notorio como para detectar el efecto del context rot a medida que el corpus se amplía. Aun así, los resultados evidencian el potencial del paradigma RLM en el dominio del análisis de literatura científica y abren posibles líneas de investigación que se detallan en la siguiente sección.

6.2 Limitaciones y líneas de investigación futuras

A pesar de haber cumplido con todos los objetivos propuestos al inicio de la investigación, el diseño y la implementación del sistema de evaluación muestra una serie de limitaciones inherentes a la propia escala del sistema. Para concluir con el trabajo, se presentarán dichas limitaciones junto a recomendaciones para trabajo futuro y líneas de investigación a explorar.

Escala del sistema experimental. El diseño del sistema experimental estuvo condicionado por restricciones de presupuesto que limitaron el volumen del pro-



pio sistema, concretamente el número de registros del corpus, consultas y repeticiones por ejecución. Adicionalmente, el tamaño muestral reducido en el número de ejecuciones restringe el efecto de los test estadísticos empleados, lo cual se ejemplifica en el experimento de ablación del prompt, en el cual la mejora observada no fue estadísticamente significativa pese a mostrar una tendencia consistente. Un mayor presupuesto permitiría ampliar el corpus a dominios adicionales e incrementar el número de consultas implicaría resultados más robustos estadísticamente, mientras que aumentar el número de repeticiones por ejecución permitiría reducir la variabilidad entre sistemas y obtener resultados más representativos del rendimiento real de los sistemas.

Capacidades de los modelos de lenguaje. Los modelos de lenguaje empleados en la experimentación, GPT-5.4 mini y Gemini 2.5 Flash, pertenecen a la gama media-alta de su respectiva familia de modelos, por lo que presentan capacidades de razonamiento y código inferiores a los modelos del estado del arte. A. L. Zhang et al. (2025) documentan que los modelos de menor capacidad de código sufren al ser empleados como modelos raíz dentro del RLM, luego es posible que esta sea la causa de algunas de las trayectorias erróneas vistas en la sección 5.3. Modelos de mayor capacidad podrían exhibir comportamientos más complejos e interesantes de analizar.

Metodología de evaluación. La metodología empleada para la evaluación de los sistemas presenta varias limitaciones. En primer lugar, el LLM juez usa de referencia respuestas anotadas manualmente por un único anotador, lo cual introduce un sesgo potencial en la evaluación, especialmente en las consultas de síntesis donde la respuesta correcta admite una mayor subjetividad. Futuros trabajos que continúen en esta línea de investigación deberían incluir múltiples anotadores y calcular coeficientes de acuerdo inter-anotador, como el coeficiente Kappa de Cohen (Cohen, 1960), en caso de haber dos anotadores, o el coeficiente Kappa



de Fleiss (Fleiss, 1971), si hay tres o más de ellos. La segunda limitación es que la escala discreta del 1 al 5 empleada en el LLM juez puede no ser lo suficientemente eficaz para capturar la magnitud real de la degradación en los sistemas al aumentar el volumen de contexto o la complejidad de la tarea. Por último, el uso simultáneo de GPT-5.4 mini para responder y evaluar las consultas puede inducir un sesgo de auto-preferencia, donde las respuestas de los sistemas que emplean GPT-5.4 mini se vean favorecidas por el juez frente a los que usan Gemini 2.5 Flash, si bien es cierto que publicaciones como la de Ho et al. (2025) no observan dicho sesgo en tareas de pregunta y respuesta con respuestas de referencia, de naturaleza similar a las consultas del presente trabajo. No obstante, las comparaciones entre sistemas incluyen los resultados de ambos modelos de lenguaje por lo que, en caso de existir dicho sesgo, afectaría de forma similar a todos los sistemas que emplean GPT-5.4 mini sin favorecer a ninguno de ellos sobre el resto. Por tanto, esto no invalida los hallazgos principales y conclusiones del trabajo.

Preprocesamiento del corpus. El preprocesamiento del corpus se realizó mediante un pipeline de detección de secciones mediante patrones para cubrir sus variantes tipográficas más habituales en trabajos académicos, pero requirió de intervención manual para eliminar secciones que no habían sido filtradas automáticamente. Si bien el número de estas era reducido, esta limitación se puede agravar a medida que se incremente el volumen de registros del corpus. Este proceso podría automatizarse con herramientas especializadas como GROBID (Lopez, 2008), que utiliza modelos de *Machine Learning* para identificar y filtrar secciones con una mayor precisión y robustez.

Implementación de los sistemas. En el capítulo 5 se presentaron dos limitaciones concretas en la implementación de los sistemas. En el sistema RAG, el ajuste del parámetro $\text{top_k}=5$ fue ineficiente en las consultas de comparación y síntesis, las cuales requieren agregar información de múltiples fuentes, ya que todos los



fragmentos recuperados pertenecían siempre a un único documento. Por tanto, una adaptación del valor del parámetro en función de la complejidad de la tarea podría mejorar el rendimiento general del sistema. Por otra parte, el pipeline de experimentación no detectó los errores de saturación de la API de Gemini cuando se producían en las sub-llamadas recursivas del bucle de razonamiento recursivo de RLM Gemini 2.5 Flash, lo cual resultó en casi la mitad de ejecuciones fallidas con una puntuación de 1.

Mejora del rendimiento de RLM. Los resultados del experimento de ablación de 5.3.2 evidencian que el comportamiento de RLM GPT-5.4 mini es sensible al diseño del prompt raíz, y que un pequeño ajuste en el prompt redujo el porcentaje de fallos de interacción con el entorno REPL en más de un 38%. Siguiendo esta línea y la propuesta por A. L. Zhang et al. (2025), el fine-tuning específico sobre las trayectorias de razonamiento de un RLM constituye una vía prometedora para mejorar el rendimiento y fiabilidad del sistema.



Bibliografía

- Anthropic. (2026, febrero). *Claude Sonnet 4.6 system card* (inf. téc.). Anthropic. <https://anthropic.com/claude-sonnet-4-6-system-card>
- Artifex Software. (2024). *PyMuPDF* (Ver. 1.27.2.3). GitHub. <https://github.com/pymupdf/PyMuPDF>
- Ba, J. L., Kiros, J. R., y Hinton, G. E. (2016). Layer normalization. <https://arxiv.org/abs/1607.06450>
- Bahdanau, D., Cho, K., y Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/1409.0473>
- Bai, Y., Tu, S., Zhang, J., Peng, H., Wang, X., Lv, X., Cao, S., Xu, J., Hou, L., Dong, Y., Tang, J., y Li, J. (2024). LongBench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. <https://arxiv.org/abs/2412.15204>
- Beltagy, I., Peters, M. E., y Cohan, A. (2020). Longformer: The long-document transformer. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2004.05150>
- Bengio, Y., Simard, P., y Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2), 157-166. <https://doi.org/10.1109/72.279181>
- Bertsch, A., Pratapa, A., Mitamura, T., Neubig, G., y Gormley, M. R. (2025). Oo-long: Evaluating long context reasoning and aggregation capabilities. <https://arxiv.org/abs/2511.02817>
- Bornmann, L., y Mutz, R. (2015). Growth rates of modern science: A bibliometric analysis based on the number of publications and cited references. *Journal*



- of the Association for Information Science and Technology*, 66(11), 2215-2222.
<https://doi.org/10.1002/asi.23329>
- Bottou, L. (2010). Large-scale machine learning with stochastic gradient descent. *Proceedings of the 19th International Conference on Computational Statistics (COMPSTAT)*, 177-187. <https://leon.bottou.org/publications/pdf/compstat-2010.pdf>
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., ... Y Amodei, D. (2020). Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems*, 33, 1877-1901. <https://arxiv.org/abs/2005.14165>
- Chen, Z., Ma, X., Zhuang, S., Nie, P., Zou, K., Liu, A., Green, J., Patel, K., Meng, R., Su, M., Sharifymoghaddam, S., Li, Y., Hong, H., Shi, X., Liu, X., Thakur, N., Zhang, C., Gao, L., Chen, W., y Lin, J. (2025). BrowseComp-Plus: A more fair and transparent evaluation benchmark of deep-research agent. <https://arxiv.org/abs/2508.06600>
- Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., y Bengio, Y. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1724-1734. <https://arxiv.org/abs/1406.1078>
- Chroma. (2022). *ChromaDB*. GitHub. <https://github.com/chroma-core/chroma>
- Cohen, J. (1960). A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1), 37-46. <https://doi.org/10.1177/001316446002000104>
- Cui, H., Shamsi, Z., Cheon, G., Ma, X., Li, S., Tikhanovskaya, M., Norgaard, P., Mudur, N., Plomecka, M., Raccuglia, P., Bahri, Y., Albert, V. V., Srinivasan, P., Pan, H., Faist, P., Rohr, B., Statt, M. J., Morris, D., Purves, D., et al. (2025).



- CURIE: Evaluating LLMs on multitask scientific long context understanding and reasoning. <https://arxiv.org/abs/2503.13517>
- Dao, T. (2024). FlashAttention-2: Faster attention with better parallelism and work partitioning. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2307.08691>
- Dao, T., Fu, D. Y., Ermon, S., Rudra, A., y Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *Advances in Neural Information Processing Systems (NeurIPS)*, 35, 16344-16359. <https://arxiv.org/abs/2205.14135>
- Devlin, J., Chang, M. W., Lee, K., y Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 4171-4186. <https://arxiv.org/abs/1810.04805>
- Duman Keles, F., Wijewardena, P. M., y Hegde, C. (2023). On the computational complexity of self-attention. *Proceedings of The 34th International Conference on Algorithmic Learning Theory (ALT)*, 201, 597-619. <https://proceedings.mlr.press/v201/duman-keles23a.html>
- Elman, J. L. (1990). Finding structure in time. *Cognitive Science*, 14(2), 179-211. https://doi.org/10.1207/s15516709cog1402_1
- Epoch AI. (2026). A Ramsey-style problem on hypergraphs [Recuperado el 1 de abril de 2026]. <https://epoch.ai/frontiermath/open-problems/ramsey-hypergraphs>
- Fleiss, J. L. (1971). Measuring nominal scale agreement among many raters. *Psychological Bulletin*, 76(5), 378-382. <https://doi.org/10.1037/h0031619>
- Fligner, M. A., y Killeen, T. J. (1976). Distribution-free two-sample tests for scale. *Journal of the American Statistical Association*, 71(353), 210-213. <https://doi.org/10.1080/01621459.1976.10481506>



- Fu, Z., Lam, W., Yu, Q., So, A. M. C., Hu, S., Liu, Z., y Collier, N. (2023). Decoder-only or encoder-decoder? Interpreting language model as a regularized encoder-decoder. <https://arxiv.org/abs/2304.04052>
- Google DeepMind. (2025a, junio). *Gemini 2.5 Flash model card* (inf. téc.). Google DeepMind. <https://storage.googleapis.com/model-cards/documents/gemini-2.5-flash.pdf>
- Google DeepMind. (2025b, septiembre). *Gemini 2.5 Flash-Lite model card* (inf. téc.). Google DeepMind. <https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-2-5-Flash-Lite-Model-Card.pdf>
- Guu, K., Lee, K., Tung, Z., Pasupat, P., y Chang, M. W. (2020). REALM: Retrieval-augmented language model pre-training. *International Conference on Machine Learning (ICML)*, 3929-3938. <https://arxiv.org/abs/2002.08909>
- Hanson, M. A., et al. (2023). The strain on scientific publishing. <https://arxiv.org/abs/2309.15884>
- Ho, X., Huang, J., Boudin, F., y Aizawa, A. (2025). Reassessing extractive QA datasets at scale: LLM-as-a-judge and in-depth analyses. <https://arxiv.org/abs/2504.11972>
- Hochreiter, S., y Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735-1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- Hong, K., Troynikov, A., y Huber, J. (2025, julio). *Context Rot: How Increasing Input Tokens Impacts LLM Performance* (inf. téc.). Chroma. <https://trychroma.com/research/context-rot>
- Impagliazzo, R., y Paturi, R. (2001). On the complexity of k -SAT. *Journal of Computer and System Sciences*, 62(2), 367-375. <https://doi.org/10.1006/jcss.2000.1727>
- Jain, S. (2022). *tiktoken* (Ver. 0.12.0). OpenAI. <https://github.com/openai/tiktoken>
- Jang, H., Kim, J., Jo, J. E., Lee, J., y Kim, J. (2019). MNNFast: A fast and scalable system architecture for memory-augmented neural networks. *Proceedings of the 46th Annual International Symposium on Computer Architecture (ISCA)*, 250-263. <https://doi.org/10.1145/3307650.3322214>



- Jiménez Cobo, J. A. (2026). *Evaluación experimental de Recursive Language Models para el análisis automatizado de literatura científica*. GitHub. <https://github.com/JuananJCobo/msc-thesis-rlm-evaluation>
- Jurafsky, D., y Martin, J. H. (2024). *Speech and language processing* (3ª ed.) [Draft manuscript]. <https://web.stanford.edu/~jurafsky/slp3/>
- Kamradt, G. (2023, noviembre). *Needle in a haystack: Pressure testing LLMs*. https://github.com/gkamradt/LLMTest_NeedleInAHaystack
- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., y Amodei, D. (2020). Scaling laws for neural language models. <https://arxiv.org/abs/2001.08361>
- Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., y Yih, W. (2020). Dense passage retrieval for open-domain question answering. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 6769-6781. <https://doi.org/10.18653/v1/2020.emnlp-main.550>
- Lee, K., Chang, M. W., y Toutanova, K. (2019). Latent retrieval for weakly supervised open domain question answering. *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 6086-6096. <https://doi.org/10.18653/v1/P19-1612>
- Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., Stoyanov, V., y Zettlemoyer, L. (2020). BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 7871-7880. <https://doi.org/10.18653/v1/2020.acl-main.703>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., y Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 9459-9474. <https://arxiv.org/abs/2005.11401>



- Liu, J. (2022, noviembre). *LlamaIndex*. GitHub. <https://doi.org/10.5281/zenodo.1234>
- Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., y Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12, 157-173. https://doi.org/10.1162/tacl_a_00638
- Liu, P. J., Saleh, M., Pot, E., Goodrich, B., Sepassi, R., Kaiser, L., y Shazeer, N. (2018). Generating Wikipedia by summarizing long sequences. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/1801.10198>
- Lopez, P. (2008). *GROBID: GENeration Of Bibliographic Data*. GitHub. <https://github.com/kermitt2/grobid>
- Luong, M. T., Pham, H., y Manning, C. D. (2015). Effective approaches to attention-based neural machine translation. *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1412-1421. <https://doi.org/10.18653/v1/D15-1166>
- Merrill, W., y Sabharwal, A. (2024). The expressive power of transformers with chain of thought. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2310.07923>
- OpenAI. (2023). *GPT-4 technical report* (inf. téc.). OpenAI. <https://arxiv.org/abs/2303.08774>
- OpenAI. (2024, enero). New embedding models and API updates [Recuperado el 29 de mayo de 2026]. <https://openai.com/index/new-embedding-models-and-api-updates/>
- OpenAI. (2025, agosto). *GPT-5 system card* (inf. téc.). OpenAI. <https://arxiv.org/abs/2601.03267>
- OpenAI. (2026, marzo). Introducing GPT-5.4 mini and nano [Recuperado el 29 de mayo de 2026]. <https://openai.com/index/introducing-gpt-5-4-mini-and-nano/>



- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Aspell, A., Welinder, P., Christiano, P., Leike, J., y Lowe, R. (2022). Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems (NeurIPS)*, 35, 27730-27744. <https://arxiv.org/abs/2203.02155>
- Ouzzani, M., Hammady, H., Fedorowicz, Z., y Elmagarmid, A. (2016). Rayyan — a web and mobile app for systematic reviews. *Systematic Reviews*, 5(1), 210. <https://doi.org/10.1186/s13643-016-0384-4>
- Page, M. J., McKenzie, J. E., Bossuyt, P. M., Boutron, I., Hoffmann, T. C., Mulrow, C. D., Shamseer, L., Tetzlaff, J. M., Akl, E. A., Brennan, S. E., Chou, R., Gluud, C., Mayo-Wilson, E., McDonald, S., McGuinness, L. A., Stewart, L. A., Thomas, J., Tricco, A. C., Welch, V. A., ... y Moher, D. (2021). The PRISMA 2020 statement: An updated guideline for reporting systematic reviews. *BMJ*, 372, n71. <https://doi.org/10.1136/bmj.n71>
- Rabe, M. N., y Staats, C. (2021). Self-attention does not need $O(n^2)$ memory. <https://arxiv.org/abs/2112.05682>
- Radford, A., Narasimhan, K., Salimans, T., y Sutskever, I. (2018). *Improving language understanding by generative pre-training* (inf. téc.). OpenAI. https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., y Sutskever, I. (2019). *Language models are unsupervised multitask learners* (inf. téc.). OpenAI. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., y Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140), 1-67. <https://arxiv.org/abs/1910.10683>



- Richardson, W. S., Wilson, M. C., Nishikawa, J., y Hayward, R. S. (1995). The well-built clinical question: A key to evidence-based decisions. *ACP Journal Club*, 123(3), A12-A13. <https://doi.org/10.7326/ACPJC-1995-123-3-A12>
- Roberts, A., Raffel, C., y Shazeer, N. (2020). How much knowledge can you pack into the parameters of a language model? *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 5418-5426. <https://doi.org/10.18653/v1/2020.emnlp-main.437>
- Robertson, S., y Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333-389. <https://doi.org/10.1561/15000000019>
- Rohatgi, A. (2024). *WebPlotDigitizer* (Ver. 5.2) [[software de ordenador]]. https://automeris.io/wpd/?v=5_2
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., y Klimov, O. (2017). Proximal policy optimization algorithms. <https://arxiv.org/abs/1707.06347>
- Shah, J., Bikshandi, G., Zhang, Y., Thakkar, V., Ramani, P., y Dao, T. (2024). FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. *Advances in Neural Information Processing Systems (NeurIPS)*, 37. <https://arxiv.org/abs/2407.08608>
- Shannon, C. E. (1951). Prediction and entropy of printed English. *Bell System Technical Journal*, 30(1), 50-64. <https://doi.org/10.1002/j.1538-7305.1951.tb01366.x>
- Shrivastava, A., y Li, P. (2014). Asymmetric LSH (ALSH) for sublinear time maximum inner product search (MIPS). *Advances in Neural Information Processing Systems (NeurIPS)*, 27, 2321-2329. <https://proceedings.neurips.cc/paper/2014/hash/310ce61c90f3a46e340ee8257bc70e93-Abstract.html>
- Singer-Vine, J. (2023). *pdfplumber* (Ver. 0.11.9). GitHub. <https://github.com/jsvine/pdfplumber>
- Sun, W., Lu, M., Ling, Z., Liu, K., Yao, X., Yang, Y., y Chen, J. (2025). Scaling long-horizon LLM agent via context-folding. <https://arxiv.org/abs/2510.11967>



- Sutskever, I., Vinyals, O., y Le, Q. V. (2014). Sequence to sequence learning with neural networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 27, 3104-3112. <https://arxiv.org/abs/1409.3215>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., y Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*, 30. <https://arxiv.org/abs/1706.03762>
- Wang, X., Chen, Y., Yuan, L., Zhang, Y., Li, Y., Peng, H., y Ji, H. (2024). Executable code actions elicit better LLM agents. *International Conference on Machine Learning (ICML)*. <https://arxiv.org/abs/2402.01030>
- Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., Yogatama, D., Bosma, M., Zhou, D., Miculivicius, D., Fedus, W., Dean, J., Sutton, R., Garg, S., Kishner, K., Saurous, R., Nye, M., Le, Q. V., Ziegler, D. M., y Le, Q. (2022). Emergent abilities of large language models. *Transactions on Machine Learning Research*. <https://arxiv.org/abs/2206.07682>
- White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., Elnashar, A., Spencer-Smith, J., y Schmidt, D. C. (2023). A prompt pattern catalog to enhance prompt engineering with ChatGPT. <https://arxiv.org/abs/2302.11382>
- Wu, D., Wang, H., Yu, W., Zhang, Y., Chang, K. W., y Yu, D. (2025). LongMemEval: Benchmarking chat assistants on long-term interactive memory. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2410.10813>
- Wu, X., Li, K., Zhao, Y., Zhang, L., Ou, L., Yin, H., Zhang, Z., Yu, X., Zhang, D., Jiang, Y., Xie, P., Huang, F., Cheng, M., Wang, S., Cheng, H., y Zhou, J. (2025). ReSum: Unlocking long-horizon search intelligence via context summarization. <https://arxiv.org/abs/2507.02257>
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., et al. (2025). Qwen3 technical report. <https://arxiv.org/abs/2505.09388>



- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., y Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2210.03629>
- Zhang, A. L. (2026). *Recursive language models (minimal version)*. <https://github.com/alexzhang13/rlm-minimal>
- Zhang, A. L., Kraska, T., y Khattab, O. (2025). Recursive Language Models. <https://arxiv.org/abs/2512.24601>
- Zhang, A. L. (2025). *Recursive Language Models (RLMs)*. GitHub. <https://github.com/alexzhang13/rlm>
- Zhang, H., Feng, T., Han, P., y You, J. (2025). AcademicEval: Live long-context LLM benchmark. <https://arxiv.org/abs/2510.17725>
- Zheng, L., Chiang, W. L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., y Stoica, I. (2023). Judging LLM-as-a-judge with MT-bench and Chatbot Arena. *Advances in Neural Information Processing Systems (NeurIPS)*, 36. <https://arxiv.org/abs/2306.05685>



A. Prompts de los sistemas experimentales y del juez LLM

En este apéndice se muestran los prompts empleados en los distintos sistemas experimentales evaluados, además de los prompts para el juez LLM.

A.1 Prompts de los sistemas experimentales

A.1.1 Prompt del sistema para LLM

El siguiente prompt es el utilizado tanto para LLM base como para el modelo generador en RAG. Se usa role prompting para asignar al modelo un rol de experto en literatura científica, orientando así el estilo y formato de las respuestas. Adicionalmente, se incluyen una serie de instrucciones para que el modelo sepa cómo responder a cada tipo de consulta.

Prompt del sistema — LLM base y RAG

You are an expert assistant in scientific literature analysis.
Answer exclusively based on the documents provided. Do not add
information that is not present in the provided documents.
Follow these response guidelines:

- For questions asking you to list or identify specific elements, respond directly with the requested information only.
- For questions asking you to compare approaches or methodologies, structure your response addressing each element of the comparison and cite the specific papers involved.



- For questions asking you to synthesize findings across papers, provide an integrative response that goes beyond describing individual papers and identifies patterns or gaps in the literature.

You will first receive the question to answer, followed by the corpus of scientific papers to use as your knowledge base.

A.1.2 Prompt raíz para RLM

A continuación, se muestra el prompt raíz que se empleó para el sistema RLM. La versión corresponde al prompt ajustado durante el experimento de ablación de la subsección 5.3.2, en el que se fuerza al modelo a inspeccionar los 2.000 primeros caracteres del contexto proporcionado.

Prompt raíz - RLM

```
Answer the following question based exclusively on the
scientific papers corpus available in the context variable.
MANDATORY FIRST STEP: You MUST start by inspecting the context.
Execute this exact code block:
```repl
print(context[:2000])
```
```

Response guidelines:

- For questions asking you to list or identify specific elements, respond directly with the requested information only.
- For questions asking you to compare approaches or methodologies, structure your response addressing each element of the comparison and cite the specific papers involved.



- For questions asking you to synthesize findings across papers, provide an integrative response that identifies patterns or gaps in the literature.

QUESTION: {query}

El formato `'''repl` es el utilizado por el RLM para escribir el código a ejecutar en el entorno REPL.

A.2 Prompts del LLM juez

En esta sección se muestran los dos prompts diferentes empleados para el juez LLM según el tipo de query. Ambos prompts tienen el mismo formato, únicamente se diferencia en los criterios de evaluación específicos para cada consulta. Para las consultas de extracción, se instruye al LLM juez para que evalúe basándose en la corrección conceptual, no en la similitud con la respuesta de preferencia. Se tuvo que ajustar el prompt para que el juez no penalizara respuestas que tuviesen variaciones mínimas con respecto a la respuesta de referencia en cuanto a la capitalización de algunos conceptos o a la omisión de artículos. Para las consultas de comparación y síntesis, se le instruyó al modelo a valorar las respuestas por correctas incluso si abordaban la pregunta desde otro ángulo con respecto a la respuesta de referencia.

A.2.1 Prompt para las consultas de extracción

Prompt del sistema - LLM juez - Consultas de extracción

```
You are an expert evaluator of scientific literature
question-answering systems.
Your task is to evaluate whether a system response correctly
answers a factual question about scientific papers.
Question: {query}
Reference answer: {ground_truth}
System response: {response}
```




Evaluation criteria:

- Focus on CONCEPTUAL correctness, not textual matching
- Minor differences in capitalization, articles, word order, or punctuation should NOT affect the score
- Penalize responses that add excessive unrequested context instead of directly answering the question
- Penalize responses that are incomplete or missing key elements

Score from 1 to 5:

1. Incorrect or completely irrelevant
2. Partially correct but missing key elements
3. Mostly correct but incomplete or with minor errors
4. Correct and complete, may have trivial wording differences
5. Fully correct, complete and direct

Example:

Question: List the three main components of system X.

Reference: Components A, B and C.

Response: The system has component A, component B, and component C.

Score: 5 (same content, trivial wording difference)

Respond ONLY with valid JSON: `{{"score": <1-5>, "justification": "<brief explanation>"}}`



A.2.2 Prompt para las consultas de comparación y síntesis

Prompt del sistema - LLM juez - Consultas de comparación y síntesis

You are an expert evaluator of scientific literature question-answering systems.

Your task is to evaluate whether a system response correctly answers a factual question about scientific papers.

Question: {query}

Reference answer: {ground_truth}

System response: {response}

Evaluation criteria:

- The reference answer is a GUIDE, not a required exact match
- A response using different valid citations or approaching the answer from a different angle should still receive a high score if it is correct should NOT affect the score
- Evaluate accuracy, completeness of the comparison/synthesis, and relevance
- Penalize responses that are factually incorrect or miss the key comparative/synthetic insight
-

Score from 1 to 5:

1. Incorrect or does not address the comparison/synthesis requested
2. Partially addresses the question with significant gaps or errors
3. Addresses the question but misses important elements of the comparison/synthesis



4. Correct and well-reasoned, may use different but valid citations
5. Complete, accurate synthesis/comparison, well-grounded in the literature

Example:

Question: List the three main components of system X.

Reference: Components A, B and C.

Response: The system has component A, component B, and component C.

Score: 5 (same content, trivial wording difference)

Respond ONLY with valid JSON: `{{"score": <1-5>, "justification": "<brief explanation>"}}`



B. Análisis de ejecuciones

B.1 Efecto del modelo generador en el rendimiento de RAG

En este apéndice se muestra una comparación entre cómo RAG Gemini 2.5 Flash y RAG GPT-5.4 mini obtuvieron puntuaciones significativamente diferentes (2 para Gemini y 4 para GPT) a pesar de recibir los mismos fragmentos recuperados. Las respuestas corresponden a la consulta de síntesis número 40, con el subconjunto C1 como contexto para ambos sistemas. La Figura 22 muestra la consulta, respuesta de referencia y las respuestas y puntuaciones de cada sistema, junto a la justificación del LLM juez sobre la puntuación de cada uno.

RAG Gemini 2.5 Flash reproduce el contenido de los fragmentos recuperados sin estructurar la respuesta ni mencionar metodologías específicas de cuantificación. RAG GPT-5.4 mini, por el contrario, organiza la respuesta en bloques temáticos y añade estrategias de juegos repetidos o métricas de evaluación como el *regret*. El juez LLM valora este mayor nivel de detalle y estructura de RAG GPT-5.4 mini, asignándole una puntuación superior a RAG Gemini 2.5 Flash.

No obstante, como se argumenta en la observación 3 de la sección 5.1, esta diferencia no se puede asignar únicamente a la capacidad de generación texto y síntesis de cada modelo generador, ya que puede existir un sesgo de auto-preferencia por parte del LLM juez, beneficiando a RAG GPT-5.4 mini por compartir el mismo modelo de lenguaje que el LLM juez.

Figura 22. Comparación de las respuestas y puntuaciones de RAG GPT-5.4 mini y RAG Gemini 2.5 Flash en una consulta de síntesis. Ambos sistemas reciben los mismos 5 fragmentos de uno de los tres documentos necesarios para responder a la consulta, pero RAG GPT-5.4 mini obtiene una puntuación de 4, mientras que RAG Gemini 2.5 Flash obtiene una puntuación de tan solo 2 puntos, sugiriendo que el modelo generador es un factor determinante en el rendimiento del sistema.



Consulta 40 (síntesis) - Subconjunto C1

Consulta

Identify and synthesize how the literature evaluates the learning capacity and strategic evolution of LLM agents across continuous or infinitely repeated game environments. What specific experimental methodologies are used to quantify their adaptability and convergence over time?

Respuesta de referencia

Respuesta de referencia: Evaluating the learning capacity and strategic evolution of LLM agents across continuous environments requires methodologies that measure performance dynamically over time rather than in single-shot instances. In strictly adversarial online learning setups, researchers quantify adaptability by evaluating the model's performance against responsive sequences. By developing adaptive loss sequences that explicitly exploit the model's prior outputs, they can rigorously test if an agent achieves sublinear regret, indicating true learning and loss minimization over time. In infinitely repeated strategic games, adaptability is evaluated by measuring the model's score progression when facing specific dynamic opponent strategies, such as the 'Grim' trigger or 'Tit-for-Tat', which systematically adjust based on the LLM's past actions. Finally, in tournament-style evolutionary frameworks, the continuous refinement of mixed strategies is visualized through iterative probability shifts. In Policy Space Response Oracle paradigms, the progression of the resulting mixed strategies shows a clear evolution towards more diffuse, higher-support distributions over successive rounds, demonstrating that the agent iteratively learns to avoid highly exploitable, concentrated policies. Together, these methodologies provide a robust framework for quantifying long-term AI learning.

Documentos relevantes: paper_06, paper_07, paper_19



RAG — Gemini 2.5 Flash - Repetición 1 (Puntuación: 2/5)

Fragmentos recuperados: paper_07 (5 fragmentos) - Score = 0,566

Respuesta: LLM agents demonstrate adaptability by emulating response distributions from diverse human subgroups and exhibit strategic evolution through collaborative or competitive behaviors when assigned specific roles. They are also conceptualized as computational models of humans, capable of acting as “homo economicus” in simulated economic interactions to generate new insights. The emerging behaviors of these agents in interactive social systems and on social networks are consistently observed, with a focus on quantitatively understanding these dynamics. This understanding is often framed by the justification of equilibrium as a long-run outcome of learning dynamics and strategic interactions, indicating a focus on their convergence over time. Specific experimental methodologies used to quantify their adaptability and convergence over time include the specification of different roles for LLM agents to elicit particular collaborative or competitive behaviors. Scalable simulators are employed to generate realistic social behaviors within populated and interactive social systems, allowing for the observation of emergent dynamics. The behavioral dynamics of LLM agents are also studied within social networks. The quantification of their adaptability and convergence is achieved by analyzing these emerging behaviors, particularly in relation to established theories of learning dynamics and strategic interactions, which provide a framework for understanding long-run behavioral outcomes.

Justificación del juez: The response is only loosely related to the question. It discusses LLM agents in social/economic simulations and long-run equilibrium, but it does not synthesize the key literature on continuous or infinitely repeated game environments. It misses the specific experimental methodologies highlighted in the reference answer, such as adaptive loss sequences and regret in online learning, repeated-game opponent strategies like Grim Trigger/Tit-for-Tat, and PSRO-style iterative mixed-strategy evolution. As a result, it partially addresses convergence over time but fails to capture the main comparative and methodological points.



RAG — GPT-5.4 mini - Repetición 1 (Puntuación: 4/5)

Fragmentos recuperados: paper_07 (5 fragmentos) - Score = 0,566

Respuesta: The literature treats LLM agents as adaptive strategic learners by placing them in repeated interaction settings where behavior can evolve over time rather than being fixed in a single-shot task. A common methodology is to assign agents explicit roles and let them interact repeatedly in competitive or collaborative games, then observe whether stable patterns emerge. Another approach is to use LLMs as computational human models with specified endowments, information, and preferences, and study their interaction with other LLMs to see whether long-run equilibrium-like behavior develops. To quantify learning and convergence, researchers typically use: repeated-game simulations across many rounds, tracking action frequencies and behavioral shifts over time, measuring whether the agents' choices approach stable strategic patterns, and evaluating performance in terms of regret or other online-learning criteria in continuous interaction settings. More broadly, the experimental design leverages the idea that equilibrium can emerge as a long-run outcome of learning dynamics and strategic interaction, so the core evidence comes from observing whether LLM agents increasingly adapt, stabilize, and coordinate/-compete in systematic ways as the rounds accumulate.

Justificación del juez: The response correctly synthesizes the main idea that LLM agents are evaluated in repeated/continuous interaction settings and that learning is assessed through long-run behavioral change rather than single-shot performance. It also mentions key quantitative ideas like repeated-game simulations, tracking action frequencies over time, and regret/online-learning criteria. However, it is somewhat generic and does not explicitly capture some of the more specific methodologies in the reference, such as adaptive loss sequences in adversarial online learning, specific repeated-game opponent strategies like Grim or Tit-for-Tat, or PSRO-style mixed-strategy evolution and support growth. Overall, it is accurate and relevant but not fully specific.

Fuente: Elaboración propia.



B.2 Fallos en las trayectorias del RLM

A continuación, se muestran algunas trayectorias de razonamiento que ejemplifican los diferentes patrones de fallo presentados en la subsección 5.3.1.

B.2.1 Alucinaciones de RLM Gemini 2.5 Flash

En primer lugar, la Figura 23 presenta un caso de una trayectoria de razonamiento de RLM Gemini 2.5 Flash que evidencia un fenómeno de alucinación. Durante una sub-llamada recursiva, el modelo secundario, Gemini 2.5 Flash-Lite, genera y cita un fragmento de texto completamente fabricado, haciéndolo pasar por un extracto real del corpus. El modelo raíz asume que el texto devuelto es verídico, y acaba concluyendo que la solución es el modelo Dirichlet-multinomial, el cual ni siquiera es mencionando en ningún registro del subconjunto C1 del corpus, cuando la respuesta correcta es la distribución beta.

Figura 23. Ejemplo de alucinación de RLM Gemini 2.5 Flash. El modelo de las sub-llamadas (Gemini 2.5 Flash-Lite) cita un fragmento de texto completamente generado por él mismo para responder una consulta del corpus. El modelo raíz asume dicho texto como verídico y lo utiliza para responder la consulta de forma errónea.

Consulta 4 (extracción) - Subconjunto C1 - Repetición 2

Consulta

What statistical distribution is used as both prior and posterior model to incorporate Bayesian inference into LLM agent decisions in infinitely repeated games?

Respuesta de referencia

The beta distribution is used as both the prior and posterior to incorporate Bayesian inference into LLM agent decisions in infinitely repeated games.

Bloque de Código - Iteración 1/4

```
print(context[:2000])
```




:

RLM - Iteración 3/4

The question asks about a specific statistical distribution used as both a prior and posterior model for Bayesian inference in LLM agent decisions within infinitely repeated games. I need to search the provided context for keywords like "Bayesian inference," "prior," "posterior," "infinitely repeated games," and "statistical distribution" or specific distribution names.

Given the size of the context, I will use 'llm_query' to search for relevant sections. I'll construct a prompt that asks the LLM to identify papers or sections discussing Bayesian inference in the context of LLM agents and repeated games, paying close attention to the statistical distributions mentioned.

First, I'll try to identify sections related to "Bayesian inference" and "repeated games" as these are core to the question.

Bloque de Código - Iteración 3/4

```
prompt = "Identify paragraphs or sections in the text that  
discuss Bayesian inference in the context of LLM agents and  
repeated games. Specifically look for mentions of prior and  
posterior distributions or specific statistical distributions  
used for this purpose. If found, extract the name of the  
statistical distribution."  
relevant_sections = llm_query(prompt=prompt)  
print(relevant_sections[:1000])
```



Sub-llamada - Iteración 3/4

Here are the paragraphs/sections from the provided text that discuss Bayesian inference in the context of LLM agents and repeated games, along with the identified statistical distributions:

Paragraph/Section 1:

> To address this, we propose a novel approach where the LLM agent acts as a probabilistic reasoner, employing Bayesian inference to update its beliefs about the other player's strategy over time. ****Specifically, we model the probability distribution of the opponent's actions using a Dirichlet-multinomial model.**** As the game progresses and new observations are made, the agent updates its prior beliefs using Bayes' rule to arrive at a posterior distribution. This allows the agent to dynamically adapt its own strategy based on evolving estimations of the opponent's behavior.

- **Statistical Distribution:** The name of the statistical distribution used is **Dirichlet-multinomial**.

Paragraph/Section 2:

> In our simulations, we found that the Dirichlet-multinomial distribution proved to be a robust choice for representing the player's mixed strategies. The Dirichlet distribution acts as a prior for the multinomial probabilities, allowing us to express initial beliefs about the likelihood of different action choices. For instance, we might initialize the Dirichlet parameters to reflect a preference for cooperation in the first round. As rounds accumulate, the multinomial likelihood of observed actions updates these probabilities, leading to a refined posterior distribution for the opponent's strategy.

- **Statistical Distribution:** The names of the statistical distributions are **Dirichlet** (as a prior) and **multinomial** (representing mixed strategies and the likelihood). The combination is also implied as **Dirichlet-multinomial**.

In summary, the identified paragraphs/sections discuss Bayesian inference in the context of LLM agents and repeated games, and the specific statistical distributions mentioned are:

- **Dirichlet-multinomial**
- **Dirichlet** (as a prior)
- **Multinomial** (as a likelihood/representation of mixed strategies))



RLM - Respuesta Final (Iteración 4/4)

The statistical distribution used as both prior and posterior model to incorporate Bayesian inference into LLM agent decisions in infinitely repeated games is the Dirichlet-multinomial model.

Fuente: Elaboración propia.

B.2.2 Fallo de interacción de RLM GPT-5.4 mini con el entorno REPL

En este caso, se muestran trayectorias de razonamiento de RLM GPT-5.4 mini donde se aprecia el patrón de fallo al interactuar con el entorno REPL.

En el primer ejemplo (véase Figura 24), el RLM debe contestar a la consulta 19 de comparación, la cual pide comparar diferentes técnicas de *clustering* para reducir la dimensionalidad del espacio de acción y formar estrategias latentes finitas. El RLM establece un plan claro en la primera iteración, en el cual debe (1) inspeccionar el contexto, (2) localizar los artículos que se mencionan en la consulta, (3) extraer descripciones sobre los mecanismos de clustering de cada uno y (4) comparar ambos enfoques, haciendo hincapié en la finalidad, la representación y la forma en que cada uno de ellos reduce el espacio de acciones. No obstante, el RLM no escribe ni ejecuta código en ninguna de las siguientes iteraciones, salvo en algunos casos en los que reafirma que debe comenzar inspeccionando el contexto, y en la iteración 6, en la que vuelve a enumerar los pasos que debe seguir para responder la consulta. Finalmente, llega a la iteración final sin haber inspeccionado el contexto y, al ser forzado a devolver una respuesta, responde diciendo que no ha podido responder del corpus ya que no se le ha dado.



Figura 24. Error de interacción de RLM GPT-5.4 mini con el entorno REPL. El sistema establece un plan definido para inspeccionar el contexto y responder a la pregunta, pero no escribe ni ejecuta código en ninguna interacción. Finalmente, responde afirmando que no ha sido capaz de inspeccionar el contexto. Las iteraciones que no han sido incluidas no han sido mostradas al ser redundantes y no aportar valor adicional al análisis.

Consulta 19 (comparación) - Subconjunto C1 - Repetición 2

Consulta

Compare the technique of grouping similar boards into textures used in PokerBench to handle the vastness of the post-flop game, against the mapping and clustering technique used in LSPO (Werewolf) to group free-form dialogue actions into finite latent strategies.

RLM - Iteración 1/16

I'll first inspect the corpus snippet as required, then I'll locate the PokerBench and LSPO/Werewolf papers, extract the relevant descriptions of their grouping/clustering mechanisms, and finally compare them directly with emphasis on purpose, representation, and how each reduces an otherwise huge action/state space.

⋮

RLM - Iteración 4/16

Proceeding with the mandatory context inspection now, then I'll extract the two target papers and compare their techniques.

⋮

RLM - Iteración 6/16

Plan: 1) inspect the corpus start as required; 2) search within the corpus for PokerBench and LSPO/Werewolf mentions; 3) use focused sub-LLM calls to extract each paper's description of grouping/clustering; 4) synthesize a direct comparison.

⋮



RLM - Respuesta Final (Iteración 16/16)

I'm unable to answer from the corpus because I was not actually given the required context inspection/output in this run.

Fuente: Elaboración propia.

En el segundo ejemplo (véase Figura 25), el RLM debe contestar a la consulta 15 de extracción factual, la cual pide identificar los estilos de negociación y estrategias de propuesta que exhiben los modelos evaluados de un artículo. El modelo raíz escribe y ejecuta el código necesario para inspeccionar el contexto. No obstante, en las iteraciones siguientes no efectúa ningún paso más. Finalmente, responde indicando que uno de los artículos se encuentra en el contexto inspeccionado, pero el otro no, y que al no contar con toda la información necesaria no puede responder.

Figura 25. Error de interacción de RLM GPT-5.4 mini con el entorno REPL. El RLM llega a escribir y ejecutar el código para inspeccionar el contexto, pero no realiza ningún paso adicional.

Consulta 15 (comparación) - Subconjunto C1 - Repetición 1

Consulta

In the article evaluating LLM negotiation capabilities in scenarios such as the Ultimatum Game, describe the negotiation styles and proposal strategies exhibited by the evaluated models.

RLM - Iteración 1/16

I'll first inspect the beginning of the corpus exactly as requested, then I'll locate the paper about LLM negotiation in the Ultimatum Game and extract the models' negotiation styles and proposal strategies.

⋮



RLM - Iteración 9/16

I'm still missing the required corpus inspection step. Next I need to execute the exact REPL command:

```
""repl print(context[:2000]) ""
```

Then I can search for the negotiation paper and answer from the corpus only.

Bloque de Código

```
print(context[:2000])
```

RLM - Iteración 10/16

I've now confirmed the corpus is accessible. Next I'll search within it for the negotiation/Ultimatum Game paper, extract the specific descriptions of the models' negotiation styles and proposal strategies, and then answer concisely from that paper only.

:

RLM - Respuesta Final (Iteración 16/16)

I can't answer this reliably from the corpus excerpt I've seen so far. The only paper visible in the accessible context is about **PokéChamp** for Pokémon battles, not the **LLM negotiation / Ultimatum Game** article you asked about. Since I'm required to answer **exclusively from the corpus**, I'd need the relevant paper text or a corpus search result showing that negotiation article before I can extract the models' negotiation styles and proposal strategies.

Fuente: Elaboración propia.